Bowne Management Systems, Inc.
235 East Jericho Turnpike
Mineola NY 11501
Phone: 516-746-2350
Fax: 516-747-1396
www.bownegroup.com

City-Wide Street Centerline (CSCL)
Phase II

Extraction of LION and Geosupport File

May 19, 2010
Version: 7.0

Revised on Feb09, 2024

DoITT - New York City
Emergency Communications Transformation Project

Document Record

Date	Author(s)	Version	Document Reference
2/20/2009	Richard Steinberg	1.0	Initial draft completed
2/26/2009	Donna Funaro	1.0	Draft delivered to City
3/13/09, 3/19/09	DCP and DoITT	1.0	Provided comments to V1
4/21/2009	Richard Steinberg	2.0	Revised in response to DoITT and DCP comments and additional DCP requirements
4/21/2009	Donna Funaro	2.0	V2 delivered to City
4/28/2009, 4/30/2009	DCP	2.0	Provided comments to V2
5/20/2009	Richard Steinberg	3.0	Revised in response to V2 DCP comments and newly provided requirements, including file naming requirements
4/29/2009	DCP	3.0	Provided comments to V3
6/4/2009	Richard Steinberg	4.0	Revised in response to V3 DCP comments, new requirement for '$' value for shoreline level codes, and new fields in AltSegmentData for twisted parity flag and LGC5 through 9
6/18/2009	DCP	4.0	Provided ETL File Discrepancies report and updated layouts for ThinLION, ThinTract, SAF and SND
7/06/2009	Richard Steinberg	5.0	Revised in response to DCP Discrepancies report, new file layouts provided by DCP, a DCP directive to exclude ThinTract from extraction, and a requirement that MapInfo LION spatial data be expressed in State Plane Coordinates
01/12/2010	Richard Steinberg	6.0	Revised to reflect program modifications to resolve discrepancies found during Bowne QC and DCP QA
5/19/2010	Richard Steinberg	7.0	Revised to reflect further program modifications based on DCP QA
11/14/2023	Jie Ding	8.0	Add all the ETL changes since 2010

TABLE OF CONTENTS

1. Introduction 1
   - 1.1. User Interface 2
   - 1.2. Summary of Output Files 8
   - 1.3. Field Identification Codes (FICs) 9
   - 1.4. Field Formatting 10

2. LION 13
   - 2.1. Geosupport LION Output Record Layout 13
   - 2.2. Generating Records from Centerline Segments 18
     - 2.2.1. Street Code and Face Code Items 20
     - 2.2.2. Node-Related Items 21
     - 2.2.3. Atomic Polygon-Based Items 22
     - 2.2.4. Split Election District Flag 23
     - 2.2.5. Traffic Direction 24
     - 2.2.6. Segment Locational Status 25
     - 2.2.7. Feature Type Code 26
     - 2.2.8. Borough Boundary Indicator 27
     - 2.2.9. Twisted Parity Flag 27
     - 2.2.10. Special Address Flag 28
     - 2.2.11. Curve-Related Fields 29
     - 2.2.12. Level Codes 30
   - 2.3. Generating Records from Shoreline Segments 30
     - 2.3.1. Borough 32
     - 2.3.2. Segment Locational Status 33
     - 2.3.3. Curve Flag 33
     - 2.3.4. Zip Codes 33
   - 2.4. Generating Records from Rail and Subway Segments 34
   - 2.5. Generating Records from NonStreetFeature Segments 35
     - 2.5.1. Sequence Number 37
     - 2.5.2. Feature Type Code 38
     - 2.5.3. Level Codes 39
   - 2.6. Coincident Segments 39
     - 2.6.1. Determining the Coincident Segment Count 43
     - 2.6.2. Generating Output Records from Proto-Segments 45
     - 2.6.3. Face Code 48
     - 2.6.4. Fields Affected by Topological Direction 48
     - 2.6.5. Traffic Direction 50
     - 2.6.6. Level Codes 52
     - 2.6.7. Segment Locational Status 52
   - 2.7. ESRI LION 53
     - 2.7.1. Segment Replication for Continuous Parity Condition 54
     - 2.7.2. Segment Replication for SAF Data 55
     - 2.7.3. Methodologies for Populating Output Fields 58
     - 2.7.4. Generating the AltNames Table 64
   - 2.8. MapInfo LION 65

3. Street Name Dictionary 67

4. Geosupport System Normalizing Tables 72
   - 4.1. Exception Table 72
   - 4.2. Universal Word Table and Last Word Table 73

4.3. Enders Table 75

5. Face Code File 76

6. District Equivalency Files 77
6.1. ThinLION 77
6.2. ThinFire 80

7. Special Address File 82
7.1. CSCL Source Components for SAF Data 84
7.1.1. Use of the Roadbed Pointer List 84
7.1.2. Normalized Names in SAF Records 85
7.2. Type A, B, C, E and P Records 85
7.3. Type D Records 88
7.4. Type G, N and X Records 91
7.5. Type I Records 98
7.6. Type O Records 99
7.7. Type S Records 101
7.8. Type V Records 106

8. SEDAT and Special SEDAT 112
8.1. SEDAT 112
8.2. Special SEDAT 114

9. Roadbed Pointer List 116

10. District Boundary Geodatabase 119

Appendix A Bytes of the Big Apple™ LION in ESRI Format 122

Appendix C Bytes LION: Populating “Empty” FieldS 127

Appendix D District Boundary Geodatabase 131

Appendix E File Naming Requirements 145

LIST OF TABLES

Table 1 – File Identifier Portion of Field Identifier Code (FIC) 9

Table 2 – Geosupport LION Output File 13

Table 3 – Methodologies for Centerline Segments 18

Table 4 – Populating Atomic Polygon-Based Fields 23

Table 5 - Translation Rules for L40 (Traffic Direction) 25

Table 6 – Assignment Rules for L42 (Feature Type Code) 26

Table 7 – Translation Rules for L47 (Twisted Parity Flag) 27

Table 8 – Methodologies for Shoreline Segments 31

Table 9 – Methodologies for Rail and Subway Segments 34

Table 10 – Methodologies for NonStreetFeature Segments 36

Table 11 – Assignment Rules for L42 (Feature Type Code) 38

Table 12 – AltSegmentData Table Layout 40

Table 13 – AltSegmentData Table: ALT_SEGDATA_TYPE Field 41

Table 14 – AltSegmentData Table: FEATURE_TYPE Field 41

Table 15 – Implicit Feature Class Definitions 42

Table 16 – AltSegmentData Table: FROM_TO_INDICATOR Field 43

Table 17 – Populating Output Records for Proto-Segments 45

Table 18 – Rules for Line Color and Style Based on Feature Type 66

Table 19 – SND Output File – Record Layout 68

Table 20 – Populating SND Output Records 68

Table 21 – Universal and Last Word Table Fields 73

Table 22 – Face Code Output File 76

Table 23 – ThinLION Output File 77

Table 24 – Intentionally left empty (was ThinTract layout) 79

Table 25 – ThinFire Output File 80

Table 26 – SAF Output File Rules 83

Table 27 – CSCL Source Components for SAF Data 84

Table 28 – Layout for SAF Record Types A, B, C, E, G, N, P and X 85

Table 29 – Populating SAF Record Types A, B, C, E and P 87

Table 30 – Layout and Populating Rules for SAF Record Type D 89

Table 31 – Populating SAF Type G, N and X Records 92

Table 32 – Layout and Populating Rules for SAF Record Type I 98

Table 33 – Layout and Populating Rules for SAF Record Type O 99

Table 34 – Layout for SAF Record Type S 102

Table 35 – Layout and Populating Rules for SAF Record Type V 106

Table 36 – SEDAT Output File 112

Table 37 – Special SEDAT Output File 114

Table 38 – Roadbed Pointer List Output File 117

Table 39 – District Boundary Features 120

TABLE OF FIGURES

Figure 1 – Initial UI Screen if Local GDB Does Not Exist 3

Figure 2 – UI Screen After Unchecking Box 4

Figure 3 – Initial UI Screen if Local GDB Exists 5

Figure 4 – UI Screen After Designation of Output Files 8

1. INTRODUCTION

An Extraction, Transformation and Load (ETL) software tool has been developed to generate LION files, Geosupport System files and district boundary polygon files from data extracted and transformed from the Citywide Street Centerline (CSCL) geodatabase. This document contains descriptions of methodologies utilized by the ETL tool to produce these output files. This document has been delivered to the City of New York Department of Information Technology and Telecommunications as part of Phase II of the CSCL project.

An interactive routine has been developed to serve as a user interface for launching the ETL tool. When successfully launched, the tool will then execute as a batch process. The user interface is discussed in detail in Section 1.1.

The ETL tool can run on any Windows machine on which ArcGIS and the CSCL Editor application are installed, both servers and desktop computers. Beyond those requirements, any desired limitations can be imposed by manual coordination. Experience over time within the live CSCL working environment with respect to performance of the ETL tool itself and its impacts on other activities can help inform policy on which machine(s) would best be utilized.

The versions of the CSCL database that are eligible to serve as the input data to the ETL process are the DCP Working version and all of its child versions, and only those versions. The user interface populates a pull-down menu “on the fly” with a list of the eligible versions that exist at that time, from which the user must select one.

During execution of the ETL tool, it is necessary for the input version to be locked against updating. Careful consideration has been given during the detailed programming design of the ETL tool to maximizing its execution performance. That will help to minimize the period during which the version being processed has to be locked.

It is anticipated that the Centerline Maintenance Group staff will run the ETL tool periodically on a regular schedule (to be determined). From time to time, the CMG staff may also run the tool on an ad hoc basis. It will be possible for multiple users to run the tool simultaneously, as long as they are generating non-overlapping sets of output files. That would have to be ensured through manual coordination. Such simultaneous executions could facilitate the expeditious production of output files for a new update cycle.

The ETL tool only produces output files; it does not modify the CSCL database in any way. It is designed to perform only a limited set of validations of the CSCL data. In general, it does not redundantly perform validations that the Maintenance System is designed to perform “on the fly” or that separate batch validation programs will perform[1]. For the validations that the ETL tool does perform, it writes messages into an output report for any errors encountered. Each message states the type of problem and identifies the invalid data in terms appropriate to the type of error, such as a SEGMENTID, a NODEID, house

[1] As part of Phase 3 of the CSCL project, various batch validation programs will be developed separately from the ETL tool. Many of them will be run at ETL time, either before or after the ETL tool is run, depending on whether the particular validation needs to process any ETL output files or only the CSCL database. These “stand-alone” batch validation programs are not further discussed in this document.

number values etc. ESRI OBJECTIDs are included in the messages, facilitating navigation in the CSCL Maintenance System to the feature or table entry that has engendered the given problem.

Upon completing the creation of each output file (including sorting it if necessary), the ETL tool reports that completion in the output report by writing out a summary message for that file. The message identifies the file (and borough, if it is borough-specific) and indicates the file's record count as well as error counts itemized by type of error. The ETL tool may run for a considerable duration, especially if a full set of output files is being generated. If execution is interrupted at any point, whether intentionally by the user or as the result of a problem, the file completion messages will enable the user to re-launch the tool without having to regenerate files that have already been successfully created. Users will be able to monitor progress during execution of the batch process by viewing the output report file via any standard text processor such as Word at any time.

The output path (directory) that serves as the repository for all output files, as well as the (possibly different) path for the output report, are set in the configuration file, and can be modified therein if desired.

Some of the output files are required to be sorted. The specific sort requirements for those files are specified in the respective chapters of this document. Requirements for the names of the ETL output files are contained in Appendix E.

1.1. User Interface

An interactive user interface is used to launch the ETL tool. It enables the user to select the database version to be processed and the output files desired for that run. It is possible to request the production of various combinations of output files, ranging from a single file to the full set. For files that are produced by borough, it is possible to request a specific borough or boroughs. Thus, the user is able to regenerate specific output files that have been found to contain errors or omissions without having to regenerate the full set of files.

The user interface utilizes the CSCL Maintenance System's Login form. When the user clicks the Login button on the screen, that form appears, and the user is then prompted to log in and select a work order or create a new one. A button has also been provided that links the user to an interface for extracting LION Differences Files (LDFs). LDF extraction is performed by a separate tool and is not discussed in this document.

A pull-down menu for the user to select the database version to be processed has been included at the top of the screen.

Below the database version pull-down menu there is a pair of "extract option" check boxes. They are used to control what geodatabase is to be processed as the input data. The two possible input geodatabases are the SDE geodatabase used by the CSCL Maintenance System (defined in the system's configuration file), and a local file geodatabase that must be named **ETL Working GDB.gdb**. If the latter does not exist, but the user wishes the ETL execution to access local data, the user can direct the ETL tool to create a local file GDB by copying the SDE database. If a local file GDB having the aforementioned name

already exists, the user can choose to use it or to delete it and create a new one by copying the SDE database.

The appearance of the initial User Interface screen depends on whether a local file GDB exists. If it does not, the initial screen appears as shown in Figure 1.

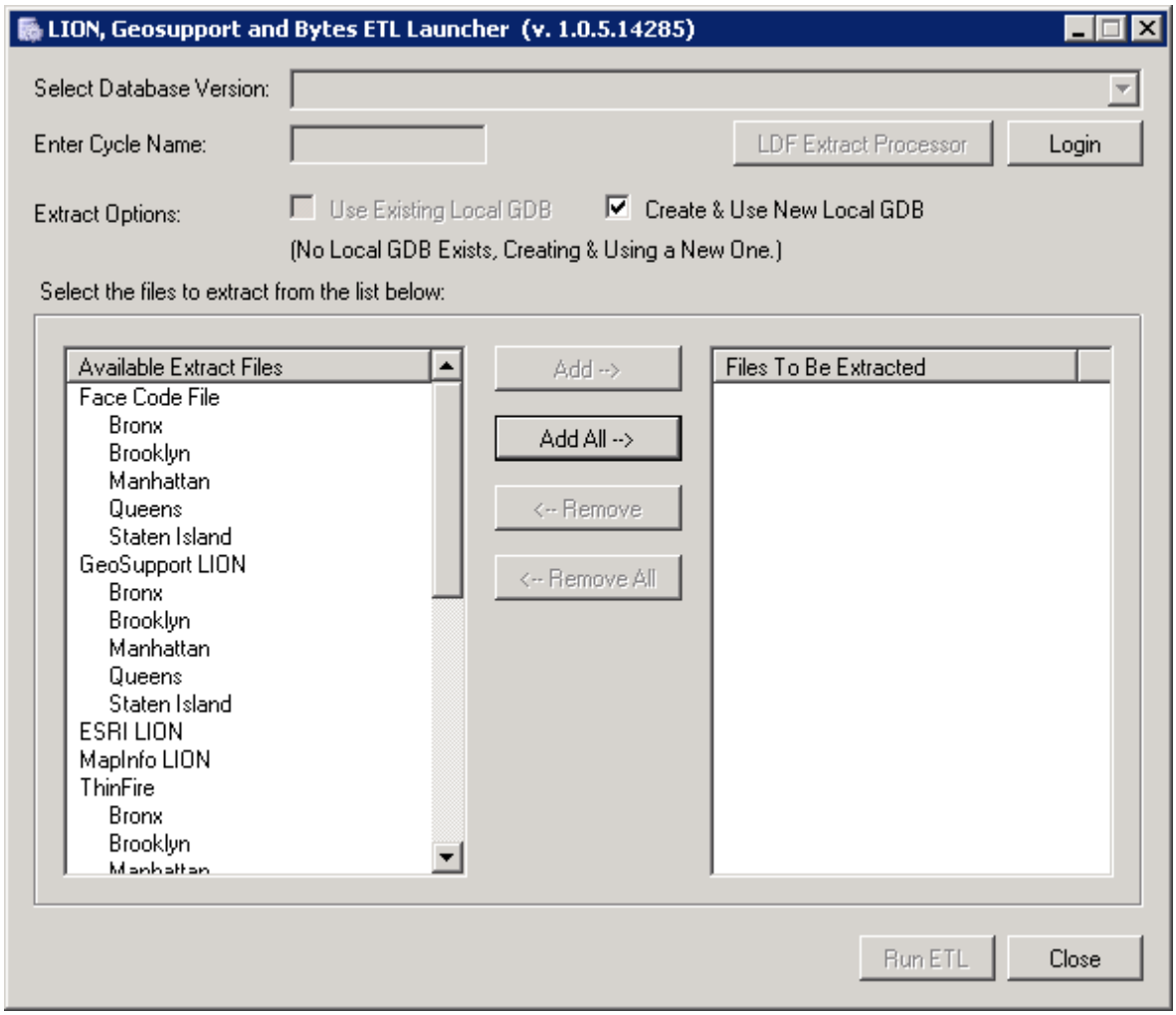

Figure 1 – Initial UI Screen if Local GDB Does Not Exist

Note the following:

- The message “No Local GDB Exists, Creating & Using a New One” is displayed.
- The “Use Existing Local GDB“ check box is disabled
- The “Create & Use New Local GDB” check box contains a check mark.

If the user leaves the Create & Use New Local GDB check box checked, the ETL tool will automatically create a new local file GDB with the required name by copying the SDE database, and will use that as the input data during this ETL run.

If the user unchecks the Create & Use New Local GDB check box, the User Interface screen will appear as shown in Figure 2. Note that the message has disappeared. In this case, the ETL tool will process the SDE GDB as the input data. It will not create a local file GDB.

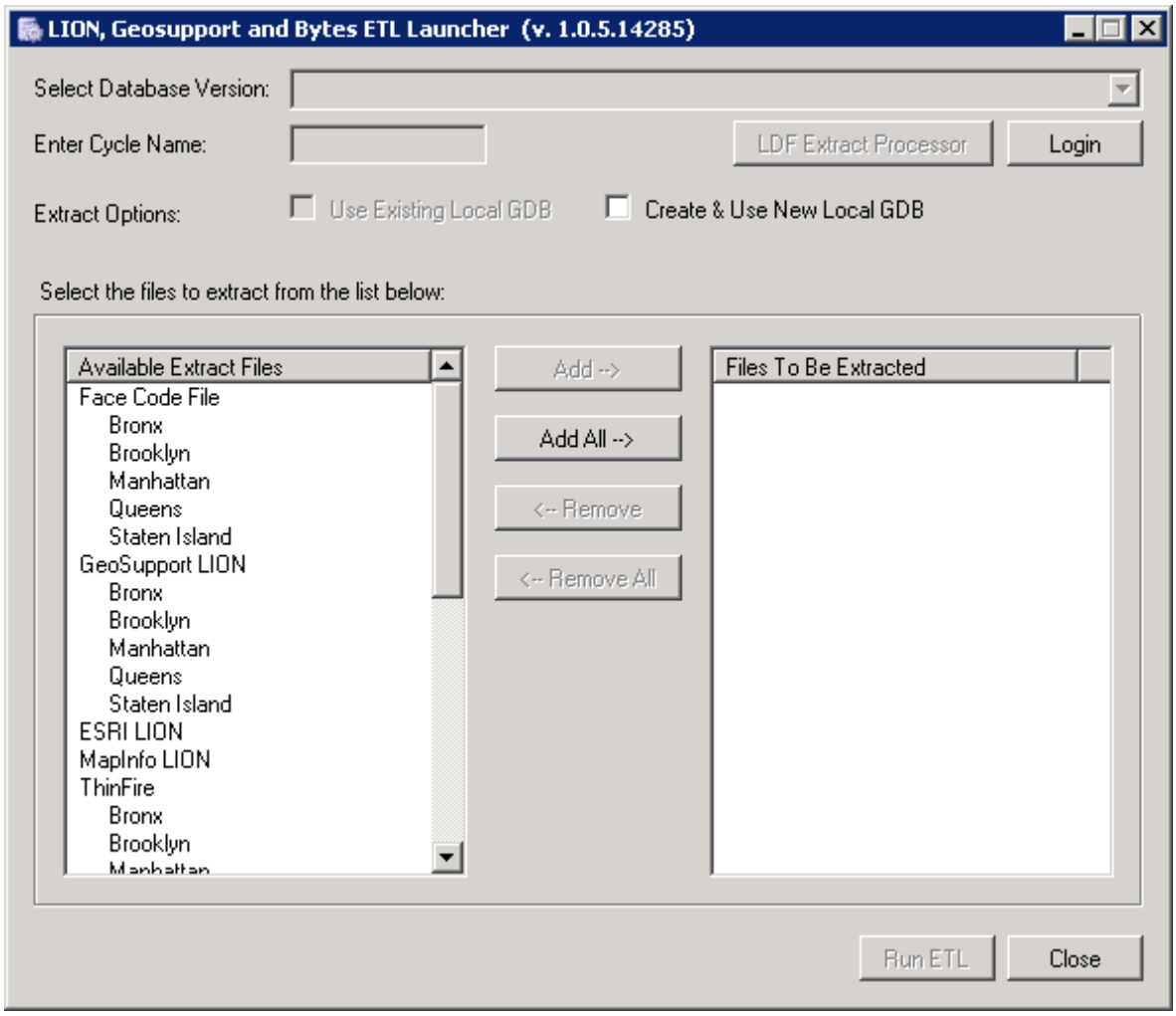

Figure 2 – UI Screen After Unchecking Box

If a local file geodatabase exists, the initial User Interface screen appears as shown in Figure 3.

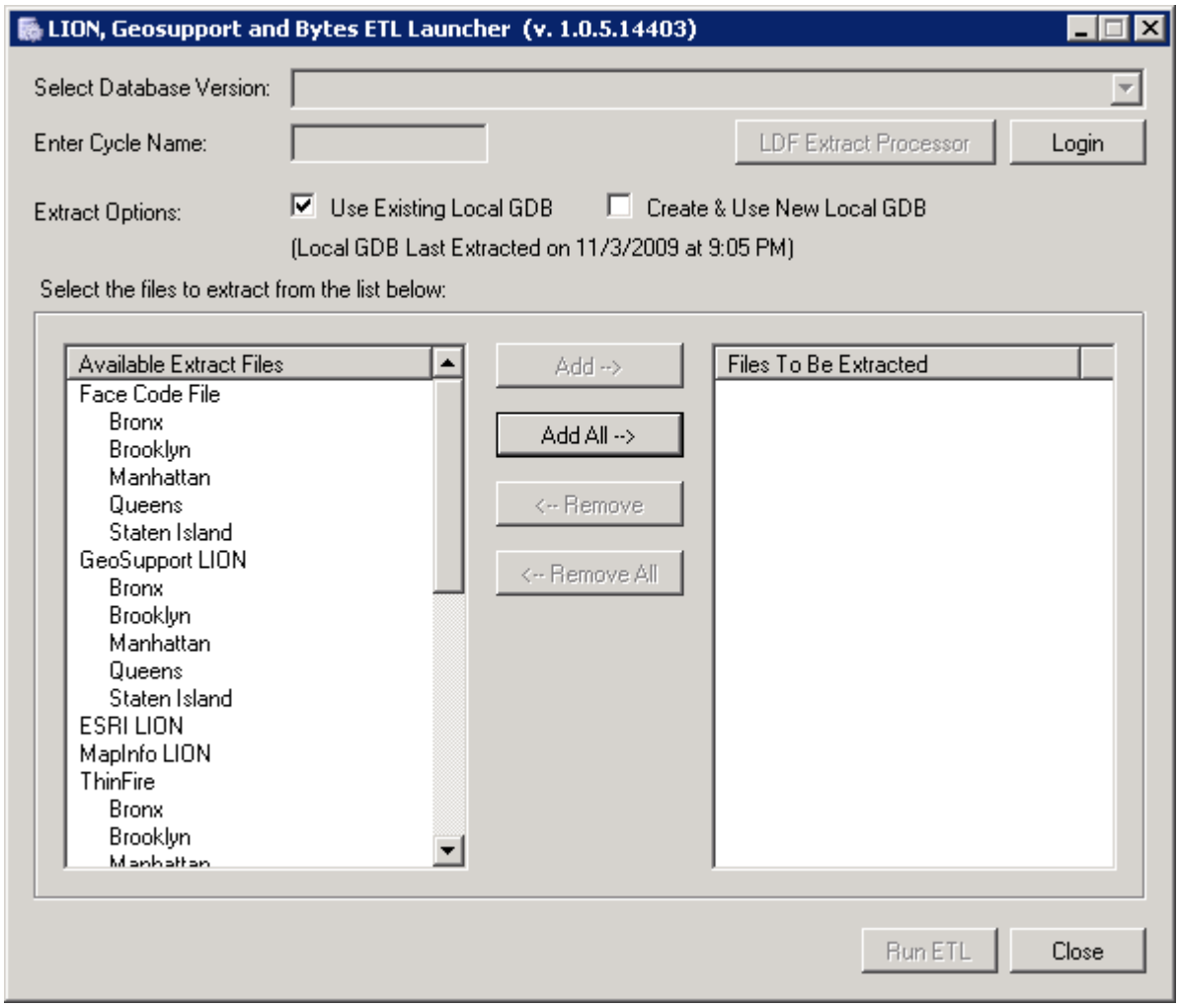

Figure 3 – Initial UI Screen if Local GDB Exists

Note the following:

- A message indicating the date and time when the existing local file GDB was created is displayed.
- Both check boxes are enabled.
- The “Use Existing Local GDB” check box contains a check mark.

If the user leaves the “Use Existing Local GDB” check box checked, the ETL tool will process the existing local file geodatabase as the input data.

If the user checks the “Create & Use New Local GDB”, the ETL tool will uncheck the “Use Existing Local GDB” check box, it will replace the message with the message “Creating & Using a New Local GDB”, it will delete the existing local file GDB and it will create a new one by copying the SDE database.

The names of some of the output files include the update cycle identifier as part of the name. (See Appendix E.) If any of those output files are to be created, it is mandatory for the user to insert a value into the text box near the upper left of the screen labeled “Enter Cycle Name”. That text box is enabled only when the user has selected at least one of those files.

The user designates the output files to be extracted and launches the batch ETL process as follows:

- The user can choose to generate any combination of the output files. When the user begins, the scrollable text box at the left side of the screen is initialized to contain a list of all of the possible ETL output files. Files that are extracted by borough are listed generically, followed by the names of the five boroughs.

- The text box at the right side of the screen contains a list of the output files that the user has designated for extraction so far. When the user begins, the right text box is initialized to empty.

There are three alternative means for the user to designate files for extraction:

- The user can double-click a file (or a specific borough of a file that is extracted by borough) in the left-hand box, whereupon it will be moved from the left-hand box to the right-hand box (that is, it will be added to the right-hand box and removed from the left-hand box). If the user double-clicks the generic name of a file that is extracted by borough, all boroughs for that file that are not currently designated will be designated (moved to the right-hand box).

- The user can select a file (or a borough of a file) in the left-hand box with a single click, and then use the Add button to move the selected file to the right-hand box. Multiple files can be selected by holding down the CTRL key and single-clicking each desired file. Using the Add button will then cause all selected files to be moved to the right-hand box. Selecting and adding the generic name of a file that is extracted by borough will cause all not currently designated boroughs for that file to be moved to the right-hand box.

- The user can utilize the Add All button to designate all files for extraction, that is, to move all files from the left-hand box to the right-hand box.

There are three alternative means for the user to de-designate currently designated files (files currently listed in the right-hand box):

- The user can double-click on a file in the right-hand box, whereupon it will be moved to the left-hand box. If the user double-clicks a generic name, all currently designated boroughs of that file will be moved to the left-hand box.

- The user can select a file (or borough) in the right-hand box with a single click, or select multiple files by holding down the CTL key while single-clicking the desired files, then use the Remove button to move the selected files to the left-hand box. If the user selects and removes a generic name, all currently designated boroughs of that file will be moved to the left-hand box.

- The user can utilize the Remove All button to de-designate (move from the right-hand box to the left-hand box) all currently designated files.

An efficient way to designate all but a few files for extraction is to first use the Add All button to designate all files simultaneously, and then individually de-designate the few specific files that are not desired.

The user interface will enable the “Run ETL” button at the bottom of the screen whenever the user has selected a database version and has designated at least one output file. If the user utilizes the “Run ETL” button, the user interface will submit a batch process to execute the ETL tool and will terminate. The ETL tool, running as a batch process, will produce the output files designated by the user in the order in which they were listed in the “Files to Be Extracted” list box.

The “Close” button at the bottom of the screen can be used at any time to exit the user interface without launching the batch ETL process.

Figure 4 depicts a sample screen after the user has designated some output files. The “Run ETL” button is enabled. The user may now launch the batch ETL process. Note that in this example, the user has chosen to process an existing local file GDB.

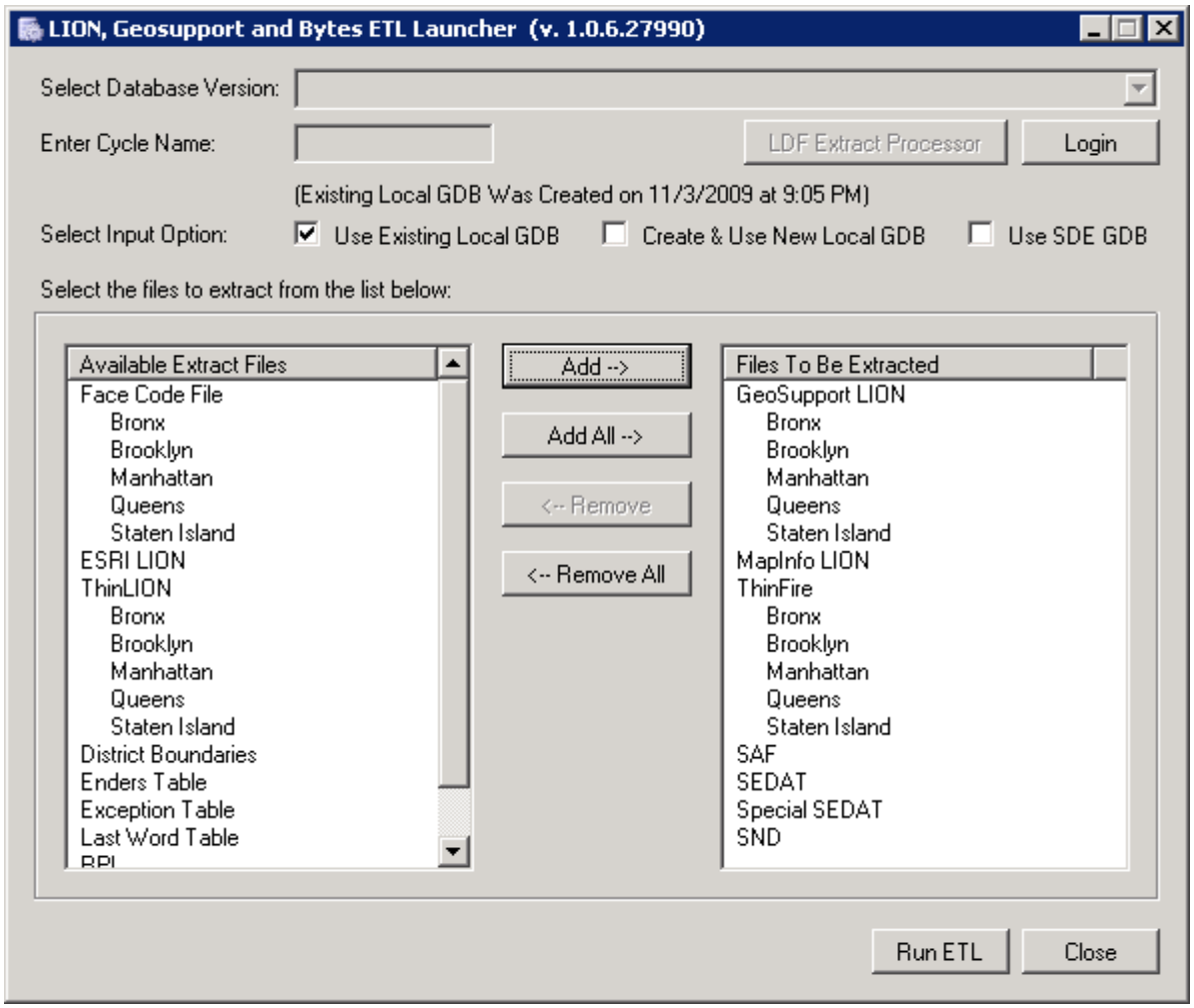

Figure 4 – UI Screen After Designation of Output Files

1.2. Summary of Output Files

The ETL tool is able to generate the following Geosupport-related files:

- LION, discussed in Chapter 2. The following formats will be produced:
  - A Geosupport version of LION (a separate file for each borough)
  - A BYTES of the BIG APPLE™ LION in ESRI file geodatabase format (a single file geodatabase for the entire city)
  - ~~A BYTES of the BIG APPLE™ LION in MapInfo MIF format (a single set of files for the entire city)~~
- Street Name Dictionary, discussed in Chapter 3
- Geosupport normalizing tables, discussed in Chapter 4:
  - Exception Table

  - Universal Word Table and Last Word Table
  - Enders Table
- Face Code File (a separate file for each borough), discussed in Chapter 5
- District equivalency files, discussed in Chapter 6:
  - ThinLION (a separate file for each borough)
  - ThinFire (a separate file for each borough)
- Special Address File (SAF), discussed in Chapter 7
- Election District-related files, discussed in Chapter 8:
  - Split ED Address table (SEDAT)
  - Special SEDAT
- Roadbed Pointer List (RPL), discussed in Chapter 9
- District boundary polygons for various types of operational, administrative and political districts in ESRI file geodatabase format, discussed in Chapter 10

Note that the ETL tool is not designed to generate LDF files. They can be generated by a separate process that has been developed as part of CSCL Phase III. The ETL tool's user interface contains a link to that process.

1.3. Field Identification Codes (FICs)

To ensure that references to fields in output files are unambiguous and consistent within this document, a Field Identifier Code (FIC) has been assigned to each field in each of the output files other than those for the BYTES of the BIG APPLE™. The FICs appear in the first column of the record layout tables provided for each output file.

The first portion of the FIC is an alphabetic string that identifies the output file, or in the case of the SAF file, which has multiple record types with differing layouts, it identifies the record type. No layouts are provided in this document for files in which the entire record consists of a single field. Therefore, those files have not been assigned FIC's. Those files are the Exception table and the Enders table.

The alphabetic file identifiers are listed in Table 1.

Table 1 – File Identifier Portion of Field Identifier Code (FIC)

File ID	File	Record Type(s)
FC	Face Code File	
L	Geosupport LION	
RPL	RPL	
SAFA	SAF	A, B, C, E, G, N, P and X
SAFD	SAF	D
SAFI	SAF	I
SAFV	SAF	O and V
SAFS	SAF	S
SEDAT	SEDAT and Special SEDAT	
SNDN	SND	Non-S
TF	ThinFire	
TL	ThinLION	
WT	Universal and Last Word Tables	

The digits that follow the alphabetic portion of the FIC identify the specific field within the given file. For example, the FIC value 'L23' identifies the 23rd field in file L (the LION File), which is the Left Department of Sanitation Subsection (see Table 2). Filler fields are not in general assigned FIC values.

1.4. Field Formatting

All output fields labeled as "filler" in the layouts in this document will be populated with blanks. All output fields populated from a CSCL field whose value is null will be populated with blanks unless otherwise noted.

The following abbreviations are used below to refer to field formats:

LJ = "left-justified"

RJ = "right-justified"

SF = "space-filled"

ZF = "zero-filled"

House number fields and census tract fields have special formatting requirements, as follows:

- House number fields:
  - Unhyphenated house numbers will be stored in output house number fields RJSF.
  - Hyphenated house numbers will be stored RJSF. There will be three bytes after the hyphen, with the value after the hyphen stored RJZF within those three bytes. (Note the **ASSUMPTION**: hyphenated house numbers will never have more than three digits to the left of the hyphen. It would be

impossible to store such a house number in a seven-byte house number field so that it is formatted with three bytes after the hyphen.)

- If there is no house number, the value stored in the output house number field will consist of a single zero RJSF. That is, the value will consist of six spaces followed by a zero.

- "Standalone" census tract fields except tract label fields:
  - Tract fields (other than label fields) are a total of six bytes in length. They consist of two subfields: a "basic tract number" subfield of four bytes followed by a "tract suffix" subfield of two bytes. Symbolically, this format can be expressed as BBBBSS, where BBBB is the four-byte basic tract number field and SS is the two-byte tract suffix subfield.
  - The basic tract number value is stored in its subfield RJSF.
  - The suffix value is stored in its subfield RJZF, except that if there is no suffix, the suffix subfield will contain blanks.
  - Examples ('*' represents a blank):
    - **107201** (basic number is 1072, suffix is 01)
    - ****5604** (basic number is 56, suffix is 04)
    - ****87**** (basic number is 87, no suffix)

- In fields that contain a Borough Code and Census Tract concatenated (for example, field TT5, BoroCT2000, in the ThinTract ETL file), the value of the basic tract number will be stored RJSF in the four-byte basic tract subfield. If there is no suffix, the suffix subfield will be filled with blanks. For example, if the Borough Code = 4, the basic tract = 32, and there is no suffix, the field will be populated with the value **4**32**,** where '*' represents a blank.

- Tract number label fields (for example, field TT1, Census Tract Label, in the ThinTract ETL file) are a total of seven bytes in length. They consist of up to three subfields: a basic tract number subfield of four bytes, followed by a one-byte subfield possibly containing a decimal point, followed by a tract suffix subfield of two bytes. Symbolically, the format can be expressed as BBBBDSS where BBBB is the four-byte basic field, D is the decimal point (if any) and SS is the two-byte suffix subfield.
  - The entire tract value will be stored LJSF within the seven-byte field.
  - If there is a suffix, a decimal point will be stored in the decimal point subfield, and the suffix will be stored RJZF within the suffix subfield.
  - If there is no suffix, blanks will be stored in the decimal point and suffix subfields.

- Examples: ('*' represents a blank):
  - **1072.01** (basic number is 1072, suffix is 01)
  - **56.04**** (basic number is 56, suffix is 04)
  - **87***** (basic number is 87, no suffix)

2. LION

The ETL tool is designed to extract a Geosupport version of LION as well as BYTES of the BIG APPLE™ versions of LION. The Geosupport LION is extracted as a sequential file, as discussed in detail in Sections 2.1 through 2.6.7. The BYTES of the BIG APPLE™ LION is extracted in the formats, ESRI file geodatabase format ~~and MapInfo (MIF) format~~, as discussed in Sections 2.7 and 2.8, respectively.

The ETL tool produces a separate Geosupport LION output file for each of the five boroughs. These files will be produced by generating the following output records:

- One record for each segment in CSCL's linear feature classes (Centerline, Shoreline, Rail, Subway and NonStreetFeature), EXCEPT:
    - Centerline segments for private streets (RWJURISDICTION = '3') that do not exist physically (STATUS not = '2')
    - Subterranean rail and subway segments that do not serve as district boundaries (ROW_TYPE = 1 or 8). (However, records for such segments will be written into the BYTES of the BIG APPLE™ versions of LION.)
    - Centerline segments for Driveway (RW=8). (However, records for such segments will be written into the BYTES of the BIG APPLE™ versions of LION)
- One record for each "proto-segment" (coincident segment entry in the CSCL AltSegmentData table), discussed in detail in Section 2.6.

2.1. Geosupport LION Output Record Layout

Table 2 contains a record layout of the Geosupport LION output file. The penultimate column indicates formatting information for the given field. A 'RJZF' in this column signifies right-justified with leading zeros. The last column indicates for which types of features the given field is populated. For output records generated from segments in feature classes not specified in the last column, the ETL tool will populate the given field with blanks.

Table 2 – Geosupport LION Output File

FIC	Field	Len	Frm	To	Format	Source CSCL Feature Classes
L1	Borough	1	1	1	Digit > 0	All
L2	Face Code	4	2	5	RJZF	All
L3	Sequence Number	5	6	10	RJZF	All
L4	Segment ID	7	11	17	RJZF	All

FIC	Field	Len	Frm	To	Format	Source CSCL Feature Classes
L5	5-Digit Street Code (5SC)	5	18	22	RJZF	All
L6	LGC1	2	23	24	RJZF	All
L7	LGC2	2	25	26	RJZF, blank if none	All
L8	LGC3	2	27	28	RJZF, blank if none	All
L9	LGC4	2	29	30	RJZF, blank if none	All
L10	Board of Elections LGC Pointer	1	31	31	Blank if none	All
L11	From-Sectional Map	2	32	33	RJZF	All
L12	From-Node ID	7	34	40	RJZF	All
L13	From-X Coordinate	7	41	47	RJZF	All
L14	From-Y Coordinate	7	48	54	RJZF	All
L15	To-Sectional Map	2	55	56	RJZF	All
L16	To-Node ID	7	57	63	RJZF	All
L17	To-X Coordinate	7	64	70	RJZF	All
L18	To-Y Coordinate	7	71	77	RJZF	All
L19	Left 2000 Census Tract	6	78	83	See Sec. 1.4	All
L20	Left Dynamic Block	3	84	86	RJZF or blank if none	All
L21	Left Low House Number	7	87	93	See Sec. 1.4	Centerline only
L22	Left High House Number	7	94	100	See Sec. 1.4	Centerline only
L23	Left Dept of Sanitation Subsection	2	101	102	RJZF, blank if none	Centerline only
L24	Left Zip Code	5	103	107	Blank if none	All
L25	Left Assembly District	2	108	109	RJZF, blank if none	All
L26	Left Election District	3	110	112	RJZF, blank if none	All
L27	Left School District	2	113	114	RJZF, blank if none	All
L28	Right 2000 Census Tract	6	115	120	See Sec. 1.4	All
L29	Right Dynamic Block	3	121	123	RJZF, blank if none	All
L30	Right Low House Number	7	124	130	See Sec. 1.4	Centerline only
L31	Right High House Number	7	131	137	See Sec. 1.4	Centerline only
L32	Right Dept of Sanitation Subsection	2	138	139	RJZF, blank if none	Centerline only

FIC	Field	Len	Frm	To	Format	Source CSCL Feature Classes
L33	Right Zip Code	5	140	144	Blank if none	All
L34	Right Assembly District	2	145	146	RJZF, blank if none	All
L35	Right Election District	3	147	149	RJZF, blank if none	All
L36	Right School District	2	150	151	RJZF, blank if none	All
L37	Split Election District Flag	1	152	152	Blank if none	Centerline only
L38	Filler (formerly Split Community School District Flag)	1	153	153	Blank	N/A
L39	Sanitation District Boundary Indicator	1	154	154	Blank if none	Centerline only
L40	Traffic Direction	1	155	155	Blank if none	Centerline only
L41	Segment Locational Status	1	156	156	Blank if none	All
L42	Feature Type Code	1	157	157	Blank if none	All
L43	Non-Pedestrian Flag	1	158	158	Blank if none	Centerline only
L44	Continuous Parity Indicator	1	159	159	Blank if none	Centerline only
L45	Filler (formerly the Near BQ-Boundary Flag)	1	160	160	Blank	N/A
L46	Borough Boundary Indicator	1	161	161	Blank if none	All
L47	Twisted Parity Flag	1	162	162	Blank if none	Centerline only
L48	Special Address Flag	1	163	163	Blank if none	All
L49	Curve Flag	1	164	164	Blank if none	All
L50	Center of Curvature X-Coordinate	7	165	171	Leading zeros	Centerline only
L51	Center of Curvature Y-Coordinate	7	172	178	Leading zeros	Centerline only
L52	Segment Length in Feet	5	179	183	RJZF	All
L53	From Level Code	1	184	184	Character	All
L54	To Level Code	1	185	185	Character	All
L55	Traffic Direction Verification Flag	1	186	186	Blank if none	Centerline only
L56	Segment Type Code	1	187	187	Character	Centerline only
L57	Coincident Segment Counter	1	188	188	Integer > 0	All
L58	Include/Exclude Flag	1	189	189	Blank if none	Centerline only

FIC	Field	Len	Frm	To	Format	Source CSCL Feature Classes
L59	Roadway Type	2	190	191	Blank if none	Centerline only
L60	PHYSICALID	7	192	198	RJZF, blank if none	Centerline only
L61	GENERICID	7	199	205	RJZF, blank if none	Centerline only
L62	NYPDID	7	206	212	RJZF, blank if none	Centerline only
L63	FDNYID	7	213	219	RJZF, blank if none	Centerline only
L64	~~Left BLOCKFACEID~~ Filler	7	220	226	RJZF, blank if none	Centerline only
L65	~~Right BLOCKFACEID~~ Filler	7	227	233	RJZF, blank if none	Centerline only
L66	STATUS	1	234	234	Blank if none	Centerline only
L67	STREETWIDTH_MIN	3	235	237	Blank if none	Centerline only
L68	STREETWIDTH_IRR	1	238	238	Blank if none	Centerline only
L69_1	BIKELANE_1	1	239	239	Blank if none	Centerline only
L70	FCC	2	240	241	Blank if none	Centerline only
L71	Right of Way Type	1	242	242	Blank if none	Rail and Subway only
L72	Left 2010 Census Tract	6	243	248	See Sec. 1.4	All
L73	Right 2010 Census Tract	6	249	254	See Sec. 1.4	All
L74	LGC5	2	255	256	01 to 99 or Blanks	All
L75	LGC6	2	257	258	01 to 99 or Blanks	All
L76	LGC7	2	259	260	01 to 99 or Blanks	All
L77	LGC8	2	261	262	01 to 99 or Blanks	All
L78	LGC9	2	263	264	01 to 99 or Blanks	All
L79	Legacy SEGMENTID	7	265	271	RJZF	All but Shoreline

FIC	Field	Len	Frm	To	Format	Source CSCL Feature Classes
L80	LEFT CENSUS BLOCK 2000 BASIC	4	272	275	RJSF	All
L81	LEFT CENSUS BLOCK 2000 SUFFIX	1	276	276	Blank if none	All
L82	RIGHT CENSUS BLOCK 2000 BASIC	4	277	280	RJSF	All
L83	RIGHT CENSUS BLOCK 2000 SUFFIX	1	281	281	Blank if none	All
L84	LEFT CENSUS BLOCK 2010 BASIC	4	282	285	RJSF	All
L85	LEFT CENSUS BLOCK 2010 SUFFIX	1	286	286	Blank if none	All
L86	RIGHT CENSUS BLOCK 2010 BASIC	4	287	290	RJSF	All
L87	RIGHT CENSUS BLOCK 2010 SUFFIX	1	291	291	Blank if none	All
L88	SNOW PRIORITY	1	292	292	Blank if none	Centerline only
L69_2	BIKELANE_2	2	293	294	RJSF, 1-11, Blank if none	Centerline only
L67_2	STREET WIDTH MAX	3	295	297	RJSF, Blank if none	Centerline only
L89	Filler	3	298	300		
L90	LEFT BLOCK FACE ID	10	301	310	RJZF, blank if none	Centerline only
L91	RIGHT BLOCK FACE ID	10	311	320	RJZF, blank if none	Centerline only
L92	NUMBER TRAVEL LANE	2	321	322	RJZF, blank if none	Centerline only
L93	NUMBER PARK LANES	2	323	324	RJSF	Centerline only
L94	NUMBER TOTAL LANES	2	325	326	RJSF	Centerline only
L95	BIKE TRAFFIC DIR	2	327	328	RJSF	Centerline only
L96	POSTED SPEED	2	329	330	RJSF	Centerline only
L97	Left NYPD Service Area	1	331	331	RJSF	All
L98	Right NYPD Service Area	1	332	332	RJSF	All
L99	Truck Route Type	1	333	333	RJSF	Centerline only

FIC	Field	Len	Frm	To	Format	Source CSCL Feature Classes
L100	LEFT 2020 CENSUS TRACT LEFT 2020 CENSUS TRACT BASIC LEFT 2020 CENSUS TRACT SUF	 4 2	 334 338	 337 339	See Sec. 1.4	All
L101	RIGHT 2020 CENSUS TRACT RIGHT 2020 CENSUS TRACT BASIC RIGHT2020 CENSUS TRACT SUF	 4 2	 340 344	 343 345	See Sec. 1.4	All
L102	LEFT CENSUS BLOCK 2020 BASIC	4	346	349	RJSF	All
L103	LEFT CENSUS BLOCK 2020 SUFFIX	1	350	350	Blank if none	All
L104	RIGHT CENSUS BLOCK 2020 BASIC	4	351	354	RJSF	All
L105	RIGHT CENSUS BLOCK 2020 SUFFIX	1	355	355	Blank if none	All
L199	Filler	45	356	400		All

(Note for Census Tract fields:
CT Basic, RJSF
CT Suf , 2 bytes RJZF if it has value; Blank if it is none)

Many of the Geosupport LION output fields are populated by directly copying the contents of a corresponding source attribute field in a CSCL linear feature class, with no re-formatting or other processing required, other than to replace null values with blanks. Other output fields are populated via translation or reformatting of the values in corresponding source CSCL fields or require more elaborate processing.

Sections 2.2, 2.3, 2.4 and 2.5 describe the methodology for populating fields in Geosupport LION output records generated from segments in the feature class Centerline, Shoreline, Rail and Subway, and NonStreetFeature, respectively. Section 2.6 describes the generation of output records for proto-segments. For simplicity, the term LION is used in these sections to refer to the Geosupport LION output file.

2.2. Generating Records from Centerline Segments

Table 3 and the sections following it indicate how each field is populated in LION output records generated from Centerline segments.

Table 3 – Methodologies for Centerline Segments

FIC	Field	Populating Method
L1	Borough	Copy Centerline attribute BOROUGHCODE
L2	Face Code	See Section 2.2.1
L3	Sequence Number	Copy Centerline attribute SEGMENT_SEQNUM

FIC	Field	Populating Method
L4	Segment ID	Copy Centerline attribute SEGMENTID
L5	5-Digit Street Code (5SC)	See Section 2.2.1
L6	LGC1	See Section 2.2.1
L7	LGC2	See Section 2.2.1
L8	LGC3	See Section 2.2.1
L9	LGC4	See Section 2.2.1
L10	Board of Elections LGC Pointer	See Section 2.2.1
L11	From-Sectional Map	See Section 2.2.2
L12	From-Node ID	See Section 2.2.2
L13	From-X Coordinate	See Section 2.2.2
L14	From-Y Coordinate	See Section 2.2.2
L15	To-Sectional Map	See Section 2.2.2
L16	To-Node ID	See Section 2.2.2
L17	To-X Coordinate	See Section 2.2.2
L18	To-Y Coordinate	See Section 2.2.2
L19	Left 2000 Census Tract	See Section 2.2.3
L20	Left Dynamic Block	See Section 2.2.3
L21	Left Low House Number	Copy Centerline attribute L_LOW_HN
L22	Left High House Number	Copy Centerline attribute L_HIGH_HN
L23	Left Dept of Sanitation Subsection	Copy leftmost 2 bytes of Centerline attribute LSUBSECT
L24	Left Zip Code	Copy Centerline attribute L_ZIP
L25	Left Assembly District	See Section 2.2.3
L26	Left Election District	See Section 2.2.3
L27	Left School District	See Section 2.2.3
L28	Right 2000 Census Tract	See Section 2.2.3
L29	Right Dynamic Block	See Section 2.2.3
L30	Right Low House Number	Copy Centerline attribute R_LOW_HN
L31	Right High House Number	Copy Centerline attribute R_HIGH_HN
L32	Right Dept of Sanitation Subsection	Copy leftmost 2 bytes of Centerline attribute RSUBSECT
L33	Right Zip Code	Copy Centerline attribute R_ZIP
L34	Right Assembly District	See Section 2.2.3
L35	Right Election District	See Section 2.2.3
L36	Right School District	See Section 2.2.3
L37	Split Election District Flag	See Section 2.2.4
L38	Filler (formerly Split Community School District Flag)	Populate with blank
L39	Sanitation District Boundary Indicator	Copy Centerline attribute SANDIST_IND
L40	Traffic Direction	See Section 2.2.5
L41	Segment Locational Status	See Section 2.2.6
L42	Feature Type Code	See Section 2.2.7
L43	Non-Pedestrian Flag	Copy Centerline attribute NONPED
L44	Continuous Parity Indicator	If Centerline attribute CONTINUOUS_PARITY_FLAG = 1 or 2, populate L44 with ‘L’ or ‘R’, respectively. Else, populate L44 with a blank.
L45	Filler (formerly the Near BQ-Boundary Flag)	Populate with blank
L46	Borough Boundary Indicator	See Section 2.2.8
L47	Twisted Parity Flag	See Section 2.2.9
L48	Special Address Flag	See Section 2.2.10
L49	Curve Flag	See Section 2.2.11
L50	Center of Curvature X-Coordinate	See Section 2.2.11
L51	Center of Curvature Y-Coordinate	See Section 2.2.11

FIC	Field	Populating Method
L52	Segment Length in Feet	Populate with value of Centerline attribute SHAPE.LEN rounded to nearest foot
L53	From Level Code	See Section 2.2.12
L54	To Level Code	See Section 2.2.12
L55	Traffic Direction Verification Flag	Copy Centerline attribute TRAFDIR_VER_FLAG Note: A CR was approved 1/09/09 to change domain of this attribute to conform to LION
L56	Segment Type Code	Copy Centerline attribute SEGMENT_TYPE
L57	Coincident Segment Counter	Copy Centerline attribute COINCIDENT_SEG_COUNT without counting any subterranean subway/rail segment
L58	Include/Exclude Flag	Copy Centerline attribute INCEX_FLAG
L59	Roadway Type	Copy Centerline attribute RW_TYPE
L60	PHYSICALID	Copy Centerline attribute PHYSICALID
L61	GENERICID	Copy Centerline attribute GENERICID
L62	NYPDID	Copy Centerline attribute NYPDID
L63	FDNYID	Copy Centerline attribute FDNYID
L64	~~Left BLOCKFACEID~~ Filler	blank
L65	~~Right BLOCKFACEID~~ Filler	blank
L66	STATUS	Copy Centerline attribute STATUS
L67	STREETWIDTH_MIN	Copy Centerline attribute STREETWIDTH_MIN
L68	Irregular Street Width Flag	Copy Centerline attribute STREETWIDTH_IRR
L69_1	Bike Lane Indicator	Copy the first digit of Centerline attribute BIKE_LANE(one digit). Value “10” converted to “A”, value “11” converted to “B”.
L70	FCC	Copy Centerline attribute FCC
L71	Right of Way Type	Populate with blank (this field used only for Rail and Subway segments)
L72	Left 2010 Census Tract	See Section 2.2.3
L73	Right 2010 Census Tract	See Section 2.2.3
L74	LGC5	See Section 2.2.1
L75	LGC6	See Section 2.2.1
L76	LGC7	See Section 2.2.1
L77	LGC8	See Section 2.2.1
L78	LGC9	See Section 2.2.1
L79	Legacy SEGMENTID	Copy Centerline attribute LEGACY_SEGMENTID
L80	LEFT CENSUS BLOCK 2000 BASIC	See Section 2.2.3
L81	LEFT CENSUS BLOCK 2000 SUFFIX	See Section 2.2.3
L82	RIGHT CENSUS BLOCK 2000 BASIC	See Section 2.2.3
L83	RIGHT CENSUS BLOCK 2000 SUFFIX	See Section 2.2.3
L84	LEFT CENSUS BLOCK 2010 BASIC	See Section 2.2.3
L85	LEFT CENSUS BLOCK 2010 SUFFIX	See Section 2.2.3
L86	RIGHT CENSUS BLOCK 2010 BASIC	See Section 2.2.3
L87	RIGHT CENSUS BLOCK 2010 SUFFIX	See Section 2.2.3
L88	SNOW PRIORITY	Copy Centerline attribute SNOW PRIORITY
L69_2	BIKELANE_2	Copy Centerline attribute BIKE_LANE(2 digits)
L67_2	STREET WIDTH MAX	Copy Centerline attribute STREETWIDTH_MAX
L89	Filler	3 digits
L90	LEFT BLOCK FACE ID	Copy Centerline attribute L_BLOCKFACEID
L91	RIGHT BLOCK FACE ID	Copy Centerline attribute R_BLOCKFACEID
L92	NUMBER TRAVEL LANE	Copy Centerline attribute NUMBER_TRAVEL_LANES
L93	NUMBER PARK LANES	Copy Centerline attribute NUMBER_PARK_LANES

FIC	Field	Populating Method
L94	NUMBER TOTAL LANES	Copy Centerline attribute NUMBER_TOTAL_LANES
L95	BIKE TRAFFIC DIR	Copy Centerline attribute BIKE_TRAFDIR
L96	POSTED SPEED	Copy Centerline attribute POSTED_SPEED
L97	Left NYPD Service Area	See Section 2.2.13
L98	Right NYPD Service Area	See Section 2.2.13
L99	Truck Route Type	Copy Centerline attribute TRUCK_ROUTE_TYPE
L100	LEFT 2020 CENSUS TRACT	See Section 2.2.3
L101	RIGHT 2020 CENSUS TRACT	See Section 2.2.3
L102	LEFT CENSUS BLOCK 2020 BASIC	See Section 2.2.3
L103	LEFT CENSUS BLOCK 2020 SUFFIX	See Section 2.2.3
L104	RIGHT CENSUS BLOCK 2020 BASIC	See Section 2.2.3
L105	RIGHT CENSUS BLOCK 2020 SUFFIX	See Section 2.2.3
L199	Filler	blank

2.2.1. Street Code and Face Code Items

FIELDS: L2 (Face Code), L5 (5-Digit Street Code (5SC)), L6 through L9 (LGC1, LGC2, LGC3, LGC4), L10 (Board of Elections Preferred LGC), L74 through L78 (LGC5, LGC6, LGC7, LGC8, LGC9)

ASSUMPTIONS: The ETL tool assumes the following conditions to be true with respect to the CSCL database, and does not perform validations for them:

- Every feature in the Centerline feature class has at least one street name and corresponding street code in the StreetName table. Every feature in the Rail, Subway, Shoreline and NonStreetFeature feature classes has at least one feature name and corresponding street code in the FeatureName table.
- Every segment (of any linear feature class) has at least one Segment_LGC table entry and has at most nine.
- All of the Segment_LGC entries for a given SEGMENTID contain the same B5SC value.
- Precisely one of the Segment_LGC entries for a given SEGMENTID contains a 'Y' in the PREFERRED_LGC_FLAG. (The LGC in that entry is the "preferred LGC" for that segment, and the B7SC formed by concatenating the B5SC and the preferred LGC is the "preferred B7SC".)
- The FACECODE field is non-empty in the StreetName and FeatureName table entries for preferred B7SCs.
- Exactly one of the Segment_LGC entries for a given SEGMENTID contains a 'Y' in the BOE_PREFERRED_LGC_FLAG.

METHODOLOGY:

- The tool will examine all entries in the Segment_LGC table for the given SEGMENTID.
- The tool will populate the output 5SC field from the last five bytes of the B5SC field in any of the Segment_LGC entries for the given SEGMENTID.
- The tool will populate the output LGC fields as follows:
    - The tool will populate LGC1 with the preferred LGC.
    - If there are additional Segment_LGC entries for the given SEGMENTID, the tool will populate the output LGC2 through LGC9 fields in sequence from the LGCs in those additional Segment_LGC entries, until all Segment_LGC entries for the given SEGMENTID have been used.
    - The tool will populate any unused output LGC fields with blanks.
    - If Segment_LGC ever contains more than nine entries for a SEGMENTID, the ETL tool will populate the output record with the first nine LGC values encountered, and will insert a message into the output report stating that the Segment_LGC table contains more than nine entries for that SEGMENTID. (If and when that event should ever occur, it would be necessary for DCP to consider adding additional LGC fields to LION and expanding the length of the BOE Pointer field to two bytes.)
- If the given segment belongs to a non-Centerline feature, the tool will populate the output Board of Elections LGC Pointer field with the value ’1’. If it belongs to a Centerline feature, the tool will populate the output Board of Elections LGC Pointer with an integer corresponding to the output LGC field that has been loaded from the Segment_LGC entry that contains a ‘Y’ in the BOE_PREFERRED_LGC_FLAG. For example, if the output LGC field that has been loaded from that Segment_LGC entry is LGC3, the tool will populate the output Board of Elections LGC Pointer with the value ‘3’.
- If the segment belongs to a Centerline feature:
    - The tool will populate the output Face Code field from the FACECODE field in the StreetName table entry (if found) for the principal street name corresponding to the preferred B7SC.
    - If the preferred B7SC is not found in StreetName, the tool will populate the output Face Code field from the FACECODE field in the FeatureName table entry (if any) for the principal feature name corresponding to the preferred B7SC.
    - If the preferred B7SC is not found in StreetName or in FeatureName, the tool will not write out an output record, and will issue an error message.

- If the segment belongs to a non-Centerline feature class, the tool will populate the output Face Code field from the FACECODE field in the FeatureName table entry for the principal feature name corresponding to the preferred B7SC.

2.2.2. Node-Related Items

FIELDS: L12 (FROM-NODE), L16 (TO-NODE), L13 (FROM-X), L14 (FROM-Y), L17 (TO-X), L18 (TO-Y), L11 (FROM-SECTIONAL-MAP), L15 (TO-SECTIONAL-MAP):

METHODOLOGY:

- From the segment's spatial data, the tool will determine which endpoint of the given segment is its "from" endpoint and which is its "to" endpoint, and will populate the FROM-X, FROM-Y, TO-X and TO-Y output fields accordingly, rounded to the nearest whole integer.

- For each endpoint of the segment, the tool will perform a spatial query against the Node feature class to obtain that endpoint's NODEID. The tool will populate the FROM-NODE and TO-NODE output fields accordingly. If there is no node at a segment endpoint the ETL tool will populate that output node field with blanks and will issue an error message in the output report file.

- For each endpoint of the segment, the tool will perform a spatial query against the SectionalMap polygon feature class to obtain that endpoint's Sectional Map identifier. The tool will populate the FROM-SECTIONAL-MAP and TO-SECTIONAL-MAP output fields accordingly.

2.2.3. Atomic Polygon-Based Items

FIELDS: L19 (LEFT-2000-CENSUS-TRACT), L28 (RIGHT-2000-CENSUS-TRACT), L20 (LEFT-DYNAMIC-BLOCK), L29 (RIGHT-DYNAMIC-BLOCK), L25 (LEFT-ASSEMBLY-DISTRICT), L26 (LEFT-ELECTION-DISTRICT), L34 (RIGHT-ASSEMBLY-DISTRICT), L35 (RIGHT-ELECTION-DISTRICT), L27 (LEFT-SCHOOL-DISTRICT), L36 (RIGHT-SCHOOL-DISTRICT), L72 (LEFT-2010-CENSUS-TRACT), L73 (RIGHT-2010-CENSUS-TRACT), L80, L81, L82, L83, L84, L85, L86, L87, L100, L101, L102, L103, L104, L105:

METHODOLOGY:

First, the ETL tool will determine the segment's left and right Atomic Polygons (APs). Certain attributes of those APs will then be used to populate the corresponding fields of the output LION record, as indicated in Table 4. In addition, certain attributes of the left and right APs will be used in determining how to populate output fields L41 (Segment Locational Status), L42 (Feature Type Code) and L46 (Borough Boundary Indicator), as described in Sections 2.2.6, 2.2.7 and 2.2.8, respectively. Therefore, the processing described in those sections must be performed in conjunction with the processing described in this section.

To determine the given segment's left and right APs, the ETL tool will generate temporary left and right "offset points" positioned on both sides of the segment, offset two feet perpendicularly from the segment's midpoint. (Note: the same offset points will be used

to populate the left and right zip code output fields for non-Centerline segments – see Section 2.3.4.) For each offset point, the tool will perform a point-in-polygon spatial query against the Atomic Polygon feature class to determine the given segment's left and right APs.

- If one offset point is not contained within an AP (presumably because the segment lies along the city limits), the output fields for that side of the segment will be populated with blanks.
- If both offset points fail to match to APs, the ETL tool will report the segment as an error with an appropriate message.

Table 4 – Populating Atomic Polygon-Based Fields

FIC	Output Field	CSCL Source
L19	Left 2000 Census Tract	CENSUSTRACT_2000 attribute of left AP
L20	Left Dynamic Block	Last three bytes of ATOMICID attribute of left AP
L25	Left Assembly District	ASSEMDIST attribute of left AP
L26	Left Election District	ELECTDIST attribute of left AP
L27	Left School District	SCHOOLDIST attribute of left AP
L28	Right 2000 Census Tract	CENSUSTRACT_2000 attribute of right AP
L29	Right Dynamic Block	Last three bytes of ATOMICID attribute of right AP
L34	Right Assembly District	ASSEMDIST attribute of right AP
L35	Right Election District	ELECTDIST attribute of right AP
L36	Right School District	SCHOOLDIST attribute of right AP
L72	Left 2010 Census Tract	CENSUSTRACT_2010 attribute of AP on left side of segment
L73	Right 2010 Census Tract	CENSUSTRACT_2010 attribute of AP on right side of segment
L80	LEFT CENSUS BLOCK 2000 BASIC	CENSUSBLOCK_2000 attribute of AP on left side of segment
L81	LEFT CENSUS BLOCK 2000 SUFFIX	CENSUSBLOCK_2000 suffix attribute of AP on left side of segment
L82	RIGHT CENSUS BLOCK 2000 BASIC	CENSUSBLOCK_2000 attribute of AP on right side of segment
L83	RIGHT CENSUS BLOCK 2000 SUFFIX	CENSUSBLOCK_2000 suffix attribute of AP on right side of segment
L84	LEFT CENSUS BLOCK 2010 BASIC	CENSUSBLOCK_2010 attribute of AP on left side of segment

FIC	Output Field	CSCL Source
L85	LEFT CENSUS BLOCK 2010 SUFFIX	CENSUSBLOCK_2010 suffix attribute of AP on left side of segment
L86	RIGHT CENSUS BLOCK 2010 BASIC	CENSUSBLOCK_2010 attribute of AP on right side of segment
L87	RIGHT CENSUS BLOCK 2010 SUFFIX	CENSUSBLOCK_2010 suffix attribute of AP on right side of segment
L100	LEFT 2020 CENSUS TRACT	CENSUSTRACT_2020 attribute of left AP
L101	RIGHT 2020 CENSUS TRACT	CENSUSTRACT_2020 attribute of right AP
L102	LEFT CENSUS BLOCK 2020 BASIC	CENSUSBLOCK_2020 attribute of AP on left side of segment
L103	LEFT CENSUS BLOCK 2020 SUFFIX	CENSUSBLOCK_2020 suffix attribute of AP on left side of segment
L104	RIGHT CENSUS BLOCK 2020 BASIC	CENSUSBLOCK_2020 attribute of AP on right side of segment
L105	RIGHT CENSUS BLOCK 2020 SUFFIX	CENSUSBLOCK_2020 suffix attribute of AP on right side of segment

If output field L46 (Borough Boundary Indicator) is populated with a non-blank value, that indicates that the given segment is a borough boundary segment. In that case, L46 will contain one of the values 'L' or 'R', indicating the side of the segment that is in a different borough from the one that the segment itself belongs to. For borough boundary segments, only those output fields listed in Table 4 that pertain to the side of the street in the borough to which the segment itself belongs will be populated. The fields for the side out of the borough will be populated with blanks. For example, if L46 contains an 'L' (Left side out of borough), the "right" fields will be populated with values and the "left" fields will be populated with blanks.

The ETL tool should determine and save the values of the fields listed in Table 4 for the side that is out of the borough, for use in generating an output record for the proto-segment that corresponds to the given borough boundary segment, as discussed in Section 2.6 et seq.

2.2.4. Split Election District Flag (Joe does not use this field anymore)

FIELD: L37 (Split Election District Flag)

METHODOLOGY:

The value that the ETL tool will populate into L37 will be based on the existence (or lack) of SEDAT table entries that pertain to the given Centerline segment, or SpecialSEDAT entries that pertain to SAF data that in turn pertains to the Centerline segment; and if there do exist any such entries, it will be based on which side(s) of the segment those entries pertain to.

Determining what SEDAT and SpecialSEDAT entries pertain to a Centerline segment, if any, must take into consideration the possibility that the segment might have a proto-segment, that is, a coincident segment entry in the AltSegmentData table that has the same SEGMENTID as that of the Centerline segment. (Proto-segments are discussed in detail in Section 2.6 et seq.)

If a Centerline segment does have a proto-segment, both the Centerline segment and the proto-segment are either borough boundary street segments or combined on-off ramps. Ramps will never have associated address information. Therefore, if there is a SEDAT or SpecialSEDAT entry that contains the SEGMENTID of a Centerline segment that has a proto-segment, they must both be borough boundary street segments. In that case, the SEDAT or SpecialSEDAT entry pertains to the entity that belongs to the same borough as that entry. The ETL tool will determine this by comparing the borough code in the first byte of the LIONKEY of the SEDAT or SpecialSEDAT entry with field L1 (Borough) in the output record being generated for the given Centerline segment and proto-segment.

Since it is necessary, for the purpose of populating field L37, to compare LION keys when an output LION record is being generated from a Centerline segment that has a proto-segment, or from a proto-segment whose implicit feature class (defined in Section 2.6) is Centerline, the processing to populate the components of the output record's LION Key (fields L1, L2 and L3) must be performed before L37 can be populated.

Once all SEDAT and SpecialSEDAT entries that pertain to the Centerline segment (if any) are ascertained, the rules for populating L37 for that Centerline segment are as follows:

- If there are no entries in the CSCL SEDAT table or SpecialSEDAT table that pertain to the Centerline segment, the tool will set L37 to blank.
- If there are only entries pertaining to the Centerline segment in which the field SIDE contains the value '1', the tool will set L37 to 'L'.
- If there are only entries pertaining to the Centerline segment in which SIDE contains the value '2', the tool will set L37 to 'R'.
- If there are entries pertaining to the Centerline segment in which SIDE contains a '1' as well as entries in which it contains a '2', the tool will set L37 to 'B'.

When creating the output LION record for a proto-segment whose implicit feature class is Centerline, the above rules apply but are based on the set of SEDAT and SpecialSEDAT entries that pertain to the proto-segment rather than to the Centerline segment.

2.2.5. Traffic Direction

FIELD: L40 (Traffic Direction)

METHODOLOGY:

L40 will be populated by translating the Centerline attribute TRAFDIR as shown in Table 5.

Table 5 - Translation Rules for L40 (Traffic Direction)

TRAFDIR Value	Output Value
blank	blank
FT (with)	W
TF(against)	A
NV(non-vehicular)	P
TW(two-way)	T

2.2.6. Segment Locational Status

FIELD: L41 (Segment Locational Status)

METHODOLOGY:

The processing described in Section 2.2.3 must be performed prior to performing the processing for field L41.

For segments of all linear feature types, the ETL tool will populate L41 based on certain assignment rules listed below. These rules involve the attributes ATOMICID, BOROUGH and ~~CENSUSTRACT_2000~~ CENSUSTRACT_2020 of the segment's left and right Atomic Polygons, which have been determined as described in Section 2.2.3.

The assignment rules for field L41 are as follows:

- If, during the processing described in Section 2.2.3, one of the segment's left and right offset points was found not to be contained in any AP, then the segment lies along New York City's boundary with one of its neighboring jurisdictions (Nassau County, Westchester County or New Jersey). The ETL tool will populate L41 with the value '9'.

- If the ATOMICIDs of the left and right AP's are identical, the ETL tool will populate L41 with one of the values 'H' or 'I', as follows. It will assign the value 'H' if there is at least one other segment of any kind connected to <u>each</u> of the given segment's two endpoints. If the segment has at least one endpoint to which no other segments are connected, it will assign the value 'I'.

- If the left and right AP's belong to two different boroughs, the ETL tool will populate L41 with the borough code that differs from that of the segment itself. For example, if the segment contains a Borough Code of '3' (= Brooklyn), and the Borough Code of the left AP is '3', and the Borough Code of the right AP is '4' (= Queens), then the segment is a Brooklyn segment lying along the border with Queens; the ETL tool will populate L41 with the value '4'.

- If the left and right AP's belong to the same borough, but the ~~CENSUSTRACT_2000~~ CENSUSTRACT2020 attributes of the left and right AP's differ, the ETL tool will populate L41 with the value 'X'.

- If none of the above conditions are satisfied, the ETL tool will populate L41 with a blank.

Centerline segments (and those of all other linear feature classes) have an attribute field called SEGLOCSTATUS. The ETL tool will validate this attribute for consistency with the above rules. If SEGLOCSTATUS is inconsistent with the rules, the ETL tool will populate output field L41 with the value specified in the rules rather than the value in SEGLOCSTATUS, but will report the discrepancy as an error. (The CSCL Maintenance System has not been designed to perform this validation on the fly, since it is believed that doing so would cause performance problems for the interactive maintenance routines.)

2.2.7. Feature Type Code

FIELD: L42 (Feature Type Code)

METHODOLOGY:

The possible values for field L42 for Centerline segments, and the rules for assigning each such value, are listed in Table 6. Add new Feature Type Code ="C". if Street Status =2 (constructed) and "RWJURISDICTION" = 5 (CCO), then street type = C.

Table 6 – Assignment Rules for L42 (Feature Type Code)

Value	Definition	Assignment Rule
Blank	Public street, bridge or tunnel that exists physically (or its generic geometry), other than Feature Type Code W	Assign if segment does not satisfy the criteria listed in this table for Feature Type Codes 5, 6, 9, A or W
5	Paper street that is not also a boundary	Assign if STATUS = 3.
6	Private street that exists physically	Assign if RWJURISDICTION = 3 (private) and STATUS = 2 (constructed)
9	Paper street that coincides with a non-physical boundary	Assign if STATUS = 9.
A	Alley	Assign if RW_TYPE = 10
W	Path, non-vehicular, addressable	Assign if TRAFDIR = NV (non-vehicular) and at least one house number field is non-zero.

Value	Definition	Assignment Rule
F	Ferry Route	Assign if RW_type=14
C	Constructed	Assign if "STATUS" = 2 & "RWJURISDICTION"=5(CCO)

2.2.8. Borough Boundary Indicator

FIELD: L46 (Borough Boundary Indicator)

METHODOLOGY:

The ETL tool will perform the processing described in this section in conjunction with the processing described in Section 2.2.3.

The ETL tool will populate L46 by comparing the borough attributes of the segment's left and right Atomic Polygons (if any) to the borough attribute of the segment itself, as follows:

- If there is no AP on one side of the given segment, the segment lies along the City limit. The tool will populate the Borough Boundary Indicator with the value ('L' or 'R') that identifies the side of the segment on which there is no AP. (Note: If there is no AP on either side of the segment, that condition will be identified and reported as an error during the processing for AP-related output fields described in Section 2.2.3.)

- If there are APs on both sides of the segment, the tool will validate that at least one of the AP boroughs is the same as that of the given segment. If neither AP borough matches the segment's borough, that will be reported as an error.

- If the segment's borough matches both AP boroughs, the segment is not a borough boundary segment. The tool will populate L46 with a blank.

If the segment's borough matches only one of the AP boroughs, the tool will populate L46 with the value ('L' or 'R') that identifies the side of the segment whose AP is in the borough that differs from that of the segment.

2.2.9. Twisted Parity Flag

FIELD: L47 (Twisted Parity Flag)

METHODOLOGY:

The ETL tool will populate L47 by translating the Centerline attribute TWISTED_PARITY_FLAG as specified in Table 7.

Table 7 – Translation Rules for L47 (Twisted Parity Flag)

Source Field Value	Output Field Value
Y	T
N	blank

2.2.10. Special Address Flag

FIELD: L48 (Special Address Flag)

METHODOLOGY:

The value that will be populated into L48 depends on the types of SAF data that pertain to the segment. Determining what SAF data, if any, pertain to a segment in a linear feature class must take into consideration the possibility that the segment might have a proto-segment, that is, a coincident segment entry in the AltSegmentData table that has the same SEGMENTID as that of the geometry-based segment. (Proto-segments are discussed in detail in Section 2.6 et seq.)

If the geometry-based segment does have a proto-segment, SAF data identified to the given SEGMENTID could pertain to either the geometry-based segment or the proto-segment. Which one it pertains to is determined as follows:

- If the proto-segment is of the borough boundary type (ALT_SEGDATA_TYPE = B in the AltSegmentData entry for the proto-segment), the SAF datum pertains to whichever entity, the geometry-based segment or the proto-segment, has the same borough value as the SAF datum.

- If the proto-segment is of the combined on-off ramp type (ALT_SEGDATA_TYPE = R):

    - It will not have any appurtenant SAF type A, B, C, D, E, F, O, P, S or V data, since these are all address-related and ramps will never have addresses.

    - If the SAF datum is stored in CommonPlace (SAFTYPE = G, N or X), it will be treated as pertaining to the geometry-based segment.

- If the proto-segment is of the boundary continuity type (ALT_SEGDATA_TYPE = C), then the SAF datum pertains to the geometry-based segment. Boundary continuity proto-segments are non-physical non-street features and will never have SAF data of any kind.

Once all the SAF data that pertain to the segment (if any) are determined, the rules for populating L48 for that segment are as follows:

- If there are no SAF data of any type that pertain to the given segment, populate L48 with a blank.

- If there is at least one type S datum pertaining to the given segment, populate L48 with the value ‘S’, regardless of what other SAF type data are associated with the

segment, if any. A type S datum is a feature in the AddressPoint feature class for which the SPECIAL_CONDITION attribute contains an 'S'.

- If there are no type S data pertaining to the given segment, but there is at least one type N datum, populate L48 with the value 'N'. A type N datum is a feature in the CommonPlace feature class for which the SAFTYPE attribute contains an 'N'.

- If there are no type S or N data pertaining to the given segment, but there is at least one type G datum, populate L48 with the value 'G'. A type G datum is a feature in the CommonPlace feature class for which the SAFTYPE attribute contains a 'G'.

- If there are no type S, N or G data for the given segment, but there is at least one SAF datum of any other type, populate L48 with the first SAFTYPE value encountered. The possibilities are type V in the AddressPoint feature class, type X in the CommonPlace feature class, and types A, B, C, D, E, F, O and P in the AltSegmentData table.

2.2.11. Curve-Related Fields

FIELD: L49 (Curve Flag), L50 (Center of Curvature X-Coordinate), L51 (Center of Curvature Y-Coordinate)

METHODOLOGY:

Curved segments are modeled in the CSCL database as polylines containing one or more or "shape points" between the segment's endpoints. The Centerline feature class has an attribute called CURVE, which contains one of the values 'I', 'L' or 'R' if the Centerline segment represents a curved real-world feature, and contains the value 'S' otherwise.

There are two types of such "logically curved" Centerline segments. "Regular" curves are arcs of circles; "irregular" curves are all other curves. For regularly curved segments, the CURVE attribute contains one of the values 'L' or 'R'. For irregular curves, CURVE contains the value 'I'.

For output records originating from Centerline segments, L49 will be populated with the value in CURVE if it is not equal to 'S', or with a blank if CURVE contains 'S'.

The fields L50 and L51 will be populated with non-blank values only for regular curved segments, that is, for output LION records in which L49 has been populated with one of the values 'L' or 'R'. In all other output records, L50 and L51 will be populated with blanks.

Since three points determine a circle, for regular curved segments, the spatial coordinates of the center of curvature can be computed by using the spatial coordinates of the "from" node, the "to" node and any one of the intermediate shape points. It appears advisable for the ETL tool to use a shape point that is positioned as midway between the two endpoints of the segment as possible. If there is an odd number of shape points, the central one will be used. If there is an even number, either one of the two central shape points could be used. For consistency in the odd case, the central shape point closest to the "from" node will be used.

The computation of the spatial coordinates of the center of the circle is straightforward. The general equation of a circle is:

$$(x - x_0)^2 + (y - y_0)^2 = r^2$$

where (x_0, y_0) is the center of the circle and r is the radius. Since the "from" node, the "to" node and the selected shape point all lie on the desired circle, the coordinate pairs of all three of those points satisfy the above equation. That is enough information to solve for the unknowns x_0 and y_0, which are the required Center of Curvature Spatial Coordinates. The results must be rounded to the nearest foot and then populated into fields L50 and L51.

2.2.12. Level Codes

FIELDS: L53 ("From" Level) and L54 ("To" Level)

METHODOLOGY:

In LION, level codes are either a single letter of the alphabet or an asterisk. An asterisk signifies that the feature is non-physical and therefore is "level-less".

In the CSCL database, level codes are two-digit numeric values ranging from 01 to 26, along with the value 99, which is assigned to the level codes of non-physical features and also to physical features in the NonStreetFeature feature class. The values from 01 to 26 correspond in LION to the respective letters of the alphabet. The value 99 corresponds to the LION value of asterisk.

Except for Shoreline segments (see Section 2.3), the ETL tool will populate L53 and L54 by converting the corresponding CSCL numeric level codes into letters of the alphabet and asterisks as follows:

CSCL Level Code	Output LION Level Code
01	A
02	B
03	C
…..	…..
etc.	etc.
…..	…..
24	X
25	Y
26	Z
99	* (asterisk)

2.2.13 NYPD Service Area

FIELDS: L97 ("Left NYPD Service Area) and L98 (Right NYPD Service Area)

METHODOLOGY:

First, the ETL tool will determine the segment's left and right NYPD Service Area.

To determine the given segment's left and right NYPD Service Area, the ETL tool will generate temporary left and right "offset points" positioned on both sides of the segment, offset two feet perpendicularly from the segment's midpoint. For each offset point, the tool will perform a point-in-polygon spatial query against the NYPD BEAT feature class to determine the given segment's left and right NYPD Service Area. The attribute will be the first byte of the attribute of "POST" of the found NYPD BEAT polygon when field of GEO_TYPE="HP".

- If one offset point is not contained within a NYPD BEAT polygon (presumably because the segment lies along the city limits), the output fields for that side of the segment will be populated with blanks.
- If both offset points fail to match to NYPD BEAT polygon, the ETL tool will report the segment as an error with an appropriate message.

2.3. Generating Records from Shoreline Segments

Table 8 and the sections following it indicate how each LION output field is populated for records generated from segments in the Shoreline feature class. When the last column in the table contains the phrase "Same as Centerline", the methodology for populating the given field is the same as that indicated in the last column of Table 8 or in the section cited in that column.

Table 8 – Methodologies for Shoreline Segments

FIC	Field	Populating Method
L1	Borough	See Section 2.3.1
L2	Face Code	Same as Centerline
L3	Sequence Number	Same as Centerline
L4	Segment ID	Same as Centerline
L5	5-Digit Street Code (5SC)	Same as Centerline
L6	LGC1	Same as Centerline
L7	LGC2	Same as Centerline
L8	LGC3	Same as Centerline
L9	LGC4	Same as Centerline
L10	Board of Elections LGC Pointer	Same as Centerline
L11	From-Sectional Map	Same as Centerline
L12	From-Node ID	Same as Centerline
L13	From-X Coordinate	Same as Centerline
L14	From-Y Coordinate	Same as Centerline
L15	To-Sectional Map	Same as Centerline
L16	To-Node ID	Same as Centerline
L17	To-X Coordinate	Same as Centerline
L18	To-Y Coordinate	Same as Centerline
L19	Left 2000 Census Tract	Same as Centerline
L20	Left Dynamic Block	Same as Centerline

FIC	Field	Populating Method
L21	Left Low House Number	Populate with six blanks followed by a zero
L22	Left High House Number	Populate with sic blanks followed by a zero
L23	Left Dept of Sanitation Subsection	Populate with blanks
L24	Left Zip Code	See Section 2.3.4
L25	Left Assembly District	Same as Centerline
L26	Left Election District	Same as Centerline
L27	Left School District	Same as Centerline
L28	Right 2000 Census Tract	Same as Centerline
L29	Right Dynamic Block	Same as Centerline
L30	Right Low House Number	Populate with zeros
L31	Right High House Number	Populate with zeros
L32	Right Dept of Sanitation Subsection	Populate with blanks
L33	Right Zip Code	See Section 2.3.4
L34	Right Assembly District	Same as Centerline
L35	Right Election District	Same as Centerline
L36	Right School District	Same as Centerline
L37	Split Election District Flag	Populate with a blank
L38	Filler (formerly Split Community School District Flag)	Populate with a blank
L39	Sanitation District Boundary Indicator	Populate with a blank
L40	Traffic Direction	Populate with a blank
L41	Segment Locational Status	See Section 2.3.2
L42	Feature Type Code	Populate with the value ‘2’
L43	Non-Pedestrian Flag	Populate with a blank
L44	Continuous Parity Indicator	Populate with a blank
L45	Filler (formerly the Near BQ-Boundary Flag)	Populate with a blank
L46	Borough Boundary Indicator	Same as Centerline
L47	Twisted Parity Flag	Populate with a blank
L48	Special Address Flag	Same as Centerline
L49	Curve Flag	See Section 2.3.3
L50	Center of Curvature X-Coordinate	Populate with blanks
L51	Center of Curvature Y-Coordinate	Populate with blanks
L52	Segment Length in Feet	Populate with value of Shoreline attribute SHAPE.LEN rounded to nearest foot
L53	From Level Code	Populate with ‘$’ (dollar sign)
L54	To Level Code	Populate with ‘$’ (dollar sign)
L55	Traffic Direction Verification Flag	Populate with a blank
L56	Segment Type Code	Populate with ‘U’
L57	Coincident Segment Counter	See Section 2.6.1
L58	Include/Exclude Flag	Populate with blank
L59	Roadway Type	Populate with blank
L60	PHYSICALID	Populate with blanks
L61	GENERICID	Populate with blanks
L62	NYPDID	Populate with blanks
L63	FDNYID	Populate with blanks
L64	~~Left BLOCKFACEID~~ Filler	Populate with blanks
L65	~~Right BLOCKFACEID~~ Filler	Populate with blanks
L66	STATUS	Populate with blank
L67	Street Width Min	Populate with blanks
L68	Irregular Street Width Flag	Populate with blank
L69_1	Bike Lane Indicator	Populate with blank
L70	FCC	Populate with blanks

FIC	Field	Populating Method
L71	Right of Way Type	Populate with blank
L72	Left 2010 Census Tract	Same as Centerline
L73	Right 2010 Census Tract	Same as Centerline
L74	LGC5	Same as Centerline
L75	LGC6	Same as Centerline
L76	LGC7	Same as Centerline
L77	LGC8	Same as Centerline
L78	LGC9	Same as Centerline
L79	Legacy SEGMENTID	Populate with blanks
L80	LEFT CENSUS BLOCK 2000 BASIC	Same as Centerline
L81	LEFT CENSUS BLOCK 2000 SUFFIX	Same as Centerline
L82	RIGHT CENSUS BLOCK 2000 BASIC	Same as Centerline
L83	RIGHT CENSUS BLOCK 2000 SUFFIX	Same as Centerline
L84	LEFT CENSUS BLOCK 2010 BASIC	Same as Centerline
L85	LEFT CENSUS BLOCK 2010 SUFFIX	Same as Centerline
L86	RIGHT CENSUS BLOCK 2010 BASIC	Same as Centerline
L87	RIGHT CENSUS BLOCK 2010 SUFFIX	Same as Centerline
L88	SNOW PRIORITY	Populate with blanks
L69_2	BIKELANE_2	Populate with blanks
L67_2	STREET WIDTH MAX	Populate with blanks
L89	Filler	3 digits
L90	LEFT BLOCK FACE ID	Populate with blanks
L91	RIGHT BLOCK FACE ID	Populate with blanks
L92	NUMBER TRAVEL LANE	Populate with blanks
L93	NUMBER PARK LANES	Populate with blanks
L94	NUMBER TOTAL LANES	Populate with blanks
L95	BIKE TRAFFIC DIR	Populate with blanks
L96	POSTED SPEED	Populate with blanks
L97	Left NYPD Service Area	Same as Centerline
L98	Right NYPD Service Area	Same as Centerline
L99	Truck Route Type	Populate with blanks
L100	LEFT 2020 CENSUS TRACT	Same as Centerline
L101	RIGHT 2020 CENSUS TRACT	Same as Centerline
L102	LEFT CENSUS BLOCK 2020 BASIC	Same as Centerline
L103	LEFT CENSUS BLOCK 2020 SUFFIX	Same as Centerline
L104	RIGHT CENSUS BLOCK 2020 BASIC	Same as Centerline
L105	RIGHT CENSUS BLOCK 2020 SUFFIX	Same as Centerline
L199	Filler	blank

2.3.1. Borough

FIELD: L1 (Borough)

METHODOLOGY:

Unlike Centerline, the Shoreline feature class does not have an attribute for the Borough Code.

The ETL tool will access the Segment_LGC table by SEGMENTID. The tool will populate L1 with the first byte of the B5SC field in the given Segment_LGC table entry. (Note: Segment_LGC can contain more than one entry for a given SEGMENTID value. However, all such entries will contain the same B5SC value. Therefore, it is immaterial from which table entry the ETL tool obtains the Borough Code.)

For performance efficiency, the Borough Code value should be obtained at the same time as the ETL tool is accessing the Segment_LGC table to populate fields L2 and L5 through L9.

2.3.2. Segment Locational Status

FIELD: L41 (Segment Locational Status)

METHODOLOGY:

L41 is populated via the same methodology for Shoreline as for Centerline, including the consistency validation described in Section 2.2.6.

2.3.3. Curve Flag

FIELD: L49 (Curve Flag)

METHODOLOGY:

Unlike Centerline, the other CSCL linear feature classes do not have a CURVE attribute.

For output records originating from segments in the Shoreline and other non-Centerline feature classes, L49 will be populated based on whether the segment has any shape points. If so, L49 will be populated with the value 'I'; otherwise, it will be populated with a blank.

2.3.4. Zip Codes

FIELDS: L24 (Left Zip Code) and L33 (Right Zip Code)

METHODOLOGY:

Unlike the Centerline feature class, the Shoreline and other linear feature classes do not contain zip code attributes. However, the CSCL database does have a ZipCode polygon feature class. For output records generated from segments in the Shoreline and other non-Centerline linear feature classes, the fields L24 and L33 will be populated via spatial processing, as follows.

A point-in-polygon routine will be used to spatially match the left and right offset points computed as part of the determination of left and right Atomic Polygons (see Section 2.2.3) against the ZipCode polygons. Upon a match, the ZIP_CODE attribute of the matching

polygon will be populated into the respective zip code output field, either L24 or L33. If there is no match for one or both of the offset points, the respective output zip code field will be populated with blanks. Non-matches will not be treated as errors, since the zip code geography does not necessarily cover the entire territory of the city, particularly water areas.

2.4. Generating Records from Rail and Subway Segments

Table 9 indicates how each LION output field is populated for records generated from segments in the Rail and Subway feature classes. When the last column in the table contains the phrase "Same as Centerline" or "Same as Shoreline", the methodology for populating the given field is the same as that indicated in the last column of Table 3 or Table 8, respectively, or in the section cited in that column.

Table 9 – Methodologies for Rail and Subway Segments

FIC	Field	Populating Method
L1	Borough	Same as Shoreline
L2	Face Code	Same as Centerline
L3	Sequence Number	Same as Centerline
L4	Segment ID	Same as Centerline
L5	5-Digit Street Code (5SC)	Same as Centerline
L6	LGC1	Same as Centerline
L7	LGC2	Same as Centerline
L8	LGC3	Same as Centerline
L9	LGC4	Same as Centerline
L10	Board of Elections LGC Pointer	Same as Centerline
L11	From-Sectional Map	Same as Centerline
L12	From-Node ID	Same as Centerline
L13	From-X Coordinate	Same as Centerline
L14	From-Y Coordinate	Same as Centerline
L15	To-Sectional Map	Same as Centerline
L16	To-Node ID	Same as Centerline
L17	To-X Coordinate	Same as Centerline
L18	To-Y Coordinate	Same as Centerline
L19	Left 2000 Census Tract	Same as Centerline
L20	Left Dynamic Block	Same as Centerline
L21	Left Low House Number	Populate with six blanks followed by a zero
L22	Left High House Number	Populate with six blanks followed by a zero
L23	Left Dept of Sanitation Subsection	Populate with blanks
L24	Left Zip Code	Same as Shoreline
L25	Left Assembly District	Same as Centerline
L26	Left Election District	Same as Centerline
L27	Left School District	Same as Centerline
L28	Right 2000 Census Tract	Same as Centerline
L29	Right Dynamic Block	Same as Centerline
L30	Right Low House Number	Populate with six blanks followed by a zero
L31	Right High House Number	Populate with six blanks followed by a zero
L32	Right Dept of Sanitation Subsection	Populate with blanks
L33	Right Zip Code	Same as Shoreline

FIC	Field	Populating Method
L34	Right Assembly District	Same as Centerline
L35	Right Election District	Same as Centerline
L36	Right School District	Same as Centerline
L37	Split Election District Flag	Populate with blanks
L38	Filler (formerly Split Community School District Flag)	Populate with blanks
L39	Sanitation District Boundary Indicator	Populate with a blank
L40	Traffic Direction	Populate with a blank
L41	Segment Locational Status	Same as Shoreline
L42	Feature Type Code	Populate with the value '1'
L43	Non-Pedestrian Flag	Populate with a blank
L44	Continuous Parity Indicator	Populate with a blank
L45	Filler (formerly the Near BQ-Boundary Flag)	Populate with a blank
L46	Borough Boundary Indicator	Same as Centerline
L47	Twisted Parity Flag	Populate with a blank
L48	Special Address Flag	Same as Centerline
L49	Curve Flag	Same as Shoreline
L50	Center of Curvature X-Coordinate	Populate with blanks
L51	Center of Curvature Y-Coordinate	Populate with blanks
L52	Segment Length in Feet	Populate with value of Rail and Subway attribute SHAPE.LEN rounded to nearest foot
L53	From Level Code	Same as Centerline
L54	To Level Code	Same as Centerline
L55	Traffic Direction Verification Flag	Populate with a blank
L56	Segment Type Code	Populate with 'U'
L57	Coincident Segment Counter	See Section 2.6.1
L58	Include/Exclude Flag	Populate with blank
L59	Roadway Type	Populate with blank
L60	PHYSICALID	Populate with blanks
L61	GENERICID	Populate with blanks
L62	NYPDID	Populate with blanks
L63	FDNYID	Populate with blanks
L64	Left BLOCKFACEID	Populate with blanks
L65	Right BLOCKFACEID	Populate with blanks
L66	STATUS	Populate with blank
L67	Street Width Min	Populate with blanks
L68	Irregular Street Width Flag	Populate with blank
L69_1	Bike Lane Indicator	Populate with blank
L70	FCC	Populate with blanks
L71	Right of Way Type	Copy from Rail or Subway attribute ROW_TYPE
L72	Left 2010 Census Tract	Same as Centerline
L73	Right 2010 Census Tract	Same as Centerline
L74	LGC5	Same as Centerline
L75	LGC6	Same as Centerline
L76	LGC7	Same as Centerline
L77	LGC8	Same as Centerline
L78	LGC9	Same as Centerline
L79	Legacy SEGMENTID	Same as Centerline
L80	LEFT CENSUS BLOCK 2000 BASIC	Same as Centerline
L81	LEFT CENSUS BLOCK 2000 SUFFIX	Same as Centerline
L82	RIGHT CENSUS BLOCK 2000 BASIC	Same as Centerline

FIC	Field	Populating Method
L83	RIGHT CENSUS BLOCK 2000 SUFFIX	Same as Centerline
L84	LEFT CENSUS BLOCK 2010 BASIC	Same as Centerline
L85	LEFT CENSUS BLOCK 2010 SUFFIX	Same as Centerline
L86	RIGHT CENSUS BLOCK 2010 BASIC	Same as Centerline
L87	RIGHT CENSUS BLOCK 2010 SUFFIX	Same as Centerline
L88	SNOW PRIORITY	Populate with blanks
L69_2	BIKELANE_2	Populate with blanks
L67_2	STREET WIDTH MAX	Populate with blanks
L89	Filler	3 digits
L90	LEFT BLOCK FACE ID	Populate with blanks
L91	RIGHT BLOCK FACE ID	Populate with blanks
L92	NUMBER TRAVEL LANE	Populate with blanks
L93	NUMBER PARK LANES	Populate with blanks
L94	NUMBER TOTAL LANES	Populate with blanks
L95	BIKE TRAFFIC DIR	Populate with blanks
L96	POSTED SPEED	Populate with blanks
L97	Left NYPD Service Area	Same as Centerline
L98	Right NYPD Service Area	Same as Centerline
L99	Truck Route Type	Populate with blanks
L100	LEFT 2020 CENSUS TRACT	Same as Centerline
L101	RIGHT 2020 CENSUS TRACT	Same as Centerline
L102	LEFT CENSUS BLOCK 2020 BASIC	Same as Centerline
L103	LEFT CENSUS BLOCK 2020 SUFFIX	Same as Centerline
L104	RIGHT CENSUS BLOCK 2020 BASIC	Same as Centerline
L105	RIGHT CENSUS BLOCK 2020 SUFFIX	Same as Centerline
L199	Filler	blank

2.5. Generating Records from NonStreetFeature Segments

Table 10 and the following sections indicate how each LION output field is populated for records generated from segments in the NonStreetFeature (NSF) feature class. When the last column in the table contains the phrase “Same as Centerline” or “Same as Shoreline”, the methodology for populating the given field is the same as that indicated in the last column of Table 3 or Table 8, respectively, or in the section cited in that column.

Table 10 – Methodologies for NonStreetFeature Segments

FIC	Field	Populating Method
L1	Borough	Same as Shoreline
L2	Face Code	Same as Centerline
L3	Sequence Number	See Section 2.5.1
L4	Segment ID	Same as Centerline
L5	5-Digit Street Code (5SC)	Same as Centerline
L6	LGC1	Same as Centerline
L7	LGC2	Same as Centerline
L8	LGC3	Same as Centerline

FIC	Field	Populating Method
L9	LGC4	Same as Centerline
L10	Board of Elections LGC Pointer	Same as Centerline
L11	From-Sectional Map	Same as Centerline
L12	From-Node ID	Same as Centerline
L13	From-X Coordinate	Same as Centerline
L14	From-Y Coordinate	Same as Centerline
L15	To-Sectional Map	Same as Centerline
L16	To-Node ID	Same as Centerline
L17	To-X Coordinate	Same as Centerline
L18	To-Y Coordinate	Same as Centerline
L19	Left 2000 Census Tract	Same as Centerline
L20	Left Dynamic Block	Same as Centerline
L21	Left Low House Number	Populate with zeros
L22	Left High House Number	Populate with zeros
L23	Left Dept of Sanitation Subsection	Populate with blanks
L24	Left Zip Code	Same as Shoreline
L25	Left Assembly District	Same as Centerline
L26	Left Election District	Same as Centerline
L27	Left School District	Same as Centerline
L28	Right 2000 Census Tract	Same as Centerline
L29	Right Dynamic Block	Same as Centerline
L30	Right Low House Number	Populate with six blanks followed by a zero
L31	Right High House Number	Populate with six blanks followed by a zero
L32	Right Dept of Sanitation Subsection	Populate with blanks
L33	Right Zip Code	Same as Shoreline
L34	Right Assembly District	Same as Centerline
L35	Right Election District	Same as Centerline
L36	Right School District	Same as Centerline
L37	Split Election District Flag	Populate with blanks
L38	Filler (formerly Split Community School District Flag)	Populate with blanks
L39	Sanitation District Boundary Indicator	Populate with blank
L40	Traffic Direction	Populate with blank
L41	Segment Locational Status	Same as Shoreline
L42	Feature Type Code	See Section 2.5.2
L43	Non-Pedestrian Flag	Populate with blank
L44	Continuous Parity Indicator	Populate with blank
L45	Filler (formerly the Near BQ-Boundary Flag)	Populate with blank
L46	Borough Boundary Indicator	Same as Centerline
L47	Twisted Parity Flag	Populate with blank
L48	Special Address Flag	Same as Centerline
L49	Curve Flag	Same as Shoreline
L50	Center of Curvature X-Coordinate	Populate with blanks
L51	Center of Curvature Y-Coordinate	Populate with blanks
L52	Segment Length in Feet	Populate with value of NonStreetFeature attribute SHAPE.LEN rounded to nearest foot
L53	From Level Code	Populate with an asterisk (*) – see Section 2.5.3
L54	To Level Code	Populate with an asterisk (*) – see Section 2.5.3
L55	Traffic Direction Verification Flag	Populate with blank
L56	Segment Type Code	Populate with ‘U’
L57	Coincident Segment Counter	See Section 2.6.1
L58	Include/Exclude Flag	Populate with blank

FIC	Field	Populating Method
L59	Roadway Type	Populate with blank
L60	PHYSICALID	Populate with blanks
L61	GENERICID	Populate with blanks
L62	NYPDID	Populate with blanks
L63	FDNYID	Populate with blanks
L64	~~Left BLOCKFACEID~~ Filler	Populate with blanks
L65	~~Right BLOCKFACEID~~ Filler	Populate with blanks
L66	STATUS	Populate with blank
L67	Street Width Min	Populate with blanks
L68	Irregular Street Width Flag	Populate with blank
L69_1	Bike Lane Indicator	Populate with blank
L70	FCC	Populate with blanks
L71	Right of Way Type	Copy from Rail or Subway attribute ROW_TYPE
L72	Left 2010 Census Tract	Same as Centerline
L73	Right 2010 Census Tract	Same as Centerline
L74	LGC5	Same as Centerline
L75	LGC6	Same as Centerline
L76	LGC7	Same as Centerline
L77	LGC8	Same as Centerline
L78	LGC9	Same as Centerline
L79	Legacy SEGMENTID	Same as Centerline
L80	LEFT CENSUS BLOCK 2000 BASIC	Same as Centerline
L81	LEFT CENSUS BLOCK 2000 SUFFIX	Same as Centerline
L82	RIGHT CENSUS BLOCK 2000 BASIC	Same as Centerline
L83	RIGHT CENSUS BLOCK 2000 SUFFIX	Same as Centerline
L84	LEFT CENSUS BLOCK 2010 BASIC	Same as Centerline
L85	LEFT CENSUS BLOCK 2010 SUFFIX	Same as Centerline
L86	RIGHT CENSUS BLOCK 2010 BASIC	Same as Centerline
L87	RIGHT CENSUS BLOCK 2010 SUFFIX	Same as Centerline
L88	SNOW PRIORITY	Populate with blanks
L69_2	BIKELANE_2	Populate with blanks
L67_2	STREET WIDTH MAX	Populate with blanks
L89	Filler	3 digits
L90	LEFT BLOCK FACE ID	Populate with blanks
L91	RIGHT BLOCK FACE ID	Populate with blanks
L92	NUMBER TRAVEL LANE	Populate with blanks
L93	NUMBER PARK LANES	Populate with blanks
L94	NUMBER TOTAL LANES	Populate with blanks
L95	BIKE TRAFFIC DIR	Populate with blanks
L96	POSTED SPEED	Populate with blanks
L97	Left NYPD Service Area	Same as Centerline
L98	Right NYPD Service Area	Same as Centerline
L99	Truck Route Type	Populate with blanks
L100	LEFT 2020 CENSUS TRACT	Same as Centerline
L101	RIGHT 2020 CENSUS TRACT	Same as Centerline
L102	LEFT CENSUS BLOCK 2020 BASIC	Same as Centerline
L103	LEFT CENSUS BLOCK 2020 SUFFIX	Same as Centerline
L104	RIGHT CENSUS BLOCK 2020 BASIC	Same as Centerline
L105	RIGHT CENSUS BLOCK 2020 SUFFIX	Same as Centerline

FIC	Field	Populating Method
L199	Filler	blank

2.5.1. Sequence Number

FIELD: L3 (Sequence Number)

METHODOLOGY:

Unlike the Centerline, Rail, Subway and Shoreline feature classes, which have a SEGMENT_SEQNUM attribute field that will be maintained "on the fly" (that is, assigned at the time that the given segment is created), the NonStreetFeature feature class does not have such an attribute.

For output LION records generated from the NonStreetFeature feature class, or generated from AltSegmentData coincident segment entries whose implicit feature class is NonStreetFeature, the ETL tool will assign Sequence Number values based on the combination of Borough and Face Code, but not based on relative spatial positions of the segments. That is, the Sequence Number assigned to each output record will be unique within the set of LION records containing the given Borough and Face Code values. Unlike the Sequence Numbers assigned to the segments of the other linear feature types, those that the ETL tool assigns to NonStreetFeature types of segments will bear no relationship to the spatial positioning of the given segment relative to other segments for the given Borough and Face Code.

Note: It is expected that, at some future time, sequence numbers will be converted to be based on B5SC instead of on Borough and Face Code. If and when that is done, the ETL tool's routine to generate sequence numbers for NonStreetFeature segments will have to be modified accordingly.

2.5.2. Feature Type Code

FIELD: L42 (Feature Type Code)

METHODOLOGY:

The possible values for field L42 and the rules for assigning them are listed in Table 11.

Table 11 – Assignment Rules for L42 (Feature Type Code)

Value	Definition	Assignment Rule
3	Non-physical census block boundary	Assign if LINETYPE = 3
4	Other non-street feature	Assign if LINETYPE = 7

Value	Definition	Assignment Rule
7	Non-physical boundary other than census	Assign if LINETYPE = 1, 2 or 6
8	Physical boundary such as cemetery wall	Assign if LINETYPE = 4 or 5

Currently, all non-street features are either physical or are district boundaries. Therefore, if a segment satisfies the assignment rule for Feature Type Code '4', it is likely either that the segment has been assigned an erroneous LINETYPE value, or it has been stored in an inappropriate feature class. The ETL tool will write out an output LION record, but will also write a warning message in the output report.

It is not anticipated that any features will be added in the future to the NonStreetFeature feature class that are neither physical features nor district boundaries. If that should ever occur, it would be advisable to consider modifying the ETL tool so that it no longer issues the warning message for Feature Type Code '4'.

2.5.3. Level Codes

FIELDS: L53 ("From" Level Code) and L54 ("To" Level Code)

NOTE: The ETL tool will populate L53 and L54 with asterisks for all NonStreetFeature (NSF) segments, including those that are physical (specifically, those with the LINETYPE value '5'). No level code attribute fields are defined in the NSF feature class. Therefore, it is not currently possible to extract level codes for physical NSF features. If that should ever be deemed to be required, a data model enhancement would have to be made to add level code fields to the NSF feature class, those fields would have to be appropriately populated and maintained, and the ETL tool would have to be modified accordingly.

2.6. Coincident Segments

Except for multi-decked physical features (such as a multi-level bridge or highway, or an elevated highway or rail line running along the bed of a surface street), coincident segments are modeled differently in the CSCL database than in LION.

- In LION, all coincident segments are represented by coincident geometry.
- In the CSCL database, as in LION, the segments of multi-decked physical features are represented by coincident geometry. Each of the segments has a distinct SEGMENTID.
- In LION, in addition to multi-decked physical features, there are three other types of coincident segments: borough boundary segments, combined "on" and "off" highway ramp segments, and boundary continuity segments. Such coincident segments exist in sets in which all the segments have the same SEGMENTID.

(Typically, such sets consist of two segments, but there can be more than two.) In the CSCL database, these coincident sets are modeled as follows:

- Only one segment of the set is represented in a linear feature class. It is referred to in this document as the "geometry-modeled segment".

- The other segment or segments of the set are represented in the CSCL database only by entries in the AltSegmentData table, so that they have no geometry representation of their own in the database. In that sense, in CSCL, they are "latent" segments. When the ETL tool extracts these into LION records, they will be transformed into actual segments (with geometry). The AltSegmentData entries are referred to as "proto-segments". Tacitly, proto-segments have the same geometry as the geometry-modeled segments with which they coincide, except that they may have the reverse topological (from–to) direction.

The ETL tool will generate an output LION record for a coincident segment entry in AltSegmentData only if a geometry-modeled segment exists with the same SEGMENTID. Otherwise, the ETL tool cannot generate a LION record, since the geometry to do so is lacking. However, it is expected that there will never be any such "orphaned" proto-segments, since the CSCL Maintenance System has been designed to prevent them from being created.

To aid in optimizing performance, the ETL tool is designed so that each time it generates an output record for a geometry-modeled segment, it immediately determines whether there is a corresponding proto-segment or proto-segments, and if there are any, it immediately generates an output record for each such proto-segment. This leverages the processing performed to populate fields in the output record for the geometry-based segment without later having either to repeat those algorithms or to retrieve the output record for the geometry-based segment.

Table 12 lists the fields in AltSegmentData.

Table 12 – AltSegmentData Table Layout

Field	Comments
OBJECTID	
SEGMENTID	
LIONKEY	
SOS_INDICATOR	For proto-segments, this field serves as a continuous parity indicator.
SAFTYPE	
L_LOW_HN	
L_HIGH_HN	
R_LOW_HN	
R_HIGH_HN	
LOW_HN_SUFFIX	

Field	Comments
HIGH_HN_SUFFIX	
LGC1	
LGC2	
LGC3	
LGC4	
LGC5	
LGC6	
LGC7	
LGC8	
LGC9	
BOROUGH	
ZIPCODE	
CREATED_BY	
CREATED_DATE	
MODIFIED_BY	
MODIFIED_DATE	
ALT_SEGMENT_SEQNUM	
BOE_PREFERRED_LGC_FLAG	
B5SC	
FEATURE_TYPE	See Table 14 for domain
FROM_TO_INDICATOR	See Table 16 for domain
ALT_SEGDATA_TYPE	See Table 13 for domain
TWISTED_PARITY_FLAG	Populated for proto-segments only, not SAF entries

AltSegmentData contains two types of entries: SAF entries and coincident segment entries. The field ALT_SEGDATA_TYPE indicates the type, as shown in Table 13.

Output Geosupport LION records will not be generated for SAF entries in AltSegmentData (entries with ALT_SEGDATA_TYPE = S), but they will be utilized to determine how to populate field L48 (Special Address Flag) in output LION records. They will also be utilized when extracting ESRI LION, ~~MapInfo LION and~~ SAF files.

Table 13 – AltSegmentData Table: ALT_SEGDATA_TYPE Field

Value	Meaning
B	Coincident borough boundary segment
C	Coincident boundary continuity segment
R	Coincident on-off ramp
S	SAF entry

To support the extraction of LION records for proto-segments, it has been necessary to store a Feature Type code in AltSegmentData coincident segment entries, since the feature type of a proto-segment might be different from that of the geometry-modeled segment with which it coincides. The field FEATURE_TYPE serves that purpose, as shown in Table 14.

Table 14 – AltSegmentData Table: FEATURE_TYPE Field

Value	Meaning
null	None of the below (i.e., public street that is not either paper or an addressable non-vehicular path)
1	Rail
2	Shoreline
3	Non-physical census block boundary
4	Other non-street feature
5	Paper street other than FEATURE_TYPE_CODE 9
6	Private street that exists physically
7	Non-physical district boundary other than census
8	Physical boundary, such as cemetery wall
9	Paper street that also serves as a boundary of any kind including a census boundary
A	Alley segment
W	Addressable non-vehicular path

The term “implicit feature class” is used in this document to refer to the CSCL feature class that a proto-segment would belong to if it were geometry-modeled. The implicit feature class of a proto-segment affects how the ETL tool will generate its LION output record.

Note that, except for coincident on-off ramps, the implicit feature class of a proto-segment is not necessarily the same as the feature class of the geometry-modeled segment with which the proto-segment is coincident. Examples:

- Boundary continuity segments spatially coincide with, and have the same SEGMENTID as, their associated geometry-modeled segments. However, a boundary continuity segment is a non-physical non-street feature, so its implicit feature class is NonStreetFeature. The geometry-modeled segment with which it coincides is always a street segment, represented in the Centerline feature class.

- Along part of the Manhattan-Brooklyn border, CSCL will contain a Manhattan borough boundary proto-segment whose implicit feature class is NonStreetFeature, coincident with a Brooklyn geometry-modeled segment in the Shoreline feature class.

The implicit feature class of a proto-segment is determined from its FEATURE_TYPE as specified in Table 15.

Table 15 – Implicit Feature Class Definitions

Implicit Feature Class	AltSegmentData FEATURE_TYPE Values
Centerline	Null, 5, 6, 9, A, W
Rail or Subway	1
Shoreline	2
NonStreetFeature	3, 4, 7, 8

Note that Rail and Subway have been grouped into a single implicit feature class, since no distinction between these types of features is made in LION.

The field FROM_TO_INDICATOR indicates whether the proto-segment has the same or reverse topological direction as that of the geometry-based segment with which it is coincident. That affects the manner in which the ETL tool will populate numerous fields in the output LION records generated from proto-segments.

Table 16 – AltSegmentData Table: FROM_TO_INDICATOR Field

Value	Meaning
null	Proto-segment has the same topological direction as that of the coincident geometry-based segment
R	Proto-segment has the reverse topological direction from that of the coincident geometry-based segment

2.6.1. Determining the Coincident Segment Count

The output field L57 (Coincident Segment Count) must be populated with the appropriate value in every LION output record. If there are no segments coincident with the one being written out, the ETL tool will populate its L57 field with the value ‘1’.

The Centerline feature class has an attribute field called COINCIDENT_SEG_COUNT. It contains an integer greater than '1' for Centerline segments that have coincident segments, whether they are physical multi-decked segments, proto-segments or both. The value in COINCIDENT_SEG_COUNT indicates the total number of coincident segments and proto-segments, including the given Centerline segment.

When processing Centerline segments, to optimize efficiency, the ETL tool will only attempt to determine if AltSegmentData has any proto-segments coincident with the given Centerline segment when the COINCIDENT_SEG_COUNT of the Centerline segment is greater than 1. That determination will be facilitated utilizing the relationship class CenterlinesHaveAltSegmentData, with SEGMENTID serving as the access key.

If a Centerline segment does have proto-segments, the ETL tool will generate their output LION records immediately after generating the one for the Centerline segment. This will enable the tool to capture certain values from the output record generated from the Centerline segment without having to repeat all the processing to determine those values for the proto-segments. Among those values is L57, the Coincident Segment Count. In other words, if the geometry-based segment to which a proto-segment pertains is a Centerline segment, the latter's Coincident Segment Count value will be used to populate the proto-segment's L57 field.

The CSCL linear feature classes other than Centerline do not contain a COINCIDENT_SEG_COUNT attribute, nor do they have a relationship class linking the feature class to AltSegmentData. Therefore, to populate L57 for an output LION record generated from a segment in one of those feature classes (as well as for that segment's proto-segments, if any), it will be necessary to determine if the segment has any coincident segments, and if so, how many. The ETL tool will do so as follows:

1. First, if the given non-Centerline segment belongs to the Rail, Subway or Shoreline feature class (but not to NonStreetFeature), the ETL tool must determine if it is part of a set of coincident multi-decked physical segments, and if so, it must determine how many segments there are in the set. It will do so spatially utilizing ESRI's feature intersection tools. (It cannot do so by utilizing the SEGMENTID, since coincident multi-decked physical segments have unique SEGMENTIDs).

2. Whether the given segment is or is not part of a set of multi-decked physical segments, the ETL tool will search AltSegmentData by SEGMENTID to determine if there are any proto-segments coincident with it (there can be any number of such proto-segments).).

3. The ETL tool will populate L57 with the total number of coincident segments as determined above, counting the segment being written out, all multi-decked physical segments coincident with it (if any), and all proto-segments if there are any.

4. To avoid having to repeat the above processing to determine the coincident segment count for the other coincident segments, the ETL tool will be designed to generate the output records for all segments in a coincident set consecutively, both the multi-decked ones if any, and the proto-segments, if any.

RECOMMENDATION: An improvement in execution performance of the ETL tool, and a simplification of its logic flow, might be achieved by adding a COINCIDENT_SEGMENT_COUNT attribute to the linear feature classes other than Centerline and maintaining it "on the fly" during database updating, just as it will be maintained for Centerline segments. That would obviate the need for the ETL tool, when processing every non-Centerline segment, to find and count the number of coincident segments. (The Maintenance System could potentially also make use of this attribute to facilitate ensuring that coincident segments are kept spatially synchronized with each other. For example, if an elevated Rail segment is split, it would only be necessary to determine whether there is a coincident street segment that also has to be split if the Rail segment's COINCIDENT_SEGMENT_COUNT is greater than one.)

2.6.2. Generating Output Records from Proto-Segments

When generating a LION record from a proto-segment, the ETL tool populates the output fields as indicated in Table 17.

Table 17 – Populating Output Records for Proto-Segments

FIC	Field	Populating Methodology
L1	Borough	Copy AltSegmentData field BOROUGH
L2	Face Code	See Section 2.6.3
L3	Sequence Number	If implicit feature class is Centerline, Copy AltSegmentData field ALT_SEGMENT_SEQNUM. Else, implicit f.c. is NonStreetFeature, in which case assign L3 value "on the fly" based on Borough-Face Code in accordance with Section 2.5.1.
L4	SegmentID	Copy AltSegmentData field SEGMENTID
L5	5-Digit Street Code (5SC)	Copy last 5 bytes of AltSegmentData field B5SC
L6	LGC1	Copy AltSegmentData field LGC1
L7	LGC2	Copy AltSegmentData field LGC2
L8	LGC3	Copy AltSegmentData field LGC3
L9	LGC4	Copy AltSegmentData field LGC4
L10	Board of Elections LGC Pointer	Copy AltSegmentData field BOE_PREFERRED_LGC_FLAG
L11	From-Sectional Map	See Section 2.6.4
L12	From-NodeID	See Section 2.6.4
L13	From-X Coordinate	See Section 2.6.4
L14	From-Y Coordinate	See Section 2.6.4
L15	To-Sectional Map	See Section 2.6.4
L16	To-NodeID	See Section 2.6.4
L17	To-X Coordinate	See Section 2.6.4
L18	To-Y Coordinate	See Section 2.6.4

FIC	Field	Populating Methodology
L19	Left 2000 Census Tract	See Section 2.6.4
L20	Left Dynamic Block	See Section 2.6.4
L21	Left Low House Number	See Section 2.6.4
L22	Left High House Number	See Section 2.6.4
L23	Left Dept of Sanitation Subsection	See Section 2.6.4
L24	Left Zip Code	See Section 2.6.4
L25	Left Assembly District	See Section 2.6.4
L26	Left Election District	See Section 2.6.4
L27	Left School District	See Section 2.6.4
L28	Right 2000 Census Tract	See Section 2.6.4
L29	Right Dynamic Block	See Section 2.6.4
L30	Right Low House Number	See Section 2.6.4
L31	Right High House Number	See Section 2.6.4
L32	Right Dept of Sanitation Subsection	See Section 2.6.4
L33	Right Zip Code	See Section 2.6.4
L34	Right Assembly District	See Section 2.6.4
L35	Right Election District	See Section 2.6.4
L36	Right School District	See Section 2.6.4
L37	Split Election District Flag	If implicit feature class is Centerline populate as described in Section 2.2.4, else populate with a blank
L38	Filler (formerly Split Community School District Flag)	N/A
L39	Sanitation District Boundary Indicator	See Section 2.6.4
L40	Traffic Direction	See Section 2.6.5
L41	Segment Locational Status	See Section 2.6.7
L42	Feature Type Code	Copy AltSegmentData field FEATURE_TYPE
L43	Non-Pedestrian Flag	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with blank
L44	Continuous Parity Indicator	If implicit feature class is Centerline and AltSegmentData field SOSINDICATOR contains ‘1’ or ‘2’, populate with ‘L’ or ‘R’ respectively; else, populate with a blank
L45	Filler (formerly the Near BQ-Boundary Flag)	N/A
L46	Borough Boundary Indicator	See Section 2.6.4
L47	Twisted Parity Flag	Translate AltSegmentData field TWISTED_PARITY_FLAG as specified in Section 2.2.9
L48	Special Address Flag	See Section 2.2.10

FIC	Field	Populating Methodology
L49	Curve Flag	See Section 2.6.4
L50	Center of Curvature X-Coordinate	Copy from geometry-based segment
L51	Center of Curvature Y-Coordinate	Copy from geometry-based segment
L52	Segment Length in Feet	Copy from geometry-based segment
L53	From Level Code	See Section 2.6.6
L54	To Level Code	See Section 2.6.6
L55	Traffic Direction Verification Flag	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L56	Segment Type Code	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with 'U'
L57	Coincident Segment Counter	See Section 2.6.1
L58	Include/Exclude Flag	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L59	RW_TYPE	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L60	PHYSICALID	If ALT_SEGDATA_TYPE = 'C', populate with blanks. Else, copy from output record for geometry-based segment
L61	GENERICID	If ALT_SEGDATA_TYPE = 'C', populate with blanks. Else, copy from output record for geometry-based segment
L62	NYPDID	If ALT_SEGDATA_TYPE = 'C', populate with blanks. Else, copy from output record for geometry-based segment
L63	FDNYID	If ALT_SEGDATA_TYPE = 'C', populate with blanks. Else, copy from output record for geometry-based segment
L64	~~Left BLOCKFACEID~~, Filler	See Section 2.6.4
L65	~~Right BLOCKFACEID, Filler~~	See Section 2.6.4
L66	STATUS	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L67	STREETWIDTH MIN	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L68	STREETWIDTH_IRR	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank

FIC	Field	Populating Methodology
L69_1	BIKELANE	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L70	FCC	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with blanks
L72	Left 2010 Census Tract	See Section 2.6.4
L73	Right 2010 Census Tract	See Section 2.6.4
L74	LGC5	Copy AltSegmentData field LGC5
L75	LGC6	Copy AltSegmentData field LGC6
L76	LGC7	Copy AltSegmentData field LGC7
L77	LGC8	Copy AltSegmentData field LGC8
L78	LGC9	Copy AltSegmentData field LGC9
L79	Legacy SEGMENTID	Copy from geometry-based segment
L80	LEFT CENSUS BLOCK 2000 BASIC	See Section 2.6.4
L81	LEFT CENSUS BLOCK 2000 SUFFIX	See Section 2.6.4
L82	RIGHT CENSUS BLOCK 2000 BASIC	See Section 2.6.4
L83	RIGHT CENSUS BLOCK 2000 SUFFIX	See Section 2.6.4
L84	LEFT CENSUS BLOCK 2010 BASIC	See Section 2.6.4
L85	LEFT CENSUS BLOCK 2010 SUFFIX	See Section 2.6.4
L86	RIGHT CENSUS BLOCK 2010 BASIC	See Section 2.6.4
L87	RIGHT CENSUS BLOCK 2010 SUFFIX	See Section 2.6.4
L88	SNOW PRIORITY	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L69_2	BIKELANE_2	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L67_2	STREET WIDTH MAX	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L89	Filler	3 digits
L90	LEFT BLOCK FACE ID	See Section 2.6.4
L91	RIGHT BLOCK FACE ID	See Section 2.6.4
L92	NUMBER TRAVEL LANE	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L93	NUMBER PARK LANES	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L94	NUMBER TOTAL LANES	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank

FIC	Field	Populating Methodology
L95	BIKE TRAFFIC DIR	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L96	POSTED SPEED	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L97	Left NYPD Service Area	See Section 2.6.4 and 2.2.13
L98	Right NYPD Service Area	See Section 2.6.4 and 2.2.13
L99	Truck Route Type	If implicit feature class is Centerline, copy from geometry-based segment; else, populate with a blank
L100	LEFT 2020 CENSUS TRACT	See Section 2.6.4
L101	RIGHT 2020 CENSUS TRACT	See Section 2.6.4
L102	LEFT CENSUS BLOCK 2020 BASIC	See Section 2.6.4
L103	LEFT CENSUS BLOCK 2020 SUFFIX	See Section 2.6.4
L104	RIGHT CENSUS BLOCK 2020 BASIC	See Section 2.6.4
L105	RIGHT CENSUS BLOCK 2020 SUFFIX	See Section 2.6.4
L199	Filler	blank

2.6.3. Face Code

A proto-segment always belongs to a different feature from that of its associated geometry-modeled segment, and its implicit feature class may even be different from the feature class of the geometry-modeled segment. Therefore, the values of feature-specific fields for the proto-segment cannot be obtained from the geometry-modeled segment. Most of these output fields will be obtained from fields in the AltSegmentData entry for the given proto-segment, as indicated in Table 17. Field L2 (Face Code) will be obtained as follows:

- The preferred B7SC for the proto-segment will be constructed by concatenating the AltSegmentData fields B5SC and LGC1.
- If the implicit feature class of the proto-segment is Centerline, the StreetName table will be accessed by the preferred B7SC with the PRINCIPAL_FLAG = 'Y'. If an entry matching those criteria is found and it contains a Face Code value, it will be populated into L2.
- If the implicit feature class of the proto-segment is not Centerline, the FeatureName table will be accessed as described above instead of the StreetName table.
- If the ETL tool is unable to obtain a Face Code value via the above procedure, it will not write an output LION record for the given proto-segment, and it will issue an appropriate error message. (LION records must not be written out without a valid Face Code value, since it is part of the LION Key, which serves as the primary key.)

2.6.4. Fields Affected by Topological Direction

When generating a LION output record from a proto-segment, the ETL tool examines the FROM_TO_INDICATOR field in the AltSegmentData entry to enable the output record to be generated with the proper topological direction.

The following discussion applies to combined on-off ramp proto-segments and boundary continuity proto-segments. The case of borough boundary proto-segments must be handled differently, and is discussed at the bottom of this section.

<u>Proto-Segments Other Than Borough Boundaries</u>

If the FROM_TO_INDICATOR does not contain the value 'R', then the ETL tool will populate all of the output fields for the proto-segment that are discussed in this section by copying the values that are in the corresponding fields in the output record generated from the geometry-based segment.

If the FROM_TO_INDICATOR does contain 'R', that signifies that the topological direction of the proto-segment is opposite to that of the geometry-based segment. In that case, the manner in which output fields affected by topological direction should be populated is as follows:

- The values that the ETL tool will populate into all corresponding pairs of "from" and "to" fields in the LION segment generated from the proto-segment will be interchanged from the values in the LION segment generated from the geometry-based segment. Specifically, the ETL tool will interchange the following pairs:
    - L11 and L15 (Sectional Map numbers)
    - L12 and L16 (NODEIDs)
    - L13 and L17 (X Coordinates)
    - L14 and L18 Y Coordinates)
    - L53 and L54 (Level Codes) – see discussion in Section 2.6.6
- The values that the ETL tool will populate into all corresponding pairs of "left" and "right" fields in the LION segment generated from the proto-segment will be interchanged from those in the LION segment generated from the geometry-based segment. Specifically, the ETL tool will interchange the following pairs:
    - L19 and L28 (2000 Census Tracts)
    - L20 and L29 (Dynamic Blocks)
    - L21 and L30 (Low House Numbers)
    - L22 and L31 (High House Numbers)

- L23 and L32 (Department of Sanitation Subsections)(NO??? could not find the corresponding data layer in CSCL)
- L24 and L33 (Zip Codes)
- L25 and L34 (Assembly Districts)
- L26 and L35 (Election Districts)
- L27 and L36 (School Districts)
- L64 and L65 (BLOCKFACEIDs)
- L72 and L73 (2010 Census Tracts)

  Etcs…

- The values that the ETL tool will populate into certain other fields must be modified from the values that are in those fields for the geometry-based segment, as follows:
  - L39 (Sanitation District Boundary Indicator): If the value of this field for the geometry-based segment is either 'L' or 'R', the ETL tool will populate this field for the proto-segment with the opposite value. If the geometry-based value is other than 'L' or 'R', it will be populated into the proto-segment unchanged.
  - L40 (Traffic Direction) - see discussion in Section 2.6.5.
  - L46 (Borough Boundary Indicator): If the value of this field for the geometry-based segment is either 'L' or 'R', the ETL tool will populate this field for the proto-segment with the opposite value. If the geometry-based value is other than 'L' or 'R', it will be populated into the proto-segment unchanged.
  - L49 (Curve Flag): If the value of this field for the geometry-based segment is either 'L' or 'R', the ETL tool will populate this field for the proto-segment with the opposite value. If the geometry-based value is other than 'L' or 'R', it will be populated into the proto-segment unchanged.

<u>Borough Boundary Proto-Segments</u>

The ETL tool populates the "from" and "to" output fields in the output record generated from a borough boundary proto-segment in the same manner as described above for non-borough boundary proto-segments. However, borough boundary proto-segments require special handling with respect to populating their "left" and "right" output fields.

In the output record generated from the geometry-modeled segment, these fields contain non-blank values only for the side of the segment that is in that segment's borough. Therefore, that record does not contain values for the side that is in the proto-segment's borough. However, when that record was generated, the ETL tool determined the values for the side in the proto-segment's borough (with the exception of the zipcode – see below),

and saved them for use in generating the output record for the proto-segment (see the discussion at the bottom of Section 2.2.3).

In the output record generated from the (borough boundary) proto-segment, the ETL tool populates the side-related output fields only for the side that is in the borough to which the proto-segment belongs. If the FROM_TO_INDICATOR does not contain an 'R', that is the side opposite that for which the geometry-modeled segment's side-related fields are populated. If the FROM_TO_INDICATOR does contain an 'R', it is the same side as that for which the geometry-modeled segment's side-related fields are populated.

In either case, the ETL tool will populate the side-related fields for the appropriate side using the values determined and saved for this purpose when the geometry-modeled segment was processed, as described in Section 2.2.3, except for the zipcode field, which will be populated from the ZIPCODE field in the proto-segment's AltSegmentData entry. The tool will populate the side-related fields for the opposite side with blanks.

2.6.5. Traffic Direction

The manner in which the ETL tool populates field L40 (Traffic Direction) in the output record generated from a proto-segment depends on the proto-segment's topological direction relative to that of the geometry-based segment, on its feature type and on its coincident segment type.

The proto-segment represents a physical street, bridge or tunnel if the AltSegmentData field FEATURE_TYPE = null (non-paper street, bridge or tunnel), 6 (physical private street), A (alley) or W (pedestrian street or path).

Field L40 (Traffic Direction) will be populated in the output record for the proto-segment as follows:

- If the proto-segment does not represent a physical street, bridge or tunnel (as defined above), L40 will be populated with a blank.
- If the proto-segment represents a physical street, bridge or tunnel and is a borough boundary segment (ALT_SEGDATA_TYPE = 'B' or 'R'):
    - If the value in L40 for the geometry-based segment is either of the one-way values, 'A' and 'W', then the value to be populated into L40 for the proto-segment depends on the value of its FROM_TO_INDICATOR, as follows:
        - If the FROM_TO_INDICATOR does not contain the value 'R', the value populated into L40 for the proto-segment will be the same value as that of the geometry-based segment.
        - If the FROM_TO_INDICATOR does contain an 'R', the value populated into L40 for the proto-segment will be opposite to that of the geometry-based segment. That is, if the geometry-based value is 'A', the proto-segment will be populated with a 'W', and vice versa.

- If the geometry-based value is other than ‘A’ or ‘W’, that value will be populated into L40 for the proto-segment unchanged.

- ~~If the proto-segment represents a physical street, bridge or tunnel and is a combined on-off ramp (ALT_SEGDATA_TYPE = ‘R’) :~~

  - ~~If the value in L40 for the geometry-based segment is either of the one-way values, ‘A’ and ‘W’, then the value to be populated into L40 for the proto-segment depends on the value of its FROM_TO_INDICATOR, as follows:~~

    - ~~If the FROM_TO_INDICATOR does not contain the value ‘R’, the value populated into L40 for the proto-segment will be the opposite value to that of the geometry-based segment (since the on-ramp and off-ramp have opposite one-way traffic directions). That is, if the geometry-based value is ‘A’, the proto-segment will be populated with a ‘W’, and vice versa.~~

    - ~~If the FROM_TO_INDICATOR does contain an ‘R’, the value populated into L40 for the proto-segment will be the same value as that of the geometry-based segment (since the on-ramp’s and off-ramp’s opposite traffic directions are “cancelled out” by their reverse topological directions).~~

  - ~~If the geometry-based value is other than ‘A’ or ‘W’, that value will be populated into L40 for the proto-segment unchanged.~~

2.6.6. Level Codes

The manner in which the ETL tool populates fields L53 and L54 (Level Codes) in the output record generated from the proto-segment depends on the proto-segment’s topological direction as well as on its feature type, as follows:

- If the proto-segment is not a physical feature (i.e., if FEATURE_TYPE = 3, 4, 5, 7 or 9), L53 and L54 will be populated with asterisks.

- If the proto-segment is a physical feature but FEATURE_TYPE is not 2 (Shoreline) or 8 (a physical feature in the implicit feature class NonStreetFeature):

  - If the FROM_TO_INDICATOR in the AltSegmentData entry for the proto-segment contains the value ‘R’, the ETL tool will populate L53 and L54 in the output record for the proto-segment by interchanging the values in those fields for the geometry-based segment.

  - If the FROM_TO_INDICATOR in the AltSegmentData entry for the proto-segment contains a value other than ‘R’, the ETL tool will populate L53 and L54 in the output record for the proto-segment by copying the geometry-based values without interchanging them.

- Proto-segments are expected never to belong to the implicit feature class of Shoreline (FEATURE_TYPE = 2).

- If the proto-segment is a physical feature in the implicit feature class NonStreetFeature (FEATURE_TYPE = 8), the ETL tool will populate L53 and L54 with asterisks.

2.6.7. Segment Locational Status

If the value of field L41 (Segment Locational Status) for the geometry-modeled segment is one of the integers from '1' to '5', that signifies that the geometry-modeled segment is a borough boundary segment. In that case, the following should be true:

- The borough boundary geometry-modeled segment should have a coincident proto-segment.

- The AltSegmentData entry for the proto-segment should be classified as a borough boundary coincident segment entry. That is, the ALT_SEGDATA_TYPE field in that entry should contain the value 'B'.

- The AltSegmentData entry for the proto-segment should belong to the other borough than the one to which the geometry-modeled segment belongs. Specifically, the BOROUGH field in the AltSegmentData entry should equal the value of field L41 in the output record generated from the geometry-modeled segment.

The ETL tool validates for the above conditions. If there are no proto-segments coincident with a borough boundary geometry-based segment, the ETL tool will report that as a data discrepancy. If there is a coincident proto-segment, but either of the last two conditions listed above is not satisfied, that will be reported as a data discrepancy, but the ETL tool will continue processing that AltSegmentData entry for output.

Although it is hypothetically possible for a geometry-modeled borough boundary segment to have more than one coincident proto-segment, it is expected that, in practice, there will never be more than one. (There would appear to be no need to have a coincident boundary continuity segment along a borough boundary, and it appears highly unlikely that a combined on-off ramp would ever be constructed in the future lying directly along a borough boundary.)

The ETL tool will populate field L41 in the output record for a proto-segment as follows:

- If the proto-segment is coincident with a borough boundary geometry-modeled segment, the ETL tool will populate the value of field L41 with the value that is in field L1 (Borough Code) in the output record generated for the geometry-based segment.

- If the proto-segment is coincident with a non-borough boundary geometry-modeled segment (one whose output record contains a value other than '1' to '5' in field L41), the ETL tool will populate the value of field L41 in the output record generated from the proto-segment with the value that is in field L41 in the output record generated from the geometry-based segment.

2.7. ESRI LION

In addition to the Geosupport LION extract, the ETL tool can also extract LION in the format for DCP's BYTES of the BIG APPLE™ data products: ESRI file geodatabase format ~~and MapInfo (MIF) format~~. This section and its sub-sections discuss the ESRI LION. Section 2.8 and its sub-sections discuss the MapInfo LION.

Many of the fields in the BYTES LION files are identical to those in the Geosupport LION. The ETL tool developer may deem it advisable, if it is technically feasible, to write out records into the ESRI LION ~~and the MapInfo LION~~ in tandem with writing records into the Geosupport LION file. That would avoid unnecessarily repeating the processing required to populate many of the fields. (However, that would not be possible if the user requested the production of a BYTES LION but not the Geosupport LION, unless in those instances the ETL tool were designed to generate a Geosupport LION "under the covers" solely for the purpose of leveraging its algorithms. This document assumes that approach.) References of the form L<integer> below are FICs of Geosupport LION fields.

Records will not be written into the Geosupport LION for subterranean rail and subway segments that do not serve as district boundaries, as noted in the "bulleted" items at the beginning of Section 2. However, such records will be written into the BYTES versions. To facilitate that, it may be advantageous for the developer to design the ETL tool to create "virtual" (not written into the output file) Geosupport LION records for such segments, from which the BYTES records can then be derived.

The ESRI LION will be extracted as a single file geodatabase for the entire city. Appendix A contains a geodatabase schema for the ESRI LION that has been provided by DCP. The ESRI LION consists of two object classes: a linear feature class called LION and a table of alternative street names (aliases) called Altnames. The field JOIN_ID relates the two components.

Certain segments that are represented in the Geosupport LION by a single record are represented in the ESRI LION ~~(as well as the MapInfo LION)~~ by multiple records that have identical geometry but differ in some of their attribute data. The ETL tool will generate these records as required. There are two cases of this: continuous parity segments and segments that have Special Address File (SAF) data (Plus boro boundary segments).

The Geodatabase Lion data model can be found at folder of "Templates\lion.gdb".

Fields of Byte LION and the field definitions

FIC	Field	Populating Method
BL1	Street	See 2.7.3
BL2	SAFStreetName	See 2.7.3
BL3	FeatureTyp	Same as Geo-support LION L42
BL4	SegmentTyp	Same as Geo-support LION L56
BL5	IncExFlag	Same as Geo-support LION L58
BL7	NonPed	See 2.7.3
BL6	RB_Layer	Same as Geo-support LION L43
BL7	TrafDir	Same as Geo-support LION L40

FIC	Field	Populating Method
BL8	TrafSrc	Same Geo-support LION L55, Also see 2.7.3
BL9	SpecAddr	Same as Geo-support LION L48, Also see 2.7.2, and 2.7.3
BL10	FaceCode	Same as Geo-support LION L2
BL11	SeqNum	Same as Geo-support LION L3
BL12	StreetCode	See 2.7.3
BL13	SAFStreetCode	See 2.7.3
BL14	LGC1	Same as Geo-support LION L6
BL15	LGC2	Same as Geo-support LION L7
BL16	LGC3	Same as Geo-support LION L8
BL17	LGC4	Same as Geo-support LION L9
BL18	LGC5	Same as Geo-support LION L74
BL19	LGC6	Same as Geo-support LION L75
BL20	LGC7	Same as Geo-support LION L76
BL21	LGC8	Same as Geo-support LION L77
BL22	LGC9	Same as Geo-support LION L78
BL23	BOE_LGC	Same as Geo-support LION L10
BL24	SegmentID	Same as Geo-support LION L4
BL25	SegCount	Same as Geo-support LION L57, also see 2.7.3
BL26	LocStatus	Same as Geo-support LION L41
BL27	Lzip	Same as Geo-support LION L24
BL28	Rzip	Same as Geo-support LION L33
BL29	Lboro	See 2.7.3
BL30	Rboro	See 2.7.3
BL31	L_CD	Get left Community district (Left side CD are based on segment direction, borough boundary, MH, RI)
BL32	R_CD	Get right community district (segment direction (Right side CD are based on segment direction, borough boundary, MH, RI)
BL33	LATOMICPOLYGON	Same as Geo-support LION L20
BL34	RATOMICPOLYGON	Same as Geo-support LION L29
BL35	LCT2010	Same as Geo-support LION L72.basic
BL36	LCT2010Suf	Same as Geo-support LION L72.suffix
BL37	RCT2010	Same as Geo-support LION L73.basic
BL38	RCT2010Suf	Same as Geo-support LION L73.suffix
BL39	LCB2010	Same as Geo-support LION L84 except RJZF
BL40	LCB2010Suf	Same as Geo-support LION L85
BL41	RCB2010	Same as Geo-support LION L86 except RJZF
BL42	RCB2010Suf	Same as Geo-support LION L87
BL43	LCT2000	Same as Geo-support LION L19.basic
BL44	LCT2000Suf	Same as Geo-support LION L19.suffix
BL45	RCT2000	Same as Geo-support LION L28.basic
BL46	RCT2000Suf	Same as Geo-support LION L28.suffix
BL47	LCB2000	Same as Geo-support LION L80 except RJZF
BL48	LCB2000Suf	Same as Geo-support LION L81
BL49	RCB2000	Same as Geo-support LION L82 except RJZF
BL50	RCB2000Suf	Same as Geo-support LION L83
BL51	LCT1990	Same as Geo-support LION L19.basic except getting data from Census tract 1990 in AP feature class
BL52	LCT1990Suf	Same as Geo-support LION L19.suffix except getting data from census tract 1990 in AP feature class
BL53	RCT1990	Same as Geo-support LION L28.basic except getting data from Census tract 1990 in AP feature class
BL54	RCT1990Suf	Same as Geo-support LION L28.suffix except getting data from census tract 1990 in AP feature class

FIC	Field	Populating Method
BL55	LAssmDist	Same as Geo-support LION L25
BL56	LElectDist	Same as Geo-support LION L26
BL57	RAssmDist	Same as Geo-support LION L34
BL58	RElectDist	Same as Geo-support LION L35
BL59	SplitElect	Same as Geo-support LION L37
BL60	LSchlDist	Same as Geo-support LION L27
BL61	RSchlDist	Same as Geo-support LION L36
BL62	SplitSchl	Blank, it is one digit filler in Geo-support Lion
BL63	LSubSect	Same as Geo-support LION L23
BL64	RSubSect	Same as Geo-support LION L32
BL65	SanDistInd	Same as Geo-support LION L39
BL66	MapFrom	See 2.7.3
BL67	MapTo	See 2.7.3
BL68	BoroBndry	Same as Geo-support LION L46
	MH_RI_Flag	See 2.7.3
BL69	XFrom	Same as Geo-support LION L13
BL70	YFrom	Same as Geo-support LION L14
BL71	XTo	Same as Geo-support LION L17
BL72	YTo	Same as Geo-support LION L18
BL73	ArcCenterX	Same as Geo-support LION L50
BL74	ArcCenterY	Same as Geo-support LION L51
BL75	CurveFlag	Same as Geo-support LION L49
BL76	Radius	Calculate it using grid1 method
BL77	NodeIDFrom	Same as Geo-support LION L12
BL78	NodeIDTo	Same as Geo-support LION L16
BL79	NodeLevelF	Same as Geo-support LION L53
BL80	NodeLevelT	Same as Geo-support LION L54
BL81	ConParity	Same as Geo-support LION L44, Also see 2.7.1
BL82	Twisted	Same as Geo-support LION L47
BL83	RW_TYPE	Same as Geo-support LION L59
BL84	PhysicalID	Same as Geo-support LION L60
BL85	GenericID	Same as Geo-support LION L61
BL86	NYPDID	Same as Geo-support LION L62
BL87	FDNYID	Same as Geo-support LION L63
BL88	LBlockFaceID	Same as Geo-support LION L90
BL89	RBlockFaceID	Same as Geo-support LION L91
BL90	LegacyID	Same as Geo-support LION L79
BL91	Status	Same as Geo-support LION L66
BL92	StreetWidth_Min	Same as Geo-support LION L67
BL93	StreetWidth_Irr	Same as Geo-support LION L68
BL94	BikeLane	Same as Geo-support LION L69_2
BL95	FCC	Same as Geo-support LION L70
BL96	ROW_Type	Same as Geo-support LION L71
BL97	LLo_Hyphen	Text, 7 digits, same as Centerline except “0” will be “null”. Also see 2.7.3
BL98	LHi_Hyphen	Text, 7 digits, same as Centerline, see 2.7.3 except “0” will be “null”. Also see 2.7.3
BL99	RLo_Hyphen	Text, 7 digits, same as Centerline except “0” will be “null”. Also see 2.7.3
BL100	RHi_Hyphen	Text, 7 digits, same as Centerline except “0” will be “null”. Also see 2.7.3
BL101	FromLeft	Long integer, see 2.7.3 for hyphen address normalization
BL102	ToLeft	Long integer, see 2.7.3 for hyphen address normalization

FIC	Field	Populating Method
BL103	FromRight	Long integer, see 2.7.3 for hyphen address normalization
BL104	ToRight	Long integer, see 2.7.3 for hyphen address normalization
BL105	Join_ID	See 2.7.3
BL106	Snow_Priority	Same as Geo-support LION L88
BL107	Number_Travel_Lanes	Same as Geo-support LION L92
BL108	Number_Park_Lanes	Same as Geo-support LION L93
BL109	Number_Total_Lanes	Same as Geo-support LION L94
BL110	Carto_Display_Level	
BL111	StreetWidth_Max	Same as Geo-support LION L67_2
BL112	BIKE_TRAFDIR	Same as Geo-support LION L95
BL113	Active_Flag	Subway Active Flag
BL114	POSTED_SPEED	Same as Geo-support LION L96
BL115	L_PD_Service_Area	Same as Geo-support LION L97
BL116	R_PD_Service_Area	Same as Geo-support LION L98
BL117	Truck_Route_Type	Same as Geo-support LION L99
BL118	LCT2020	Same as Geo-support LION L100.basic
BL119	LCT2020Suf	Same as Geo-support LION L100.suf
BL120	RCT2020	Same as Geo-support LION L101.basic
BL121	RCT2020Suf	Same as Geo-support LION L101.suf
BL122	LCB2020	Same as Geo-support LION L102 except RJZF
BL123	LCB2020Suf	Same as Geo-support LION L103
BL124	RCB2020	Same as Geo-support LION L104 except RJZF
BL125	RCB2020Suf	Same as Geo-support LION L105

(Notes: The attributes of Census block fields of Byte Lion are the same as the ones in AP)

2.7.1. Segment Replication for Continuous Parity Condition

A continuous parity segment is a street segment in which odd and even addresses exist in reality on the same side of the street. In the Geosupport LION, a continuous parity segment is modeled as a single record, with the fields nominally labeled as the left low and high house number fields used to store the address range of one of the parities and the nominally labeled right house number fields used to store the address range of the other parity. Continuous parity segments are identified by the value 'L' or 'R' in the Continuous Parity Indicator field, which indicates on which side of the segment both address ranges actually exist in reality. (This is also how a continuous parity segment is modeled in the CSCL database, both in the Centerline feature class and with respect to proto-segment entries in the AltSegmentData table, except that instead of the LION values 'L' and 'R', in the CSCL database the values '1' and '2' are used, respectively. For continuous parity proto-segments, the SOSINDICATOR field serves as the Continuous Parity Indicator.)

In the ESRI LION, a continuous parity segment is modeled as two distinct, spatially coincident features, one containing the odd range and the other containing the even range, with both ranges stored (in their respective records) in the house number fields for the side of the street on which the addresses actually exist in reality, that is, the side identified by the Continuous Parity Indicator. The two features are identical in all other attribute data aside from the house numbers.

For example, segment 34262 is the segment of Central Park West between West 61 Street and West 62 Street in Manhattan. The odd address range 13 to 21 and the even range

14 to 22 are allocated to this segment. In reality, both odd and even addresses exist on the left (west) side of the segment, and there are no addresses on the right (east) side, since that side adjoins Central Park.

- In the single Geosupport LION record for segment 34262, the odd range, 13 to 21 is stored in the left house number fields; the even range, 14 to 22 is stored in the right house number fields; and the Continuous Parity Indicator contains an 'L'.

- In the ESRI LION, two features must be generated in the LION linear feature class representing segment 34262. One feature contains 13 and 21 in the left house number fields and zeros in the right house number fields. The other feature contains 14 and 22 in the left house number fields and zeros in the right house number fields. Aside from the house number fields, both records are identical in every respect, that is, in both geometry and attribute data (including LION keys, SEGMENTID etc.).

During ESRI LION extraction, when the ETL tool is processing a CSCL Centerline segment whose CONTINUOUS_PARITY_FLAG contains '1' or '2', or a proto-segment that contains house number values and whose SOSINDICATOR contains '1' or '2', the ETL tool will generate two output features. In one feature, the tool will store the odd range in the house number fields for the side of the segment indicated by the CONTINUOUS_PARITY_FLAG or SOSINDICATOR, and will populate zeros into the house number fields for the other side. In the other feature, the tool will store the even range in the house number fields for the side indicated by the flag, and zeros in the house number fields for the other side. The geometry and all other attribute values will be identical for the two features. The ETL tool will set the output SegCount field value accordingly (reflecting the existence of both features), as discussed in detail in Section 2.7.3.

2.7.2. Segment Replication for SAF Data

The ETL tool creates coincident output records for segments that have SAF data. The methodology for determining what SAF data, if any, are associated with a segment or proto-segment is described in detail at the end of Section 2.2.10.

In addition to the "regular" output ESRI LION record generated from a CSCL segment or proto-segment, one additional output record (or two, if the SAF datum has a continuous parity condition) is generated for <u>each</u> SAF datum associated with that segment or proto-segment. Such "SAF replicants" will have identical geometry to that of the "regular" output record, and most of their attribute items will also be identical to those of the "regular" output record. However, certain of their attributes are obtained from the SAF data instead of from the CSCL segment or proto-segment, as described below.

SAF replicants will be generated for the following CSCL SAF data:

- All SAF entries in the AltSegmentData table (entries in which the SAFTYPE field contains a value)

- All points in the CommonPlace feature class for which the SAFTYPE attribute contains a value

- All points in the AddressPoint feature class for which the SPECIAL_CONDITION attribute = 'S' or 'V'

In generating SAF replicants, care must be taken to interpret the SOSINDICATOR field correctly, as follows:

- In SAF type A, B, C, E and P AltSegmentData entries, the SOSINDICATOR serves as a continuous parity indicator, and the output record will be generated in accordance with the methodology described in Section 2.7.1. For example, if a type A SAF entry in AltSegmentData contains '1' in the SOSINDICATOR, and the entry contains address ranges in the left and right house number fields, two output records will be generated for that entry, with both address ranges stored in the left house number fields in the respective output records. Those records are in addition to the "regular" output record or records generated for the underlying street segment.

- In SAF type D, F and O AltSegmentData entries, the SOSINDICATOR serves as a side of street indicator, not a continuous parity indicator. Type D and O entries contain only a single address range, which pertains to one side of the underlying street segment (the side indicated by the SOSINDICATOR). One output record will be generated (in addition to the "regular" output record or records.) In that output record, side-related output fields, such as house numbers and zip codes, will be populated only for the side identified by the SOSINDICATOR value. For example, in a record generated for a type D SAF entry in which SOSINDICATOR = '1', the zip code value from the AltSegmentData ZIPCODE field will be stored in the RZip field in the output ESRI LION feature, and the LZip output field will be populated with null values.

- SAF type G, N and X features are stored in the CommonPlace feature class. They represent Non-Addressable Placenames (NAPs). Since these names do not have associated house numbers, the continuous parity concept is not applicable to them. NAPs pertain to a specific side of the underlying street segment. The SOSINDICATOR attribute serves as the side of street indicator. One output record will be generated (in addition to the "regular" record or records).

- SAF type S and V features are stored in the AddressPoint feature class. They are identified by SPECIAL_CONDITION = 'S' or 'V'. These are individual addresses or address ranges for individual buildings, and pertain to a specific side of the street. The SOSINDICATOR serves as a side of street indicator. One output record will be generated (in addition to the "regular" record or records).

Output ESRI LION features generated for SAF data are populated as follows. All data items not obtained from the CSCL SAF data as specified below are obtained from attribute fields and corresponding street names of the underlying segment, including in particular the output fields Street, StreetCode (a B5SC field), LGC1 through LGC4 and the BOE_LGC field (= BOE LGC Pointer).

- The output field SpecAddr will contain the SAF type, populated as follows:

  - If the SAF replicant is being generated from a common place SAF feature, or from an AltSegmentData SAF entry other than SAFTYPE = 'F', SpecAddr will be populated from the SAFTYPE field of that common place or entry.

  - If the SAF replicant is being generated from an AltSegmentData SAF entry with SAFTYPE = F, SpecAddr will be populated with the value 'D'.

  - If the SAF replicant is being generated from an address point SAF feature, SpecAddr will be populated from its SPECIAL_CONDITION attribute (that is, with the value 'S' or 'V').

  - Note that the SpecAddr output field may not contain the same value in SAF replicants as it does in the output record generated directly from the geometry-modeled CSCL segment. The latter is populated with the same value that is (or would be) populated into field L48 of the Geosupport LION record, as described in Section 2.2.10.

- For SAF replicants generated from type A SAF data:

  - The house number items (LLo_Hyphen, LHi_Hyphen, RLo_Hyphen, RHi_Hyphen, FromLeft, ToLeft, FromRight and ToRight) will be populated from the corresponding fields in the AltSegmentData entry (rather than from the Centerline segment).

  - The output field SAF Streetname will be copied from the output field Street.

  - The output field SAF Streetcode (a B5SC field) will be copied from the output field StreetCode.

- For SAF replicants generated from type B, C, D, E, F, O and P SAF data:

  - The output fields for house numbers and the SAF Streetcode (a B5SC field) will be populated from the corresponding fields in the AltSegmentData SAF entry, rather than from the Centerline segment's attribute data.

  - The output field SAF Streetname will be populated with the principal name of the preferred SAF B7SC, obtained from the LOOKUP_KEY field in the CSCL StreetName table entry that is flagged as principal for the SAF preferred B7SC value. The SAF preferred B7SC value for this lookup will be formed by concatenating the values in the fields B5SC and LGC1 in the AltSegmentData SAF entry.

  - For type D and F records, the zip code output field for the side of the segment indicated by the SOSINDICATOR will be populated from the AltSegmentData ZIPCODE field.

- For SAF replicants generated from SAF type G, N and X data:

  - The SAF Streetcode output field will be populated from the first six bytes (the B5SC) of the B7SC attribute field of the given CommonPlace feature.

  - The SAF Streetname will be populated with the name obtained from the LOOKUP_KEY field in the FeatureName table entry that is flagged as principal for the aforementioned B7SC value.

  - All house number fields will be populated with blanks.

- For SAF replicants generated from SAF type S or V data:

  - The house number and zip code output fields for the side of the segment indicated by the SOSINDICATOR will be populated from the corresponding AddressPoint attributes. See the discussion of house number fields in Section 2.7.3 for further details on how to populate the output house number fields for type S and V replicants.

  - For type S replicants, the SAF StreetCode output field will be populated from the first six bytes of the B7SC_ACTUAL AddressPoint attribute. For type V replicants, the SAF StreetCode output field will be populated from the first six bytes of the B7SC_VANITY AddressPoint attribute.

  - The SAF Streetcode output field will be populated with the name obtained from the LOOKUP_KEY field in the StreetName table entry that is flagged as principal for the given B7SC value (that is, for the B7SC_ACTUAL for type S replicants, and the B7SC_VANITY for type V replicants).

2.7.3. Methodologies for Populating Output Fields

Fields in the output ESRI LION that are “empty” (devoid of meaningful contents) are populated with blanks, zeros or null values, depending on the field. Appendix C contains a DCP-provided table that indicates, for both the ESRI LION and the MapInfo LION, how “empty” fields should be populated.

Aside from the special treatment of continuous parity and SAF data and the treatment of “empty” fields discussed above, almost all of the attribute fields in the ESRI LION feature class will be populated with the same values that are in the corresponding fields in the Geosupport LION file. Methodologies for populating fields that are not entirely straightforward are as follows:

- The output field SpecAddr will be populated with a blank in all output records other than SAF replicants. In SAF replicants, it will be populated with the SAF type as described in Section 2.7.2.

- The output fields <u>SAF Streetname</u> and <u>SAF Streetcode</u> are two fields that exist only in the Bytes LION files, not in the Geosupport LION. In output records generated directly from geometry-modeled segments and in continuous parity replicants, the ETL program will populate these fields with blanks. In SAF replicants, they will be populated as specified in Section 2.7.2.

- The output field <u>SegCount</u> will reflect all coincident records, regardless of the source of their generation. Records generated from proto-segments (coincident segment entries in AltSegmentData), continuous parity replicants, SAF replicants,

and records generated from physically coincident features that are geometry-modeled in the CSCL database, will all be accounted for in the SegCount value. Note that Bytes LION files can have many more coincident segments than Geosupport LION, because of the segment replication for continuous parity conditions and for SAF data. In particular, the number of actual coincident segments in Bytes LION can exceed 9. That is expected never to occur in Geosupport LION. However, the SegCount field in Bytes LION, having been defined based on the Coincident Segment Counter field (field L57) in the Geosupport LION, is a one-byte string, and therefore cannot accommodate a value greater than 9. When there are more than 9 coincident segments in Bytes LION, the ETL program will populate SegCount with the value '9' in those output records, and will issue a warning message that will state the true segment count. **It is recommended** that DCP consider enhancing the length of the SegCount field in the Bytes LION layout to two bytes. If and when that is done, the ETL program should be modified to remove the warning and to populate SegCount with the true value when the number of coincident segments exceeds 9.

- The output field Street will be populated from the LOOKUP_KEY field in the StreetName or FeatureName table entry for the principal name of the Preferred B7SC associated with the segment (or the underlying segment, in the case of an SAF record). The Preferred B7SC is the concatenation of the fields L1 (Borough), L5 (5-Digit Street Code) and L6 (LGC1) in the corresponding Geosupport LION record.

- The output field TrafSrc is defined as a three-byte string. The CSCL source item for this field is the Centerline attribute TRAFDIR_VER_FLAG, which is defined as a one-byte string, the domain of which consists of the three values 'F', 'P' and 'T'. The ETL tool will populate TrafSrc with the values 'FLD', 'DCP' and 'DOT', respectively.

- The ESRI LION contains two sets of house number fields:

    - One set consists of the fields LLo_Hyphen, LHi_Hyphen, RLo_Hyphen and RHi_Hyphen. With the exception of certain types of segments as discussed below, these fields will be populated directly from the CSCL house numbers with no reformatting. In this format, Queens-style hyphenated house numbers are stored with an explicit hyphen character and with three digits following the hyphen.

    - The other set consists of the fields FromLeft, ToLeft, FromRight and ToRight. With the exception of certain types of segments as discussed below, these fields will be populated as follows. Queens-style hyphenated house numbers are stored in the following "normalized" format: The explicit hyphen is deleted. The digits to the left of the (deleted) hyphen are retained unmodified. The digits to the right of the (deleted) hyphen are formatted into three bytes, right-justified and zero-filled. For example, 45-03 would be stored in these fields as 45003; 45-12 would be stored as 45012; and 45-102 would be stored as 45102. If the house numbers are not Queens-style hyphenated house numbers, these fields will be populated with the same

values that are in the fields LLo_Hyphen, LHi_Hyphen, RLo_Hyphen and RHi_Hyphen.

- In output records for SAF type S and V replicants, the output house number fields will be populated as follows.

  - If the HYPHEN_TYPE attribute in the address point contains one of the values 'N' or 'Q', the address point contains a single house number, which is stored in the address point's HOUSE_NUMBER field. Form this single house number value into the output address range by populating it into all four output house number fields, "low" and "high" and "from" and "to", for the appropriate side of the segment. If HYPHEN_TYPE = 'N', the house number is unhyphenated and must be populated into all four fields unmodified. If HYPHEN_TYPE = 'Q', it is hyphenated. In that case, when populating the "from" and "to" fields, delete the hyphen character and concatenate the digits to the left and right of the hyphen as usual.

  - If HYPHEN_TYPE = 'R', the address point contains a range of unhyphenated house numbers, which is stored in the HOUSE_NUMBER field in the form of a pair of numbers separated by a dash. The dash is NOT a hyphen in this case; it is a range separation character. (Note: the dash may or may not be bracketed by blanks. For example, the range 345 to 349 might be expressed as '345–349' or as '345 – 349'.) In this case, populate the "low" and "from" output house number fields for the appropriate side of the segment with the value to the left of the dash, and populate the corresponding "high" and "to" output house number fields with the value to the right of the dash.

  - If HYPHEN_TYPE = 'X', the address point contains a range of hyphenated house numbers. In this case, the (hyphenated) low house number of the range is stored in the HOUSE_NUMBER field and the (hyphenated) high house number of the range is stored in the HOUSE_NUMBER_RANGE field. Populate the "low" and "from" output house number fields for the appropriate side of the segment with the HOUSE_NUMBER value. Populate the corresponding "high" and "to" output house number fields with the HOUSE_NUMBER_RANGE value. When populating the "from" and "to" fields, delete the hyphen character and concatenate the digits to the left and right of the hyphen as usual.

- In output records for certain types of segments, including their continuous parity and SAF replicants, all house number fields (<u>LLo_Hyphen, LHi_Hyphen, RLo_Hyphen, RHi_Hyphen, FromLeft, ToLeft, FromRight and ToRight) will be populated with zeros, regardless of the values in the CSCL source data. The applicable types of segments are as follows:</u>

  - Generic segments (Centerline segments with SEGMENT_TYPE = 'G'). (Note: the house number fields in combined generic/roadbed

segments (Centerline segments with SEGMENT_TYPE = 'B') and replicants based on them will NOT be zeroed out.)

- Proto-segments coincident with generic segments (but NOT with combined generic/roadbed segments)
- Non-physical Centerline segments (RW_TYPE = 12) (these include paper streets and faux segments)
- Proto-segments that are not physical streets (FEATURE_TYPE = any value other than null, '6' or 'W')

- The field RB_Layer will be populated based on the value of the segment's Include/Exclude Flag and its Segment Type Code (fields L58 and L56 in the Geosupport LION record, respectively). The rules are as follows:
  - If L58 = 'E', set RB_Layer = 'N' (regardless of the value in L56). Otherwise:
  - If L56 = blank, 'B', 'E' or 'U', set RB_Layer = 'B'
  - If L56 = 'G' or 'F', set RB_Layer = 'G'
  - If L56 = 'C', 'R', 'S' or 'T', set RB_Layer = 'R

  Note: If the ETL tool is programmed to use the source CSCL data to determine the populating of RB_Layer instead of the Geosupport LION output record, care must be taken to treat segments for which SEGMENT_TYPE = 'S' as roadbed segments, that is, to set RB_Layer = 'R' for those segments.

- The fields LBoro and RBoro will be populated as follows. Every output record "belongs" to a specific borough, even if it lies on a borough boundary. The borough that the output record "belongs" to is determined by the borough of the CSCL source data from which the record was generated, as follows:
  - For records generated from Centerline segments and from SAF data in the CommonPlace and AddressPoint feature classes, the borough that the output record "belongs" to is indicated by the BOROUGHCODE attribute of the given CSCL source feature.
  - For records generated from Rail, Subway, Shoreline and NonStreetFeature segments, the borough that the output record "belongs" to is the first byte of the B5SC, obtained from the Segment_LGC table by SEGMENTID, as described for the Geosupport LION in Section 2.3.1.
  - For records generated from proto-segments or from SAF data in the AltSegmentData table, the borough that the output record "belongs" to is indicated in the AltSegmentData field BOROUGH.

The output fields LBoro and RBoro will be populated based on the borough that the segment "belongs" to as defined above, and the value of the BoroBndry output field. Note that the contents of the BoroBndry field indicates the side of the segment that is out of the borough that the segment belongs to. If the BoroBdry field contains 'L' or 'R', the output field for the opposite side, RBoro or LBoro, respectively, will be populated with the borough code of the borough that the record "belongs" to, and the borough field for the other side of the segment will be populated with a null value. Note that if BoroBndry contains 'L', it is RBoro (not LBoro) that is populated with a value, and vice versa. If the BoroBdry field does not contain one of the values 'L' or 'R', both LBoro and RBoro will be populated with the borough code of the borough that the record "belongs" to.

- The field MH_RI_Flag will be populated based on the borough to which the output segment "belongs" (in the sense defined in the discussion of LBoro and RBoro above), and on the segment's left and right Borough and 2000 Census Tract values, as follows:
    - If the output segment belongs to Manhattan, and the borough/tract combination for at least one side of the segment = Manhattan tract 309 (i.e., that side is in Manhattan AND is in 2000 census tract is 309), set MH_RI_Flag = 'M'.
    - If the output segment belongs to the Bronx, and the borough/tract combination for at least one side of the segment = Bronx tract 1 (i.e., that side is in the Bronx AND is in 2000 census tract 1), set MH_RI_Flag = 'R'
    - If neither of the above conditions is satisfied, set MH_RI_Flag to null.
- The field Join_ID serves to link segments in the LION feature class with entries in the Altnames table. This field is a 15-character string and will be populated in the LION feature class as follows:
    - For output records other than those generated from SAF data, Join_ID is formed by concatenating the Borough (copied from LBoro or RBoro, whichever field is not null), the output fields Face Code, LGC1, LGC2, LGC3, LGC4, and two spaces. The LGCs are formatted numerically, that is, right-justified and zero-filled within two bytes, and are set to '00' if there is no value for the given LGC field.
    - For SAF replicants (output records generated from SAF data), Join_ID is formed by concatenating Borough (populated as described above), the SAF 5-digit street code and SAF LGC1, LGC2, LGC3, LGC4 fields (populated as described below), and SpecAddr (= the SAF type).
        - For the SAF replicants generated from AltSegmentData SAF entries (types A, B, C, D, E, O and P): the output SAF 5-digit street code subfield will be populated from the last five bytes of the B5SC field in the AltSegmentData entry. The output LGC subfields will be populated from the LGC fields in the AltSegmentData entry. The LGC fields will be formatted numerically, that is, right-justified and zero-filled within two bytes, and set to '00' if there is no value for the given LGC field.

- For SAF replicants generated from CommonPlace features (types G, N and X): the output SAF 5-digit street code subfield will be populated from the second through sixth bytes of the CommonPlace B7SC attribute. The LGC1 output field will be populated from the last two bytes of the CommonPlace B7SC attribute, right-justified and zero-filled within the two-byte subfield. The LGC2, LGC3 and LGC4 output subfields will be populated with all zeros.

- For type S SAF replicants, which are generated from AddressPoint features: the output SAF 5-digit street code subfield will be populated from the second through sixth bytes of the AddressPoint B7SC_ACTUAL attribute. The LGC1 output field will be populated from the last two bytes of the AddressPoint B7SC_ACTUAL attribute. The LGC2, LGC3 and LGC4 output subfields will be populated from AddressPointLGCs entries, if any (see Note 5 below Table 35 in Section 7.8), or with zeros if there is no corresponding AddressPointLGCs entry.

- For type V SAF replicants, which are generated from AddressPoint features: the output SAF 5-digit street code subfield will be populated from the second through sixth bytes of the AddressPoint B7SC_VANITY attribute. The LGC1 output field will be populated from the last two bytes of the AddressPoint B7SC_VANITY attribute. The LGC2, LGC3 and LGC4 output subfields will be populated from AddressPointLGCs entries, if any (see Note 5 below Table 35 in Section 7.8), or with zeros if there is no corresponding AddressPointLGCs entry.

- The fields MapFrom and MapTo are defined as three bytes each, unlike the related Geosupport LION fields L11 (From Sectional Map) and L15 (To Sectional Map), which are defined as two bytes each. Three bytes are provided in the Bytes LION fields to accommodate the inclusion of a quarter-sectional suffix, if any. As specified in Section 2.2.2, to populate L11 and L15 in Geosupport LION records, the ETL program performs a spatial query against the SectionalMap polygon feature class. For the Bytes LION files, the QuarterSectionalMap feature class will be spatially queried instead. The MapFrom and MapTo fields in the Bytes LION records will be populated as follows:

  - A spatial query will be performed for the corresponding ("from" or "to") segment endpoint against the QuarterSectionalMap polygon feature class.

  - If the spatial query for a segment endpoint fails to find a quarter-sectional polygon, the corresponding output field (MapFrom or MapTo) will be populated with blanks, and a warning message will be issued.

  - If a polygon is found, the value in the polygon's SECTIONAL_MAP attribute field and the value in the SECTIONAL_QTR_SUFFIX field (if any) will be concatenated, and the resulting string will be inserted left-justified blank-filled into the corresponding three-byte output field (MapFrom or MapTo). For example (using '#' to represent a blank):

- Sectional map 4 with no quarter-sectional suffix would be stored in the output field as '4##'.
- Map 6 with quarter-suffix C would be stored as '6C#'.
- Map 3W with no suffix would be stored as '3W#'.
- Map 2E with suffx B would be stored as '2EB'.
- Map 12 with no suffix would be stored as '12#'.
- Map 15 with suffx C would be stored as '15C'.

2.7.4. Generating the AltNames Table

The Altnames table is extracted to contain entries for all street names and feature names pertaining to all features represented in the ESRI LION. The ETL tool extracts the names from the CSCL StreetName and FeatureName tables. However, the ETL tool does not extract any names into the AltNames table for which the value of the DCP_FLAG is 'N'.

The Geodatabase AltNames Table data model can be found at folder of "Templates\lion.gdb".

A set of output AltNames records will be generated for each Join_ID value that occurs in an output record in the LION feature class. Each record in the set will contain a street name or feature name belonging to a B7SC value that is explicitly or implicitly stored in the given Join_ID. In SAF output records, the B7SCs are stored explicitly in the Join_ID, in the sense that it contains a B5SC and a list of LGCs from which B7SCs can be directly formed by concatenation. In non-SAF output records, the B7SC values in the Join_ID are stored implicitly in the sense that the Join_ID contains a Face Code value instead of a B5SC. To retrieve all the names in the StreetName or FeatureName table (for which DCP_FLAG = 'Y') for all the implicit B7SC values in such a Join_ID, it is necessary to concatenate the B5SC value that corresponds to the given Face Code value with each of the LGC values listed in that Join_ID. That B5SC value is stored in the output field StreetCode in any of the LION output segment records that contain the given Join_ID value.

Street and feature names will be stored in AltNames in two forms:

- As a single, unparsed character string in the field Street, obtained from the LOOKUP_KEY field in the given StreetName or FeatureName table entry.
- In ESRI's standard parsed format in the component fields PDir (pre-directional), PType (pre-type), SName (basic street name), SType (suffix type, also commonly known as post-type) and SDir (suffix directional, also commonly known as post-directional). These fields correspond to the parsing component fields in CSCL's StreetName table, except that there are no pre- and post-modifier fields in the ESRI LION.

The rules for populating the parsed component fields are as follows:

- For names in the CSCL FeatureName table, the entire name will be populated into the SName field from the FeatureName field LOOKUP_KEY.

- For names in StreetName, the ETL tool will populate the output ESRI LION record as follows:

1. If the StreetName entry contains values in both the PRE_MODIFIER and POST_MODIFIER fields, the ETL tool will concatenate the values in all of that entry's street name component fields, separating each consecutive pair of non-null values with a single blank, and will populate the resulting string into the 30-byte SName field, truncating on the right if necessary. If truncation was necessary, the ETL tool will issue a warning message into the output report.

2. If the StreetName entry contains a value in the PRE_MODIFIER field but not the POST_MODIFIER field, the ETL tool will concatenate the values in PRE_MODIFIER, PRE_DIRECTIONAL, PRE_TYPE and STREET_NAME, separating each consecutive pair of non-null values with a single blank, and will populate the resulting string into the 30-byte SName field, truncating on the right if necessary. If truncation was necessary, the ETL tool will issue a warning message into the output report. If there are values in the entry's POST_TYPE and/or POST_DIRECTIONAL fields, they will be copied into the SType and/or SDir field, respectively.

3. If the StreetName entry contains a value in the POST_MODIFIER field but not the PRE_MODIFIER field, the ETL tool will concatenate the values in STREET_NAME, POST_TYPE, POST_DIRECTIONAL and POST_MODIFIER, separating each consecutive pair of non-null values with a single blank, and will populate the resulting string into the 30-byte SName field, truncating on the right if necessary. If truncation was necessary, the ETL tool will issue a warning message into the output report. If there are values in the entry's PRE_DIRECTIONAL and/or PRE_TYPE fields, they will be copied into the PDir and/or PType field, respectively.

~~2.8. MapInfo LION~~

~~In general, extraction of the MapInfo (MIF format) LION is similar to that for the ESRI LION, including replication of continuous parity segments, replication of segments for SAF data, and zeroing out of house numbers for certain types of segments. One difference is that, unlike with the ESRI LION, with the MapInfo LION, segments are extracted with specific line color and line style definitions, as explained below.~~

~~The MapInfo LION consists of the following two components:~~

- ~~NYLION1.mif, a segment file (a single file for the entire City)~~

- ~~NYLION2.mif, a street name file (a single file for the entire City)~~

~~Appendix B contains an MIF directory for NYLION1.mif and NYLION2.mif provided by DCP. The spatial coordinates in NYLION1.mif will be expressed in State Plane Coordinates, NY-LI zone, NAD83, in feet. These are the same specifications as are used~~

~~for the spatial data in the CSCL geodatabase, the Geosupport LION, the ESRI LION and the District Boundaries geodatabase.~~

~~The field names in the directory for NYLION1 are generally self-explanatory. The NYLION1.mif fields correspond to fields in the ESRI LION, and they can be populated in the same manner, as described in Section 2.7 et seq. The fields FromLeft Integer, ToLeft Integer, FromRight Integer and ToRight Integer correspond to the ESRI LION fields FromLeft, FromRight, ToLeft and ToRight.~~

~~The line color and style assigned to each output segment is based on the feature type (which is populated in the output field FeatureTyp) and the non-pedestrian flag (populated in the output field NonPed). The rules based on FeatureTyp are listed in Table 18. If NonPed contains a non-blank value, the line color will be defined as red and the line style will be assigned as alternating, regardless of the FeatureTyp value.~~

~~Table 18 – Rules for Line Color and Style Based on Feature Type~~

~~FeatureTyp Value~~	~~Line Color~~	~~Line Style~~
~~0~~	~~Black~~	~~Solid~~
~~1~~	~~Red~~	~~Track line~~
~~2~~	~~Blue~~	~~Solid~~
~~3~~	~~Green~~	~~Dashed~~
~~4~~	~~Magenta~~	~~Dashed~~
~~5~~	~~Black~~	~~Dashed~~
~~6~~	~~Gray~~	~~Solid~~
~~7~~	~~Green~~	~~Dashed~~
~~8~~	~~Magenta~~	~~Dashed~~
~~9~~	~~Black~~	~~Solid~~
~~A~~	~~Light purple~~	~~Solid~~
~~B~~	~~Yellow~~	~~Solid~~

~~NYLION2.mif is a table of alternative street and feature names. Unlike the AltNames table in the ESRI LION geodatabase, which contains fields for ESRI-standard parsed components of street names, NYLION2.mif consists of only two fields: Street, a 32-byte field for the unparsed street or feature name; and Join_ID.~~

3. STREET NAME DICTIONARY

The Street Name Dictionary (SND) is extracted as a single file for the entire city. After all output records have been written, the file is sorted on the fields Borough and Street/Feature Name (that is, positions 2 through 34 of the output record).

The CSCL StreetName and FeatureName tables contain the following fields that affect SND extraction:

- LOOKUP_KEY: For names that are to be extracted to the SND, this field will contain the given street or feature name in a Geosupport-normalized format. (In particular, the character string stored in this field will always be no longer than 32 bytes for such entries.) The ETL tool will use this field to populate the street/feature name field in output SND records. (The CSCL Maintenance System will also use this field, to check for duplicates when new names are being added to StreetName and FeatureName.)

- ENDERS_FLAG: Identifies special StreetName and FeatureName entries intended only for extraction to the Enders Table output file, as described in Section 4.3. These entries will not be extracted to the SND.

- EXCEPTION_FLAG: Identifies entries to be used to generate records in the Exception Table output file, as described in Section 4.1. These entries will also be extracted to the SND, if they satisfy the filtering criteria.

Not all names in the CSCL StreetName and FeatureName tables are extracted into the SND. The filtering criteria are as follows:

- Names for which LOOKUP_KEY = null will not be extracted.

- Names for which ENDERS_FLAG = ‘Y’ will not be extracted.

- Any names that do not have a street code assigned to them will not be extracted, in spite of the fact that the LOOKUP_KEY field is not null. The ETL tool will identify these as all StreetName and FeatureName entries in which the B10SC field is unpopulated. (Note: the absence of a B10SC value is not necessarily a data error. These are likely to be names that have been supplied by City agencies such as NYPD and FDNY but have been deemed inappropriate for inclusion in the SND and thus have intentionally not had street codes assigned to them.)

Although the Geosupport SND contains two types of records, non-type S records and type S records, the ETL tool only writes out non-type S records. It will write out one non-S output record for each qualified entry in StreetName and FeatureName (each entry that satisfies the above filtering criteria).

Table 19 contains a layout of the (non-type S) output records.

Table 19 – SND Output File – Record Layout

FIC	Field	Len	Frm	To	Comments
SNDN1	Record Type	1	1	1	
SNDN2	Borough	1	2	2	
SNDN3	Street Name or Feature Name	32	3	34	
SNDN4	Primary Name Flag	1	35	35	
SNDN5	Principal Name Flag	1	36	36	
SNDN6	B10SC	11	37	47	
N/A	Filler	2	48	49	
SNDN7	Numeric Name Indicator	1	50	50	
SNDN8	Geographic Feature Type	1	51	51	
SNDN9	Length of Name	2	52	53	
SNDN10	Full Name	32	54	85	Vestige of partial names feature
N/A	Filler	2	86	87	Reserved for possible future use for a “Minimum SNL” field
N/A	Filler	20	88	107	Formerly used for 20-Byte Name, no longer required
SNDN11	Horizontal Topology Name Type Code	1	108	108	
N/A	Filler	92	109	200	

A (non-type S) record will be written for each “qualified” entry in the StreetName and FeatureName tables. The fields in the output record will be populated as indicated in Table 20 and the notes following it.

Table 20 – Populating SND Output Records

FIC	Field	Methodology to Populate
SNDN1	Record Type	Populate with the value ‘1’
SNDN2	Borough	Copy first byte of StreetName or FeatureName field B10SC
SNDN3	Street Name or Feature Name	Populate from StreetName or FeatureName field LOOKUP_KEY, store LJSF. See Note 1.
SNDN4	Primary Name Flag	If PRIMARY_FLAG = ‘Y’, populate with value ‘P’. If PRIMARY_FLAG = ‘N’, populate with ‘V’
SNDN5	Principal Name Flag	If PRINCIPAL_FLAG = ‘Y’, populate with ‘F’. If PRINCIPAL_FLAG = ‘N’, populate with ‘S’.

FIC	Field	Methodology to Populate
SNDN6	B10SC	Copy StreetName or FeatureName field B10SC
SNDN7	Numeric Name Indicator	Populate with 'N' if name contains at least one numeric digit immediately preceded by a blank; else, populate with a blank.
SNDN8	Geographic Feature Type	Copy StreetName or FeatureName field SND_FEATURE_TYPE
SNDN9	Length of Name	Populate with length of name, measured from first byte (whether or not it contains a blank) up to and including last non-blank character. Store RJZF.
SNDN10	Full Name	Copy value populated in field SNDN3 – see Note 2
SNDN11	Horizontal Topology Name Type Code	If name is from StreetName, copy StreetName field HORIZONTAL_TOPO_TYPE. If name is from FeatureName, populate with a blank.

NOTES TO TABLE 20:

1. SNDN3 (Street Name or Feature Name): This field will be populated from the StreetName or FeatureName field LOOKUP_KEY, as follows. LOOKUP_KEY contains names in a format that has been converted from the Geosupport System normalized format in which names are stored in the SND, called the "Sort Format", by compacting the name (i.e., replacing any sequences of multiple consecutive blanks with a single blank). The ETL tool will convert the name back to the Sort Format for populating into output field SNDN3 by restoring the removed blanks (if any). This will be done as follows:

    i. The name will be scanned from left to right.

    ii. If the end (last non-blank character) of the name is reached and no numeric digits ('0' through '9') have been encountered, the name will be populated into SNDN3 unmodified.

    iii. If a numeric digit is encountered, we'll use the term "current numeric string" (CNS) to refer to the string of (one or more) consecutive numeric digits that begin with the given one and are immediately followed by a non-numeric character (possibly a blank). Let L be the length of the CNS. L could be 1 or greater than 1. The CNS is processed as follows:

        A. If the CNS is at the beginning of a "word" (i.e., if it occupies the first byte position in LOOKUP_KEY or is immediately preceded by a blank), and the CNS has a length of 1, 2 or 3, a sequence of blanks called "alignment blanks" will be inserted into the name, positioned immediately preceding the CNS. The number of alignment blanks that must be inserted is equal to (4 – L). For example, if L = 2 (i.e., the CNS consists of two consecutive numeric digits), then (4 – L) = 2, and so two

alignment blanks must be inserted immediately preceding the CNS.

B. If the CNS is at the beginning of a word and L > 3, no alignment blanks will be inserted preceding the CNS.

C. If the CNS is not at the beginning of a word or at the beginning of LOOKUP_KEY (i.e., if it is immediately preceded by a non-blank character), then in addition to the alignment blanks (if any) inserted as described in step A above, a single word-separating blank will also be inserted, positioned following that preceding non-blank character, regardless of the length of the CNS.

D. If the last digit of the CNS is the last non-blank character in the name, the processing is complete. The (possibly) reformatted name will be populated into field SNDN3. Otherwise, the processing will continue below.

E. If the character immediately following the CNS is not a blank (it is necessarily non-numeric, since if it were a numeric digit it would have been included in the CNS), a single word-separating blank will be inserted, positioned immediately following the last digit of the CNS.

iv. The scan of the name in LOOKUP_KEY will continue from left to right. If another numeric digit is encountered, it will be treated as the beginning of a new CNS, and the above processing will be repeated for that CNS.

v. When the scan reaches the end (the last non-blank character) of the name, the processing is complete. The reformatted name will be populated into field SNDN3.

vi. Examples ('*' represents an inserted alignment blank, '$' represents an inserted word-separating blank; note that all existing word-separating blanks are left in place):

Name in LOOKUP_KEY	Output Name (in Sort Format)
3 AVENUE	***3 AVENUE
11 AVENUE	**11 AVENUE
108 STREET	*108 STREET
EAST 7 STREET	EAST ***7 STREET
EAST 23 STREET	EAST **23 STREET

EAST 104 STREET	EAST *104 STREET
EAST 9 STREET BLDG16	EAST ***9 STREET BLDG$**16
9/11 MEMORIAL PLAZA	***9$/$**11 MEMORIAL PLAZA

vii. SND3 will be in original format when 'SND_FEATURE_TYPE==" U"

(Misc. Structure – Rail/subway station)

2. SNDN10 (Full Name): This field is a vestige of "partial street names", a type of name that is no longer stored in the SND. (Partial street names are not to be confused with front-truncated street names.) The ETL tool will populate this field with an exact copy of the name stored in field SNDN3. **RECOMMENDATION**: If and when DCP determines definitively that SNDN10 is no longer required, the ETL tool should be modified to treat it as a filler field and populate it with blanks.

4. GEOSUPPORT SYSTEM NORMALIZING TABLES

The ETL tool can produce output files for the following Geosupport System normalizing tables:

- Exception Table, discussed in Section 4.1
- Universal Word Table and Last Word Table, discussed in Section 4.2.
- Enders Table, discussed in Section 4.3.

4.1. Exception Table

The output Exception Table is created in the form of a text file containing variable-length records. Each output record is constructed by concatenating the following three items in sequence:

- A name in the LOOKUP_KEY field obtained from an entry in the CSCL StreetName or FeatureName table, possibly modified by deleting the last word, as explained below
- A single blank
- An end-of-line (CRLF) character.

A single output file is produced for the entire city. The output file is sorted on the entire record.

Output records will be generated only for entries in the StreetName and FeatureName tables for which EXCEPTION_FLAG = 'Y' and ENDERS_FLAG = 'N'. In general, one output record will be created for each such table entry. However, in some cases, a single output record might be created corresponding to multiple table entries (see "eliminating duplicates" below).

To create the output records, the following processing must be performed:

1. <u>Examine last word for possible deletion:</u> If the last word of the exception name is listed in the CSCL UniversalWord and/or LastWord tables, that word must be deleted when formatting the exception name for output, along with the word-separating blank between the preceding word and that word. Whether such deletion is necessary will be determined by matching the last word of the exception name to the contents of each of the fields (other than OBJECTID) in the CSCL UniversalWord and/or LastWord tables. If there is an exact match to any of those fields, the last word will be deleted.

2. <u>Generate output record:</u> After deleting the last word and the blank preceding it when necessary, the output record will be formatted as described at the beginning of this section and then written into the output Exception Table file.

3. Eliminate duplicates: The dropping of last words could result in duplicate names (and therefore duplicate output records), if two or more names flagged as exceptions are identical except for their last words. The ETL tool will eliminate duplicate output records.

4. See Document of "ETL for the Exception Street Name Table.doc".

4.2. Universal Word Table and Last Word Table

The ETL tool extracts a Universal Word Table output file and a Last Word Table output file from the entries in the CSCL UniversalWord and LastWord tables, respectively. The output files are extracted in ASCII comma-delimited or "Comma-Separated Value" (CSV) format. Both output files have the same format. The fields in the output record are listed in Table 21.

Table 21 – Universal and Last Word Table Fields

FIC	Field	Source
WT1	Unique "C" Program Variable Name	See Note 1
WT2	Directional Indicator	See Note 2
WT3	Full Spelling	Corresp. field in CSCL table
WT4	Standard Abbreviation	Corresp. field in CSCL table
WT5	Shortest Abbreviation	Corresp. field in CSCL table
WT6, WT7 etc. up to WT15 (as many of these fields as necessary)	Other acceptable abbreviation(s), if any	See Note 3

NOTES TO TABLE 21:

1. WT1 (Unique "C" Program Variable Name): The ETL tool will populate this field with a four-byte character string constructed as follows. The string will be unique to the given output record within the given output file. (It will serve as part of a variable name within the source code of a Geosupport "C" program into which the Universal and Last Word table files extracted by the ETL tool will ultimately be imbedded.) The value must be unique only within each Word Table, not across both tables. The value of WT1 within each table will be derived as follows:

    - The first character will consist of the first character of the value in the field WT3 (Full Spelling). It is referred to below as the "initial letter". It will always be alphabetic.

    - The remaining three characters will consist of an integer of up to three digits, right-justified and zero-filled. The value assigned will be the next available

integer, that is, the smallest integer not yet used for the given initial letter in the given table.

- For example, the value populated into WT1 in the first Universal Table output record that is generated for an entry whose full spelling begins with the letter D would be D001. The value populated into WT1 for the next Universal Word Table output record that is generated for an entry whose full spelling also begins with D would be D002, etc. (The same values can also be used for output Last Word Table records. If the same value happens to be used in the WT1 field in a Universal Word Table output record and a Last Word Table output record, that would not imply any relationship between those records.)

- A suggested design approach is to utilize two 26 X 2 (26-row two-column) arrays in the ETL program, one for each Word Table. The first column would contain the letters of the alphabet from A to Z. The second column would contain the next available integer for the given letter. The second column would be initialized to '1' for all letters, and would be incremented when used in an output record.

2. WT2 (Directional Indicator): The ETL tool will populate field WT2 with the value 'Y' if the record is for a "directional" word, and with the value 'N' otherwise. Since the CSCL Universal and Last Word tables do not contain a field to identify directional words, the ETL tool will be "hard-coded" to identify those words. If (and only if) the field WT3 contains one of the words listed below, the record will be treated as a directional for the purpose of populating field WT2:

EAST
EASTBOUND
NORTH
NORTHBOUND
NORTHEAST
NORTHWEST
SOUTH
SOUTHBOUND
SOUTHEAST
SOUTHWEST
WEST
WESTBOUND

3. WT6 through WT15 (Other Acceptable Abbreviations): As many of these fields will be populated as necessary to accommodate the value(s) from the corresponding fields in the CSCL table.

4. See "ETL Last and Universal Word Tables.doc" for more information.

Both output files must be sorted on field WT3 (Full Spelling). Since this field is of variable length in the comma-delimited format, the ETL tool must sort the CSCL tables prior to generating the output files.

4.3. Enders Table

The Enders Table output file is extracted as a sequential, fixed-length file with a record length of 32 bytes. The ETL tool writes one output record for each entry in StreetName or FeatureName in which the ENDERS_FLAG = 'Y'. The record consists entirely of a single 32-byte field, which will be populated with the contents of the leftmost 32 bytes of the LOOKUP_KEY field from the given StreetName or FeatureName entry.

A single file is produced for the entire city. No sorting is required.

Note that the only fields that will be populated with values in StreetName and FeatureName enders entries (entries in which the ENDERS_FLAG = 'Y') are DCP_FLAG, ENDERS_FLAG, LOOKUP_KEY and EXCEPTION_FLAG. All other fields will be empty, including the B10SC, PRIMARY_FLAG etc. The LOOKUP_KEY field in enders entries always contains an (intentionally) invalid name, stored in the form that the Geosupport System normalizer would produce if it did not recognize the name as being "enders-impacted" (i.e., if the name was not listed in Geosupport's Enders Table).

As a hypothetical example, if Geosupport did not recognize the name B Q E as being enders-impacted, it would normalize it to B Q EAST. (The 'E' in B Q E actually stands for "expressway", not "east".) B Q EAST is the form in which the name will appear in the LOOKUP_KEY field in the StreetName enders entry. The ETL tool will extract B Q EAST from this enders entry into the Enders output file, where its presence will prevent Geosupport from normalizing B Q E incorrectly. (The CSCL StreetName table will also contain a "regular" (non-enders) entry for B Q E. That entry will contain a full complement of field values, including the fields B10SC, PRIMARY_FLAG etc.

See "ETL for the Enders Table V2.doc" for more information.

5. FACE CODE FILE

The ETL tool produces a separate Face Code output file for each borough. The files are sorted on bytes 2 through 5, the Face Code field (field FC2 in the layout below). Note that there is no need to include field FC1 (Borough) as a sort key, since the Face Code file will be extracted as a separate file for each borough.

Each Face Code file will contain one record for each face code value valid in the given borough. The ETL tool will create those records by writing out one output record into the appropriate borough file for each entry in the StreetName or FeatureName table that contains a non-null Face Code value.

ASSUMPTION: Each valid combination of Borough and Face Code occurs in exactly one entry in exactly one of CSCL's name tables. The Maintenance System will be designed to ensure this. The ETL tool does not validate for the existence of entries with duplicate face codes, and is not designed to eliminate such duplicates.

Table 22 contains a layout of the Face Code output file. The last column indicates the source of data for populating each field.

Table 22 – Face Code Output File

FIC	Field	Len	Frm	To	CSCL Source of Data
FC1	Borough	1	1	1	Copy first byte of StreetName or FeatureName field B10SC
FC2	Face Code	4	2	5	Copy StreetName or FeatureName field FACECODE. Store right-justified zero-filled
	Filler	1	6	6	Populate with blank
FC3	B10SC	11	7	17	Copy StreetName or FeatureName field B10SC
N/A	Filler	1	18	18	Populate with blank
FC4	Name of street or non-street feature	32	19	50	Copy StreetName or FeatureName field LOOKUP_KEY, store left-justified space-filled

6. DISTRICT EQUIVALENCY FILES

The ETL tool produces two types of district equivalency files: ThinLION and ThinFire. For each of these, the ETL tool produces a separate file for each borough. Each of these files also has a sort requirement, as noted in the respective sections below.

6.1. ThinLION

Table 23 contains a layout of the ThinLION output file. The last column in the table and the notes following the table indicate the CSCL source of data for populating each field.

ThinLION is extracted as a separate file for each borough. One output record will be written into ThinLION for each feature in the Atomic Polygon (AP) feature class belonging to the given borough. Most of the fields in the output record will be populated from attribute fields of the given Atomic Polygon.

ThinLION is sorted on the fields TL2 and TL3 (in that order of priority). Note that there is no need to include TL1 (Borough) as a sort key, since a separate file is extracted for each borough.

See "ThinLIONExtractorConfiguration.xml" and "ETL THINLION CSCL_V16 (Feb09, 2023).doc" for the output sequence of the fields.

~~Table 23 – ThinLION Output File~~

~~FIC~~	~~Field~~	~~Len~~	~~Frm~~	~~To~~	~~Format~~	~~CSCL Source~~
~~TL1~~	~~Borough~~	~~1~~	~~1~~	~~1~~		~~Copy AP attribute BOROUGH~~
~~TL2~~	~~2000 Census Tract~~	~~6~~	~~2~~	~~7~~	~~See Sec. 1.4~~	~~Populate from AP attribute CENSUSTRACT_2000~~
~~TL3~~	~~Dynamic Block~~	~~3~~	~~8~~	~~10~~		~~Populate with last 3 bytes of AP attribute ATOMICID~~
~~TL4~~	~~2000 Census Block Basic~~	~~4~~	~~11~~	~~14~~		~~Copy AP attribute CENSUSBLOCK_2000~~
~~TL5~~	~~2000 Census Block Suffix~~	~~1~~	~~15~~	~~15~~	~~Blank if none~~	~~Populate from AP attribute **CENSUSBLOCK_2000_SUFFIX**~~
~~TL6~~	~~1990 Census Tract~~	~~6~~	~~16~~	~~21~~	~~See Sec. 1.4~~	~~Populate from AP attribute CENSUSTRACT_1990~~
~~TL7~~	~~Community Development Eligibility~~	~~1~~	~~22~~	~~22~~		~~See Note 1~~
~~TL8~~	~~Community District~~	~~3~~	~~23~~	~~25~~		~~Copy AP attribute COMMDIST~~
~~TL9~~	~~Minor Census Economic Area~~	~~4~~	~~26~~	~~29~~	~~RJZF~~	~~See Note 1~~
~~TL10~~	~~Health Area~~	~~4~~	~~30~~	~~33~~	~~RJZF~~	~~See Note 1~~
~~TL11~~	~~Health Center District~~	~~2~~	~~34~~	~~35~~		~~See Note 2~~

~~FIC~~	~~Field~~	~~Len~~	~~Frm~~	~~To~~	~~Format~~	~~CSCL Source~~
~~TL12~~	~~Police Patrol Borough Command~~	~~1~~	~~36~~	~~36~~		~~See Note 3~~
~~TL13~~	~~Police Precinct~~	~~3~~	~~37~~	~~39~~	~~RJZF~~	~~See Note 3~~
~~TL14~~	~~Water Block Mapping Suppression Flag~~	~~1~~	~~40~~	~~40~~		~~Copy AP attribute WATER_FLAG~~
~~TL15~~	~~Fire Company Type~~	~~1~~	~~41~~	~~41~~	~~Blank if none~~	~~Copy first byte of AP attribute ADMIN_FIRE_COMPANY~~
~~TL16~~	~~Fire Company Number~~	~~3~~	~~42~~	~~44~~	~~RJZF, blanks if none~~	~~Copy last three bytes of AP attribute ADMIN_FIRE_COMPANY~~
~~TL17~~	~~Sanborn Borough-1~~	~~1~~	~~45~~	~~45~~	~~Blank if none~~	~~Copy field TL1~~
~~TL18~~	~~Sanborn Volume-1~~	~~3~~	~~46~~	~~48~~	~~LJSF, blanks if none~~	~~Copy AP attribute SB1_VOLUME~~
~~TL19~~	~~Sanborn Page-1~~	~~4~~	~~49~~	~~52~~	~~LJSF, blanks if none~~	~~Copy AP attribute SB1_PAGE~~
~~TL20~~	~~Sanborn Borough-2, if any~~	~~1~~	~~53~~	~~53~~	~~Blank if none~~	~~Copy field TL1~~
~~TL21~~	~~Sanborn Volume-2, if any~~	~~3~~	~~54~~	~~56~~	~~LJSF, blanks if none~~	~~Copy AP attribute SB2_VOLUME~~
~~TL22~~	~~Sanborn Page-2, if any~~	~~4~~	~~57~~	~~60~~	~~LJSF, blanks if none~~	~~Copy AP attribute SB2_PAGE~~
~~TL23~~	~~Sanborn Borough-3, if any~~	~~1~~	~~61~~	~~61~~	~~Blank if none~~	~~Copy field TL1~~
~~TL24~~	~~Sanborn Volume-3, if any~~	~~3~~	~~62~~	~~64~~	~~LJSF, blanks if none~~	~~Copy AP attribute SB3_VOLUME~~
~~TL25~~	~~Sanborn Page-3, if any~~	~~4~~	~~65~~	~~68~~	~~LJSF, blanks if none~~	~~Copy AP attribute SB3_PAGE~~
~~TL26~~	~~2010 Census Tract~~	~~6~~	~~69~~	~~74~~	~~See Sec 1.4~~	~~Copy AP attribute CENSUSTRACT_2010 – see Note 4~~
~~TL27~~	~~2010 Census Block Basic~~	~~4~~	~~75~~	~~78~~		~~Copy AP attribute CENSUSBLOCK_2010 – see Note 4~~
~~TL28~~	~~2010 Census Block Suffix~~	~~1~~	~~79~~	~~79~~	~~Blank if none~~	~~Populate from AP attribute **CENSUSBLOCK_2010_SUFFIX** – see Note 4~~

Table 23 – Thin LION Output File (updated version, Dec27, 2023)

FIC	Field	Len	Frm	To	Format	CSCL Source
TL1	Borough	1	1	1		Copy AP attribute BOROUGH
TL2	2020 Census Tract	6	2	7	See Sec. 1.4	Same as lion
TL3	Dynamic Block	3	8	10		Populate with last 3 bytes of AP attribute ATOMICID
TL4	2020 Census Block Basic	4	11	14		Copy AP attribute CENSUSBLOCK_2020
TL5	2020 Census Block Suffix	1	15	15	Blank if none	Populate from AP attribute **CENSUSBLOCK_2020_SUFFIX**
TL6	1990 Census Tract	6	16	21	See Sec. 1.4	Same as lion
TL7	Community Development Eligibility	1	22	22		See Note 1
TL8	Community District	3	23	25		Copy AP attribute COMMDIST
TL9	Minor Census Economic Area	4	26	29	RJZF	See Note 1
TL10	Health Area	4	30	33	RJZF	See Note 1
TL11	Health Center District	2	34	35		See Note 2
TL12	Police Patrol Borough Command	1	36	36		See Note 3
TL13	Police Precinct	3	37	39	RJZF	See Note 3
TL14	Water Block Mapping Suppression Flag	1	40	40		Copy AP attribute WATER_FLAG
TL15	Fire Company Type	1	41	41	Blank if none	Copy first byte of AP attribute ADMIN_FIRE_COMPANY
TL16	Fire Company Number	3	42	44	RJZF, blanks if none	Copy last three bytes of AP attribute ADMIN_FIRE_COMPANY
TL17	Sanborn Borough-1	1	45	45	Blank if none	Copy field TL1
TL18	Sanborn Volume-1	3	46	48	LJSF, blanks if none	Copy AP attribute SB1_VOLUME
TL19	Sanborn Page-1	4	49	52	LJSF, blanks if none	Copy AP attribute SB1_PAGE

FIC	Field	Len	Frm	To	Format	CSCL Source
TL20	Sanborn Borough-2, if any	1	53	53	Blank if none	Copy field TL1
TL21	Sanborn Volume-2, if any	3	54	56	LJSF, blanks if none	Copy AP attribute SB2_VOLUME
TL22	Sanborn Page-2, if any	4	57	60	LJSF, blanks if none	Copy AP attribute SB2_PAGE
TL23	Sanborn Borough-3, if any	1	61	61	Blank if none	Copy field TL1
TL24	Sanborn Volume-3, if any	3	62	64	LJSF, blanks if none	Copy AP attribute SB3_VOLUME
TL25	Sanborn Page-3, if any	4	65	68	LJSF, blanks if none	Copy AP attribute SB3_PAGE
TL26	2000 Census Tract	6	69	74	See Sec 1.4	Same as lion
TL27	2000 Census Block Basic	4	75	78		Copy AP attribute CENSUSBLOCK_2000
TL28	2000 Census Block Suffix	1	79	79	Blank if none	Populate from AP attribute **CENSUSBLOCK_2000_SUFFIX**
TL29	Assembly District	2	81	82	Blank if none	Copy AP attribute of AD
	Filler	1	83	83		
TL30	Election District	3	84	86	RJZF	Copy AP attribute of ED
TL31	Hurricane Evacuation Zone	2	87	88	LJSF	Copy AP attribute of Hurricane evacuation Zone
TL32	Patrol Borough	2	89	90	LJSF	Note 4
TL33	Police Sector	4	91	94	RJSF	Note 5
TL34	2010 Census Tract Basic	4	95	98	RJSF	Same as lion
TL34	2010 Census Tract Suffix	2	99	100	RJZF,blank filled if empty	Same as lion
TL35	2010 Census Block	4	101	104		Copy AP attribute CENSUSBLOCK_2010
TL36	2010 Census Block Suffix	1	105	105	Blank if none	Populate from AP attribute **CENSUSBLOCK_2010_SUFFIX**
TL37	NTA2020	6	106	111	blank if empty	Note 6
TL38	CDTA	4	112	115	blank if empty	Note 6

FIC	Field	Len	Frm	To	Format	CSCL Source
TL39	CWZ	4	116	119	blank if empty. RJSF	Populate from AP attribute **COMMERCIAL_WASTE_ZONE**
TL40	PUMA2020	4	120	123	blank if empty	Note 6

NOTES TO Table 23:

1. TL7 (Community Development Eligibility), TL9 (Minor Census Economic Area) and TL10 (Health Area): These fields will be populated from attributes of the feature class CensusTract2010. The access key for this feature class consists of the concatenation of the attribute fields BOROUGH and CENSUSTRACT_2010 of the AP feature class. The ETL tool will match that value to the CensusTract2010 attribute BOROCT.

    a. Populate TL7 from CensusTract2010 attribute CD_ELIGIBILITY

    b. Populate TL9 from CensusTract2010 attribute MCEA

    c. Populate TL10 from CensusTract2010 attribute HEALTH_AREA

2. TL11 (Health Center District): This field will be populated from an attribute of the feature class HealthArea. The access key for this feature class consists of the attribute field HEALTH_AREA of CensusTract2010. The ETL tool will match this value to the HEALTHAREA attribute of HealthArea. The ETL tool will populate TL11 from the HealthArea attribute HEALTH_CT_DISTRICT.

3. TL12 (Police Patrol Borough Command) and TL13 (Police Precinct): The CSCL NYPDPrecinct and NYPDBoroughCommand polygon feature classes are not based on Atomic Polygons. Consequently, the AP feature class does not contain attributes for these district types that could be captured for populating TL12 and TL13. The ETL tool will capture these values spatially, as follows. The ETL tool will generate a centroid point for the AP being processed. The ETL tool will then perform a spatial point-in-polygon search for that centroid against the NYPDPrecinct polygons. If there is an NYPDPrecinct polygon that contains the AP centroid, its BOROUGH_COMMAND and PRECINCT attribute values will be populated into TL12 and TL13 respectively. If there is no such precinct polygon, TL12 and TL13 will be populated with blanks. (The latter case will not be treated as a data error, as there may validly be areas of the city not covered by any precinct, such as some water APs.)

4. TL32 (Patrol Borough): The field will be Extracted from "PATROL_BOROUGH" in CSCL.NYPDPatrolBorough" feature class.

    the AP feature class does not contain attributes for Patrol Borough. The ETL tool will capture these values spatially, as follows. The ETL tool will generate a centroid point for the AP being processed. The ETL tool will then perform a spatial point-in-polygon search for that centroid against the NYPDPatrolBorough polygons. If there

is an NYPDPatrolBorough polygon that contains the AP centroid, its “patrol borough” will be populated into TL32

Poulate one byte from the field by using the following rules:

1- Manhattan South

2- Manhattan North

3- Bronx

4- Brooklyn South

5- Brooklyn North

6- Queens North

7- Staten Island

8- Queens South

5. TL33 (Police Sector): The field will be Extracted from "Sector" in CSCL.NYPDBeat" feature class. It is RJSF= right justified and Space filled.

   the AP feature class does not contain attributes for Police Sector. The ETL tool will capture these values spatially, as follows. The ETL tool will generate a centroid point for the AP being processed. The ETL tool will then perform a spatial point-in-polygon search for that centroid against the NYPDBeat polygons. If there is an NYPDBeat polygon that contains the AP centroid, its “Sector” will be populated into TL12

6. TL37 (NTA2020), TL38(CDTA) and TL40(PUMA2020)

   These fields will be populated from attributes of the feature class CensusTract2020. The access key for this feature class consists of the concatenation of the attribute fields BOROUGH and CENSUSTRACT_2020 of the AP feature class. The ETL tool will match that value to the CensusTract2020 attribute “BOROCT”.

   a. Populate T37 from CensusTract2020 attribute “NEIGHBORHOOD_CODE”

   b. Populate T38 from CensusTract2020 attribute “CDTA_CODE”

   c. Populate TL40 from CensusTract2020 attribute “PUMA”.

6.2. ThinFire

Table 25 contains a layout of the ThinFire output file. The last column in the table indicates the CSCL source of data for populating each field.

ThinFire is extracted as a separate file for each borough. One output record will be written into ThinFire for each feature in the FireCompany feature class. ThinFire will be sorted on the fields TF1 (Admin Fire Company Type) and TF2 (Admin Fire Company Number). Note that there is no need to include TF5 (Borough) as a sort key, since a separate file is extracted for each borough.

Table 25 – ThinFire Output File

FIC	Field	Len	Frm	To	CSCL Source of data
TF1	Admin Fire Company Type	1	1	1	First byte of FireCompany attribute UNIT_SHORT.
TF2	Admin Fire Company Number	3	2	4	Last three bytes of FireCompany attribute UNIT_SHORT.
TF3	Admin Fire Division	2	5	6	Attribute BOROUGHCOMMAND of FireCompany f.c.
TF4	Admin Fire Battalion	2	7	8	Attribute BATTALION of FireCompany f.c.
TF5	Borough	1	9	9	See Note 1

NOTE TO Table 25:

Note 1: TF5 (Borough): The FireCompany feature class does not have an attribute for Borough. Although it would be possible to determine the borough spatially by forming a fire company centroid and performing a point-in-polygon match to borough polygons or AP polygons, the following alternative method involving attribute matching appears likely to provide better performance efficiency and is recommended for consideration by the developer:

- Except for the three special cases listed below, find any AP whose ADMIN_FIRE_COMPANY attribute exactly matches the FireCompany attribute UNIT_SHORT (these two attributes, which serve as the unique identifier for a fire company, are identical data items that have been given two different names in these two feature classes). There will always be a match to at least one AP (multiple APs are likely to match in almost all cases), since FireCompany polygons are generated by dissolving APs based on ADMIN_FIRE_COMPANY. Populate TF5 with the BOROUGH attribute of any of the matching AP's.

- The following three special cases will be handled by “hard-coding”:
    - Fire Company E-81 belongs to the Bronx, even though the territory (and Atomic Polygons) it covers, the Marble Hill neighborhood, is part of Manhattan. Populate field TF5 with the value ‘2’.
    - Fire Company E-260 belongs to Queens, even though the territory it covers, Roosevelt Island, is part of Manhattan. Populate field TF5 with the value ‘4’.
    - Fire Company E-263 belongs to Queens, even though the territory it covers, Rikers Island, is part of the Bronx. Populate field TF5 with the value ‘4’.

7. SPECIAL ADDRESS FILE

There are 13 Special Address File (SAF) output record types, classified as types A, B, C, D, E, G, I, N, O, P, S, V and X. (The CSCL database also contains SAF data identified as type F, but the ETL tool re-classifies these as type D records in the output files.) Each SAF record pertains to a particular geographic feature, as follows:

- Type I records represent named street intersections and pertain to nodes, which are identified by NODEID.
- SAF records of types other than I pertain to segments of linear features. In the output SAF files, appurtenant segments are identified by LION Keys.
    - Type A, B, C, D, E, O, P, S and V records contain address information and pertain to street segments, which are represented in the CSCL database either as Centerline segments or as street feature proto-segments (coincident segment entries in the AltSegmentData table with an implicit feature class of Centerline).
    - Type G, N and X records contain Non-Addressable Place names (NAPs) and pertain either to street segments or proto-segments, or to Shoreline segments.

In the CSCL database, the SEGMENTID of a linear feature-related SAF record may identify a Centerline or Shoreline segment that has a proto-segment (i.e., an AltSegmentData coincident segment entry with the same SEGMENTID). In that case:

- The proto-segment is necessarily either a coincident borough boundary segment (ALT_SEGDATA_TYPE = ‘B’) or a boundary continuity segment (ALT_SEGDATA_TYPE = ‘C’). It is never a combined on-off ramp (ALT_SEGDATA_TYPE = ‘R’), since SAF records never pertain to ramps.
- If the geometry-represented segment is a Shoreline segment, the SAF record necessarily pertains to it, not to the proto-segment. That is because proto-segments that are coincident with Shoreline segments are always non-physical borough boundary features and thus never have appurtenant SAF data. (Note: G, N and X are the only SAFTYPEs that can pertain to a Shoreline segment.)
- If the geometry-represented segment is a Centerline segment and the proto-segment is a boundary continuity segment, the SAF record necessarily pertains to the geometry-represented segment. Boundary continuity proto-segments are non-physical features and never have appurtenant SAF data.
- If the geometry-represented segment is a Centerline segment and the proto-segment is a borough boundary segment, the SAF record pertains to the entity (the geometry-represented segment or the proto-segment) that belongs to the same borough as the SAF record.

Whether an SAF record pertains to a geometry-represented segment or a proto-segment affects how certain fields in certain SAFTYPEs will be populated, as noted in the sections

on those SAFTYPEs. That is because feature-specific data items (such as street codes, Face Codes and Sequence Numbers) differ between the geometry-represented segment and the proto-segment.

In some cases different SAF record types have the same record layout, although they may utilize different subsets of the fields in that layout or may utilize the same field differently. Specifically, the 13 SAF record types have five distinct layouts, grouped as follows:

- Record types A, B, C, E, G, N, P and X have the layout shown in Table 28.
- Record types O and V have the layout shown in Table 33 and Table 35 (the same layout is in both tables, but the populating rules differ).
- Record type D has the layout shown in Table 30.
- Record type I has the layout shown in Table 32.
- Record type S has the layout shown in Table 34.

SAF record types that have the same layout will be written into the same output file. The ETL tool will produce a single output file for the type I records. The tool will produce two output files for each of the four segment-related record layouts: a generic version and a roadbed version. Therefore, the tool will produce a total of nine SAF output files.

Except for the type I records, the file version(s) into which the ETL tool will write a particular SAF output record depend(s) on the combination of the SAFTYPE of the SAF record and the SEGMENT_TYPE and INCEX_FLAG of the Centerline segment to which the SAF record is appurtenant, as shown in Table 26. For type G, N and X records only, it is possible for the record to pertain to a segment belonging either to the Centerline feature class (or a proto-segment whose implicit feature class is Centerline) or to the Shoreline feature class; if it pertains to a Shoreline segment, the output record will be included in both the generic and roadbed versions.

In Table 26, the column labeled “FIC String” lists the alphabetic (file-identifying) portion of the FIC for the given SAF output file. Since the generic and roadbed versions of a given SAF output file have identical layouts, the same FIC String will be used for both versions.

Table 26 – SAF Output File Rules

FIC String	SAFTYPEs	SEGMENT_TYPEs to Include in the Generic Version	SEGMENT_TYPEs to Include in the Roadbed Version
SAFI	I	N/A	N/A
SAFA	A, B, C, E, G, N, P, X	B, E, G (except when INCEX_FLAG = ‘E’), U or Shoreline segment	E, R, S (proposed value), U or Shoreline segment

FIC String	SAFTYPEs	SEGMENT_TYPEs to Include in the Generic Version	SEGMENT_TYPEs to Include in the Roadbed Version
SAFD	D	B, E, G (except when INCEX_FLAG = 'E'), U	E, R, S, U
SAFV	O, V	B, E, G (except when INCEX_FLAG = 'E'), U	E, R, S, U
SAFS	S	B, E, G (except when INCEX_FLAG = 'E'), U	E, R, S, U

7.1. CSCL Source Components for SAF Data

The ETL tool will generate SAF output records from certain features or table entries in four CSCL database components, depending on the SAFTYPE, as shown in Table 27. Certain output fields will be obtained from related CSCL components, such as the StreetName and FeatureName tables and the RoadbedPointerList table.

Table 27 – CSCL Source Components for SAF Data

SAFTYPEs	CSCL Source Component
I	NamedIntersection table
A, B, C, D, E, O, P	AltSegmentData table SAF entries (see Table 13)
G, N, X	CommonPlace feature class
S, V	AddressPoint feature class

7.1.1. Use of the Roadbed Pointer List

With respect to the extraction of SAF record types A, B, C, D, E, O and P, the AltSegmentData table includes the generic and roadbed entries that are needed to produce the required records for both the generic and roadbed versions of the respective SAF output files.

In the CommonPlace and AddressPoint feature classes, the SAF type G, N, S, V and X data associated with divided street segments point only to roadbed segments or proto-segments coincident with roadbed segments. To produce the required generic output records for record types G, N, S, V and X, the ETL tool will access the CSCL RoadbedPointerList table to obtain the SEGMENTID of the corresponding generic segment or proto-segment, and will then access that generic segment in the Centerline feature class or proto-segment in the AltSegmentData table to obtain certain data items

required to construct the output generic SAF record, such as the generic segment's or proto-segment's Sequence Number.

RPL access is not required for Type G, N and X records that pertain to non-Centerline features.

7.1.2. Normalized Names in SAF Records

All of the SAF record types have a field for a name as the first 32 bytes of the record. Depending on the record type, the name is a street name, a place name or an intersection name. The name is required to be in a Geosupport System normalized format called the "Sort Format".

All output SAF name fields will be populated by obtaining the name from the field LOOKUP_KEY in the appropriate entry in the StreetName table or FeatureName table, and converting the contents of that field into the Sort Format as described in detail in Note 1 to Table 20.

7.2. Type A, B, C, E and P Records

Eight of the SAF record types, types y share the layout shown in Table 28.

Table 28 – Layout for SAF Record Types A, B, C, E, G, N, P and X

FIC	Field	Len	Frm	To	Format
SAFA1	Special Address Street Name, Neighborhood Name or Place Name	32	1	32	LJSF
SAFA2	Borough	1	33	33	1, 2, 3, 4 or 5
SAFA3	Face Code	4	34	37	RJZF
SAFA4	Sequence Number	5	38	42	RJZF
SAFA5	SOS Indicator	1	43	43	L, R or blank
SAFA6	B5SC	6	44	49	
SAFA7	Left Low HN	7	50	56	RJSF, or 6 blanks followed by a zero if none
	Filler	2	57	58	
SAFA8	Left High HN	7	59	65	RJSF, or 6 blanks followed by a zero if none
	Filler	2	66	67	
SAFA9	Right Low HN	7	68	74	RJSF, or 6 blanks followed by a zero if none
	Filler	2	75	76	

FIC	Field	Len	Frm	To	Format
SAFA10	Right High HN	7	77	83	RJSF, or 6 blanks followed by a zero if none
	Filler	2	84	85	
SAFA11	SAF Record Type Code	1	86	86	
	Filler	2	87	88	
SAFA12	LGC1	2	89	90	RJZF
SAFA13	LGC2	2	91	92	RJZF, blanks if none
SAFA14	LGC3	2	93	94	RJZF, blanks if none
SAFA15	LGC4	2	95	96	RJZF, blanks if none
SAFA16	Pointer to BOE Preferred LGC	1	97	97	1, 2, 3 or 4
SAFA17	Segment Type Code	1	98	98	
SAFA18	Segment ID	7	99	105	RJZF
SAFA19	X Coordinate of Type X,G or N	7	106	112	RJZF, blanks if none
SAFA20	Y Coordinate of Type X,G or N	7	113	119	RJZF, blanks if none
SAFA21	Side borough code X, G or N	1	120	120	blanks if none
SAFA22	Side CT2010 X,G or N (XXXXYY)	6	121	126	Same format as Geo-support Lion
SAFA23	Side AP for X,G or N	3	127	129	RJZF, blanks if none

This section discusses the generation of SAF Type A, B, C, E and P records. Section 7.4 discusses Types G, N and X.

SAF Type A, B, C, E and P records are extracted from the entries in the AltSegmentData table having those SAFTYPE values. The output records will be written into the generic SAFA file, the roadbed SAFA file or both, depending on the SEGMENT_TYPE of the appurtenant Centerline segment, in accordance with the rules in Table 26.

Table 29 contains a list of the fields in the SAFA Type A, B, C, E and P output records, along with the CSCL source of data for populating each field. The source for most of the fields is the AltSegmentData entry from which the given output record is being generated, as indicated in the last column in Table 29. The sources for other output fields are described in the notes below Table 29.

Table 29 – Populating SAF Record Types A, B, C, E and P

FIC	Field	CSCL Source of Data
SAFA1	Special Address Street Name or Neighborhood Name or Addressable Place Name	Except for type C records, obtained from StreetName table – see Note 1. For type C records, populate with the value '75 STREET'.
SAFA2	Borough	AltSegmentData field BOROUGH
SAFA3	Face Code	Obtained from bytes 2 through 5 of AltSegmentData LIONKEY field
SAFA4	Sequence Number	Obtained from bytes 6 through 10 of AltSegmentData LIONKEY field
SAFA5	SOS Indicator	AltSegmentData field SOS_INDICATOR – see Note 2
SAFA6	B5SC	AltSegmentData field B5SC
SAFA7	Left Low HN	AltSegmentData field L_LOW_HN
	Filler	N/A
SAFA8	Left High HN	AltSegmentData field L_HIGH_HN
	Filler	N/A
SAFA9	Right Low HN	AltSegmentData field R_LOW_HN
	Filler	N/A
SAFA10	Right High HN	AltSegmentData field R_HIGH_HN
	Filler	N/A
SAFA11	SAF Record Type Code	AltSegmentData field SAFTYPE
	Filler	N/A
SAFA12	LGC1	AltSegmentData field LGC1
SAFA13	LGC2	AltSegmentData field LGC2
SAFA14	LGC3	AltSegmentData field LGC3
SAFA15	LGC4	AltSegmentData field LGC4
SAFA16	Pointer to BOE Preferred LGC	AltSegmentData field BOE_PREFERRED_LGC_FLAG
SAFA17	Segment Type Code	See Note 3
SAFA18	Segment ID	AltSegmentData field SEGMENTID
N/A	14-byte filler	Populate with blanks – see Note 4

NOTES TO Table 29:

1. SAFA1 (Street Name): For type C records, the ETL Tool will be "hard-coded" to populate this field with '75 STREET'. For types A, B, E and P records, this field will be populated with the preferred street name in the Geosupport Sort Format. It will be obtained from the LOOKUP_KEY field in the pertinent StreetName or FeatureName table entry as follows.

    a. The AltSegmentData fields B5SC and LGC1 will be concatenated to form the Preferred B7SC. (**ASSUMPTION**: the AltSegmentData field LGC1 always contains the Preferred LGC.)

b. StreetName will be accessed with a composite key consisting of the Preferred B7SC and PRINCIPAL_FLAG = 'Y'.

c. If the StreetName access fails, FeatureName will be accessed with the same key. If that is successful, a warning message will be issued.

d. If one of the name table accesses is successful, the contents of LOOKUP_KEY from that table entry will be converted to Sort Format as described in Note 1 to Table 20 and then populated into SAFA1.

e. If neither name table access is successful, SAFA1 will be populated with blanks and an error message will be issued.

f. Street name in the first 32 characters should be in sort format, but spaces before & after dash or slash should be removed. To treat anything with a dash or slash as an alphabetic word, not numeric even beginning with number. The same changes also apply to SND & exception table.

2. SAFA5 (SOS Indicator): If AltSegmentData field SOS_INDICATOR = '1', SAFA5 will be populated with 'L'. If SOS_INDICATOR = '2', SAFA5 will be populated with 'R'. Otherwise, SAFA5 will be populated with a blank.

3. SAFA17 (Segment Type Code): This field will be populated with the value of the appurtenant Centerline segment's SEGMENT_TYPE attribute.

4. Filler: This 14-byte filler corresponds to fields SAFA19 and SAFA20, which are populated with spatial coordinates only for type G, N and X records. For type A, B, C, E and P records, this filler field will be populated with blanks. The purpose of the filler is to preserve the shared layout that has existed between type A, B, C, E and P records and type G, N and X records.

7.3. Type D Records

Table 30 contains a layout of the SAF Type D output records. The last column in the table and the notes immediately following the table indicate the CSCL source of data for populating each output field.

SAF Type D data are extracted from the Type D and F entries in the AltSegmentData table. Those entries are identified by the values 'D' and 'F' in the field SAFTYPE. Note that, although AltSegmentData contains SAF entries identified as Type F, their SAF Type will be changed to 'D' in the extracted SAF output file.

Type D output records will be written into the generic SAFD file, the roadbed SAFD file or both, depending on the SEGMENT_TYPE of the appurtenant Centerline segment, in accordance with the rules in Table 26.

Table 30 – Layout and Populating Rules for SAF Record Type D

FIC	Field	Len	Frm	To	Format	CSCL Source of Data
SAFD1	Special Address Street Name	32	1	32	LJSF	Obtained from StreetName table – see Note 1
SAFD2	Borough	1	33	33	1, 2, 3, 4 or 5	AltSegmentData field BOROUGH
SAFD3	Face Code	4	34	37	RJZF	Obtained from bytes 2 through 5 of AltSegmentData LIONKEY field
SAFD4	Sequence Number	5	38	42	RJZF	Obtained from bytes 6 through 10 of AltSegmentData LIONKEY field
SAFD5	SOS Indicator	1	43	43	L, R or blank	AltSegmentData field SOS_INDICATOR – see Note 2
SAFD6	DAPS B5SC	6	44	49		AltSegmentData field B5SC
SAFD7	Low HN	7	50	56	RJSF	AltSegmentData field L_LOW_HN or R_LOW_HN, whichever is non-null – see Note 3
	Filler	2	57	58		N/A
SAFD8	High HN	7	59	65	RJSF	AltSegmentData field L_HIGH_HN or R_HIGH_HN, whichever is non-null – see Note 3
	Filler	2	66	67		N/A
SAFD9	Regular B5SC	6	68	73		See Note 4
	Filler	12	74	85		N/A
SAFD10	SAF Record Type Code	1	86	86		Populate with the value ‘D’ (even if the AltSegmentData field SAFTYPE contains an ‘F’)
SAFD11	Zip Code	5	87	91		AltSegmentData field ZIPCODE
SAFD12	DAPS Type	1	92	92		See Note 5
	Filler	5	93	97		N/A
SAFD13	Segment Type Code	1	98	98		See Note 6
SAFD14	Segment ID	7	99	105	RJZF	AltSegmentData field SEGMENTID

NOTES TO Table 30:

1. SAFD1 (Street Name): This field will be populated with the primary name of the given Duplicate Address Pseudo-Street name (DAPS) in Geosupport Sort Format. It will be obtained by accessing the LOOKUP_KEY field in the pertinent StreetName table entry with a composite key consisting of the B5SC from the AltSegmentData entry and PRIMARY_FLAG = 'Y'. The contents of LOOKUP_KEY in that entry will be converted to Sort Format as described in Note 1 to Table 20.

2. SAFD5 (SOS Indicator): If AltSegmentData field SOS_INDICATOR = '1', SAFD5 will be populated with 'L'. If SOS_INDICATOR = '2', SAFD5 will be populated with 'R'. Otherwise, SAFD5 will be populated with a blank.

3. SAFD7 (Low HN) and SAFD8 (High HN): SAF Type D records contain only a single address range, which corresponds to either the left side or the right side of the appurtenant segment, as indicated by the SOS Indicator in the AltSegmentData SAF entry. In the AltSegmentData entry for a Type D SAF record, the house number fields for the side indicated by the SOS Indicator contain the given address range, and the house number fields for the other side of the segment are null. SAFD7 and SAFD8 will be populated from the non-null AltSegmentData low and high house number fields, respectively.

4. SAFD9 (Regular B5SC)[2]: The ETL program will be hard-coded to populate SAFD9 based on the value that is populated into SAFD6 as specified in the list below. For the SAFD6 value listed in the first column, the ETL program will populate SAFD9 with the value in the third column. (Street names are shown for reference purposes only.) If the SAFD6 value does not appear in the below list, the ETL tool will issue an error message and will not write an output record.

SAFD6 Value	DAPS Street Name	SAFD9 Value	"Regular" Street Name
119450	EAST 125 STREET	126294	DR M L KING JR BOULEVARD
136270	WEST 125 STREET	126294	DR M L KING JR BOULEVARD
438652	CENTRE DRIVE MALBA	438650	CENTRE DRIVE
438654	CENTRE DRIVE DOUGLASTON	438650	CENTRE DRIVE
449692	HILLSIDE AVENUE ROCKAWAYS	449690	HILLSIDE AVENUE
449694	HILLSIDE AVENUE BELLEROSE	449690	HILLSIDE AVENUE
449697	HILLSIDE AVENUE DOUGLASTON	449690	HILLSIDE AVENUE
400034	7 AVENUE MALBA	405340	7 AVENUE
400269	7 AVENUE ROCKAWAY	405340	7 AVENUE
400270	8 AVENUE MALBA	405390	8 AVENUE
400271	8 AVENUE ROCKAWAY	405390	8 AVENUE
400272	9 AVENUE MALBA	405490	9 AVENUE
400274	9 AVENUE ROCKAWAY	405490	9 AVENUE

[2] This design assumes that there are no Type D SAF records that pertain to borough boundary segments, as is true as of this writing. It is not expected that there will ever be any new duplicate addresses. However, if there should be a Type D record in the future that does pertain to a borough boundary segment, the B5SC value for the output SAF record would have to be obtained from either the geometry-based segment or the proto-segment, whichever has the same Borough Code as the given AltSegmentData SAF Type D or F entry. That would require an ETL program modification.

SAFD6 Value	DAPS Street Name	SAFD9 Value	"Regular" Street Name
400264	VAN WYCK EXPRESSWAY S OZONE PRK	466290	I 678
400268	WHITESTONE EXPRESSWAY FLUSHING	466290	I 678
200948	LELAND AVENUE HARDING PARK	245820	LELAND AVENUE
200952	LELAND AVENUE SOUNDVIEW	245820	LELAND AVENUE
200045	UNDERHILL AVENUE HARDING PARK	270320	UNDERHILL AVENUE
201091	UNDERHILL AVENUE SOUNDVIEW	270320	UNDERHILL AVENUE
200953	BRONX RIVER AVENUE HARDING PARK	215420	BRONX RIVER AVENUE
200954	BRONX RIVER AVENUE SOUNDVIEW	215420	BRONX RIVER AVENUE

Hard-coding for SAFD9 is necessary because the appropriate value is not necessarily the B5SC stored in Segment_LGC. A Centerline segment that has an associated SAF type D record might also have one or more type B records, which by definition contain alternative B5SC values applicable to the segment. The appropriate value for populating SAFD9 might be one of those values or it might be the B5SC in Segment_LGC. The ETL program has no means to make that determination.

In the future, if the street codes assigned to any of the street names in the above list are changed, or if SAF type D or F AltSegmentData entries are created for any additional streets, the ETL program hard-coding will have to be modified accordingly.

5. SAFD12 (DAPS Type): If the AltSegmentData field SAFTYPE contains an 'F', SAFD12 will be populated with the value '1' (signifying a "pseudo-DAPS"). If it contains a 'D', SAFD12 will be populated with a blank.

6. SAFD13 (Segment Type Code): This field will be populated with the value of the appurtenant Centerline segment's SEGMENT_TYPE attribute. (Note that if the SAF record pertains to a proto-segment, the SEGMENT_TYPE attribute of the Centerline segment applies to that proto-segment also.)

7.4. Type G, N and X Records

Output SAF type G, N and X records are written for each feature in the CSCL CommonPlace feature class for which the SAFTYPE attribute field contains a 'G', 'N' or 'X' and the B7SC field is populated with a value other than NULL or blanks, and "Security_level" isn't "Publis Safety Agency". No SAF records are written out for other CommonPlace features. In addition, if the SAFTYPE = 'G', 'N' or 'X' but the B7SC is NULL or blanks, an error message is issued.

The output records are written into the generic SAFA file, the roadbed SAFA file or both, depending on whether the CommonPlace feature pertains to a Centerline or Shoreline segment, and if the former, depending on the SEGMENT_TYPE, in accordance with the rules in Table 26.

When writing type G, N and X records into the generic SAFA file, if the SEGMENTID attribute of the CommonPlace feature identifies a roadbed segment or proto-segment, the ETL program will access the RPL as described in Section 7.1.1 to obtain the SEGMENTID of the corresponding generic (or combined generic and roadbed) segment or proto-segment. As detailed in the notes to Table 31, the ETL program will then populate the output fields SAFA3, SAFA4, SAFA17 and SAFA18 with the values of these items that pertain to the generic segment or proto-segment instead of to the roadbed segment or proto-segment.

Table 31 contains a layout of the Type G, N and X output records. (It is the same layout as for Type A, B, C and E records, but the CSCL data sources and the contents of some of the output fields are different.) The sources for populating the fields in a Type G, N or X output record are indicated in the last column in Table 31 and in the notes below the table. For most of the fields, the attribute data of the CommonPlace feature from which the given output record is being generated will serve as the source data for both the roadbed output file and the generic output file.

Table 31 – Populating SAF Type G, N and X Records

FIC	Field	CSCL Source of Data
SAFA1	Place Name	Obtained from FeatureName table - see Note 1
SAFA2	Borough	CommonPlace attribute BOROUGHCODE
SAFA3	Face Code	See Note 2
SAFA4	Sequence Number	See Note 2
SAFA5	SOS Indicator	CommonPlace attribute SOSINDICATOR – see Note 3
SAFA6	B5SC of NAP	First 6 bytes of CommonPlace attribute B7SC
SAFA7	Left Low HN	Populate with blanks – see Note 4
	Filler	Populate with blanks
SAFA8	Left High HN	See Note 5
	Filler	Populate with blanks
SAFA9	Right Low HN	Populate with blanks – see Note 4
	Filler	Populate with blanks
SAFA10	Right High HN	See Note 5
	Filler	Populate with blanks
SAFA11	SAF Record Type Code	CommonPlace attribute SAFTYPE
	Filler	Populate with blanks
SAFA12	LGC1 of NAP	Populate with the 7th and 8th bytes of CommonPlace attribute B7SC
SAFA13	LGC2 of NAP	Populate with blanks – see Note 6
SAFA14	LGC3 of NAP	Populate with blanks – see Note 6

FIC	Field	CSCL Source of Data
SAFA15	LGC4 of NAP	Populate with blanks – see Note 6
SAFA16	Pointer to BOE Preferred LGC	Populate with the value '1'
SAFA17	Segment Type Code	See Note 7
SAFA18	Segment ID	See Note 8
SAFA19	X Coordinate of Place	Populate with X coordinate of CommonPlace point rounded to nearest whole integer
SAFA20	Y Coordinate of Place	Populate with Y coordinate of CommonPlace point rounded to nearest integer
SAFA21	Side borough code	Blank if none
SAFA22	Side CT2010 (XXXXYY)	Same format as Geo-support Lion
SAFA23	Side AP	RJZF, blanks if none

NOTES TO Table 31:

1. SAFA1 (Place Name): This field will be populated with the preferred name of the given Non-Addressable Place name (NAP) in Geosupport Sort Format. It will be obtained from the LOOKUP_KEY field in the pertinent FeatureName or StreetName table entry. The ETL program will query FeatureName first (since the great majority of SAF type G, N and X names are in FeatureName). If not found in FeatureName, the program will try StreetName. If not found in either table, the program will not write out an output record, and will issue an error message. The given name table will be accessed using a composite key consisting of the CommonPlace field B7SC combined with setting the FeatureName or StreetName field PRINCIPAL_FLAG to 'Y'. The contents of LOOKUP_KEY in that entry will be converted to Sort Format as described in Note 1 to Table 20 and then populated into SAFA1.

2. SAFA3 (Face Code) and SAFA4 (Sequence Number): These fields will be populated with the Face Code and Sequence Number of the appropriate segment or proto-segment. In general, that is the segment or proto-segment identified by the SEGMENTID attribute of the CommonPlace feature. However, when the generic output file is being generated, if that segment or proto-segment is a roadbed feature, the appropriate segment from which to obtain the Face Code and Sequence Number is the corresponding generic segment or proto-segment, as explained below.

   a. <u>When generating the roadbed output file</u>, and when generating the generic output file in cases in which the SEGMENTID of the CommonPlace feature identifies a segment or proto-segment that is not a roadbed, then that segment or proto-segment is the one to which the SAF record pertains and whose Face Code and Sequence Number are to be used. In these cases, determining values for fields SAFA3 and SAFA4 is more complicated for Type G, N and X records than for the SAF record types that are generated from the AltSegmentData table, since unlike that table, the CommonPlace feature class does not contain a LIONKEY field from which to extract the

Face Code and Sequence Number. The segment appurtenant to an SAF Type G, N or X record might be a street segment or a shoreline segment. In addition, if the appurtenant segment happens to be a coincident segment, it might be stored in the CSCL database as a geometry-represented segment or as a proto-segment. In view of these facts, the ETL tool will obtain the appropriate Face Code and Sequence Number via the following steps:

1. Determine whether the SEGMENTID attribute of the CommonPlace feature identifies a Centerline segment. (The overwhelming majority of Type G, N and X SAF records pertain to street segments rather than to shorelines. Therefore, it is more efficient for the ETL program to search the Centerline feature class first.)

2. If there is such a Centerline segment, examine the attribute field COINCIDENT_SEG_COUNT. If that field contains an integer greater than 1, the segment might have a corresponding proto-segment (an entry in the AltSegmentData table with the same SEGMENTID and with ALT_SEGDATA_TYPE not null (signifying a coincident segment entry rather than an SAF entry)).

   If there is a corresponding proto-segment, it is either a borough boundary proto-segment or a boundary continuity proto-segment. (SAF data never pertain to ramps.) The ETL tool must determine the appropriate entity (either the geometry-represented segment or the proto-segment) from which to obtain the Face Code and Sequence Number as follows:

   <u>If the proto-segment is a borough boundary segment</u> (ALT_SEGDATA_TYPE = ‘B’): use the entity that has the same Borough Code as the SAF record as the source of the data.

   <u>If the proto-segment is a boundary continuity segment</u> (ALT_SEGDATA_TYPE = ‘C’): the appropriate source is necessarily the geometry-represented segment. Boundary continuity proto-segments are non-physical features and never have SAF data.

   <u>If there is no proto-segment, or if there is one but the appropriate source for the Face Code and Sequence Number is the geometry-represented segment</u>, the Sequence Number will be obtained from the geometry-represented segment’s SEGMENT_SEQNUM attribute, and the Face Code will be obtained by accessing the Segment_LGC table by SEGMENTID, then accessing the StreetName table by the Preferred B7SC and PRINCIPAL_FLAG = ‘Y’ in the usual manner. If StreetName contains no entry satisfying these search criteria, or the entry does exist but it does not contain a Face Code value, the ETL tool will not write out an output record and will issue an error message.

If the appropriate source is the proto-segment, the Face Code and Sequence Number will be obtained from byte positions 2 through 5 and positions 6 through 10, respectively, of the LIONKEY field in the proto-segment's AltSegmentData entry. If either of those items is not an integer value > 0, the ETL tool will not write out an output record and will issue an error message.

3. If there is no Centerline segment whose SEGMENTID attribute equals that of the given CommonPlace feature, there should be such a segment in the Shoreline feature class. If so, that segment's SEGMENT_SEQNUM attribute will be used to populate SAFA4, and SAFA3 will be populated with the Face Code obtained by accessing the Segment_LGC table by SEGMENTID, then accessing the FeatureName table by the Preferred B7SC and PRINCIPAL_FLAG = 'Y' in the usual manner.

4. If there is neither a Centerline segment nor a Shoreline segment having the SEGMENTID of the CommonPlace feature, the tool will treat that as an error condition in the usual manner (it will not write out an output record, and will issue an error message). All SAF records (other than type I records) <u>must</u> be appurtenant to a valid street segment or shoreline segment and therefore must contain a valid LION Key.

b. <u>When generating the generic output file in cases in which the SEGMENTID of the CommonPlace feature identifies a roadbed segment, the appurtenant segment is</u> the generic segment or proto-segment that corresponds to that roadbed segment. Its SEGMENTID is obtained from the RPL as discussed in Section 7.1.1. In this case, SAFA3 and SAFA4 will be populated as follows:

1. <u>SAFA3 will be populated with the Face Code of the generic segment or proto-segment, which will be obtained as follows.</u>

   - <u>If the generic feature is represented by a (geometry-modeled) Centerline segment, the Segment_LGC table will be accessed by the combination of the generic SEGMENTID and PREFERRED_FLAG = 'Y'. The B5SC and LGC from that Segment_LGC entry will be concatenated to form the preferred B7SC. Since the related segment belongs to a divided street, this B7SC should appear in the StreetName table rather than FeatureName. StreetName will be accessed by the combination of the preferred B7SC and PRINCIPAL_FLAG = 'Y' to retrieve the Face Code of the generic segment.</u> If StreetName contains no entry satisfying these search criteria, or the entry does exist but it does not contain a Face Code value, the ETL tool will not write out an output record and will issue an error message.

   - <u>If the generic feature is represented by a proto-segment, the Face Code will be obtained from the second through fifth bytes</u>

of the LIONKEY field in that proto-segment's AltSegmentData entry. If either of those items is not an integer value > 0, the ETL tool will not write out an output record and will issue an error message.

2. SAFA4 will be populated with the Sequence Number obtained from the generic segment or proto-segment, accessed by the generic SEGMENTID, as follows.

   - If the generic feature is represented by a Centerline segment, the Sequence Number will be obtained from that segment's SEGMENT_SEQNUM attribute.

   - If the generic feature is represented by a proto-segment, the Sequence Number will be obtained from the six through tenth bytes of the LIONKEY field in that proto-segment's AltSegmentData entry. If that item is not an integer value > 0, the ETL tool will not write out an output record and will issue an error message.

3. SAFA5 (SOS Indicator): If CommonPlace field SOS_INDICATOR = '1', SAFA5 will be populated with 'L'. If SOS_INDICATOR = '2', SAFA5 will be populated with 'R'. Otherwise, SAFA5 will be populated with a blank.

4. SAFA7 (Left Low HN) and SAFA9 (Right Low HN): The ETL tool will populate these fields with blanks. Note that the GRID Generation System populates the Low HN field for the side identified by the SOS Indicator with a value.

5. SAFA8 (Left High HN) and SAFA10 (Right High HN): The High HN field for the side identified by the SOS Indicator is intended to be populated with the house number of an "underlying address" for the given place name, if one is known. The CommonPlace feature class has an attribute called PRIMARYADDRESSPOINTID, which contains an ADDRESSPOINTID value if an underlying address for the place name is known. If PRIMARYADDRESSPOINTID does not contain a value, the ETL tool will populate the High HN field with blanks. If PRIMARYADDRESSPOINTID does contain a value, the ETL tool will use it to retrieve the feature in the AddressPoint feature class whose ADDRESSPOINTID attribute equals the given PRIMARYADDRESSPOINTID value, and will populate the High HN field for the side identified by the SOS Indicator as follows:

   - If the HYPHEN_TYPE attribute of the address point contains the value 'R', the address point's HOUSE_NUMBER attribute contains a range of unhyphenated house numbers expressed in the form of the low house number of the range followed by a dash followed by the high house number of the range. (Note: there might or might not be spaces between the dash and the digits to the left and/or right of the dash. For example, the data in HOUSE_NUMBER might appear as '22-28' or as '22 – 28'.) The ETL tool will populate SAFA8 or SAFA10 (whichever of these output fields corresponds to the side identified by the SOS Indicator) with the low-address

portion of the range (the string of digits to the left of the dash) only (after reformatting that house number into the special Edgewater Park format if required, as explained below); the dash and the digits to the right of the dash will not be used.

- If the HYPHEN_TYPE attribute of the address point contains any value other than 'R', the ETL tool will populate SAFA8 or SAFA10 with the value in the address point's HOUSE_NUMBER field without modification (except possibly for reformatting the house number into the special Edgewater Park format if required, as explained below).

- Note that the SAF output record layout for SAF types including G, N and X does not include fields for house number suffixes. Therefore, if the underlying house number has a suffix stored in the address point feature, the ETL tool will not utilize it (except to determine whether the address is in Edgewater Park), but will output only the basic house number component (reformatted into the special Edgewater Park format if necessary).

- Edgewater Park is a neighborhood in the Bronx that has unique house number formatting requirements. If the underlying address is an Edgewater Park address, the ETL tool will reformat it from the "real-world format" in which it is stored in the CSCL AddressPoint feature class into the "special Edgewater Park format" in which such house numbers are stored in other CSCL components and in LION. In the real-world format, the house number includes an Edgewater Park section suffix, which is a letter of the alphabet from 'A' to 'E'. In the special Edgewater Park format, the section letter is dropped (which is not relevant to the creation of SAF type G, N and X output records, since they contain no suffixes) and a corresponding multiple of 10,000 is added to the house number. For example, if the section letter is 'A', 10,000 (= 1 X 10,000) is added; if it is 'B', 20,000 (= 2 X 10,000) is added, etc. The ETL tool will process the address point as follows:

    a. If the SND_FEATURE_TYPE = 'E' in the principal StreetName entry for the B7SC_ACTUAL attribute of the address point, the address is in Edgewater Park. In that case, the HOUSE_NUMBER_SUFFIX attribute of the address point should contain an Edgewater Park section letter, either by itself or following some other suffix value and a dash (for example, '½ - B').

        - If it does not contain a section letter, the ETL tool will issue a warning including an appropriate message (e.g., "Underlying address is in Edgewater Park but has no letter suffix; field has been left blank"), and will populate both output fields SAFA8 and SAFA10 with blanks.

        - If it does contain a section letter, the ETL tool will add the appropriate multiple of 10,000 to the house number.

    b. If the SND_FEATURE_TYPE = 'F', the street is partially inside Edgewater Park and partially outside it. If

HOUSE_NUMBER_SUFFIX does not contain a section letter, the address will be assumed to lie outside of Edgewater Park; in that case, the tool will not issue a warning and it will not perform the special Edgewater Park reformatting. If it does contain a section letter, either by itself or following some other suffix value and a dash, the ETL tool will assume the address is within Edgewater Park; in that case, it will add the corresponding multiple of 10,000 to the house number.

6. SAFA13 (LGC2), SAFA14 (LGC3) and SAFA15 (LGC4): All NAPs have only a single LGC value. DCP has stated its intention to continue adhering to that rule. The ETL tool will populate these fields with blanks. In the future, if any NAP should ever be assigned more than one LGC value, the ETL tool will have to be modified accordingly.

7. SAFA17 (Segment Type Code):

   a. If the CommonPlace feature pertains to a Centerline segment:

      1. For the roadbed output file, and for the generic output file when the SEGMENTID of the CommonPlace feature identifies a non-roadbed segment or proto-segment, SAFA17 <u>will be populated from the SEGMENT_TYPE attribute of the Centerline segment.</u>

      2. For the generic output file when the SEGMENTID of the CommonPlace feature identifies a roadbed segment, SAFA17 will be populated from the SEGMENT_TYPE attribute of the corresponding generic Centerline segment.

   b. If the CommonPlace feature pertains to a Shoreline segment, SAFA17 will be populated with the value ‘U’.

8. SAFA18 (Segment ID):

   a. For the roadbed output file, and for the generic output file when the SEGMENTID attribute of the CommonPlace feature identifies a non-roadbed segment (including a Shoreline segment), SAFA18 will be populated with the SEGMENTID attribute of the CommonPlace feature.

   b. For the generic output file when the SEGMENTID of the CommonPlace feature identifies a roadbed segment, SAFA18 will be populated with the corresponding generic SEGMENTID, obtained from the RPL.

7.5. Type I Records

One output SAF Type I record is written into the SAFI output file for each entry in the CSCL NamedIntersection table. Table 32 contains a layout of the SAF Type I output records. The last column in the table indicates the CSCL source of data for populating each output field.

Table 32 – Layout and Populating Rules for SAF Record Type I

FIC	Field	Len	Frm	To	Format	CSCL Source of Data
SAFI1	Primary Intersection Name	32	1	32	LJSF	See Note 1
SAFI2	Node ID	7	33	39	RJZF	NamedIntersection field NODEID
	Filler	13	40	52		N/A
SAFI3	Multiple Field	1	53	53	M or blank	NamedIntersection field MULTIPLEFIELD – see Note 2
SAFI4	B7SC of Intersection Name	8	54	61		NamedIntersection field B7SC
SAFI5	B5SC of One Cross Street	6	62	67		NamedIntersection field B5SC_CROSS1
SAFI6	B5SC of Other Cross Street	6	68	73		NamedIntersection field B5SC_CROSS2
	Filler	12	74	85		N/A
SAFI7	SAF Record Type Code	1	86	86		'I'
	Filler	12	87	98		N/A

NOTES TO Table 32:

1. SAFI1 (Primary Intersection Name): This field will be populated with the intersection name in Geosupport Sort Format, It will be obtained from the LOOKUP_KEY field of the pertinent StreetName table entry via a composite key consisting of the B7SC of the Intersection Name and the PRINCIPAL_FLAG = 'Y'. The contents of LOOKUP_KEY in that entry will be converted to Sort Format as described in Note 1 to Table 20.

2. SAFI3 (Multiple Field): If the NamedIntersection field MULTIPLEFIELD contains a 'Y', the ETL tool will populate SAFI3 with the value 'M'. Otherwise, it will populate it with a blank.

7.6. Type O Records

SAF Type O records are extracted from the entries in the AltSegmentData table for which SAFTYPE contains an 'O'. The output records are written into the generic SAFV file, the roadbed SAFV file or both, depending on the SEGMENT_TYPE of the appurtenant Centerline segment, in accordance with the rules in Table 26.

Table 33 contains a list of the fields in the SAF Type O output records. The CSCL source for populating each field is indicated in the last column in Table 33 and in the note below the table.

Table 33 – Layout and Populating Rules for SAF Record Type O

FIC	Field	Len	Frm	To	Format	CSCL Source of Data
SAFV1	Street Name	32	1	32	LJSF	Obtained from StreetName table – see Note 1
SAFV2	Borough Code	1	33	33	1, 2, 3, 4 or 5	AltSegmentData field BOROUGH
SAFV3	Face Code of Actual Segment	4	34	37	RJZF	Obtained from bytes 2 through 5 of AltSegmentData LIONKEY field
SAFV4	Sequence Number of Actual Segment	5	38	42	RJZF	Obtained from bytes 6 through 10 of AltSegmentData LIONKEY field
SAFV5	SOS Indicator for Actual Segment	1	43	43	L, R or blank	AltSegmentData field SOS_INDICATOR – see Note 2
SAFV6	B5SC	6	44	49		AltSegmentData field B5SC
SAFV7	Low HN Basic	7	50	56	RJSF	Obtained from AltSegmentData field L_LOW_HN or R_LOW_HN depending on value of SOS_INDICATOR
SAFV8	Low HN Suffix	9	57	65	RJSF	AltSegmentData field LOW_HN_SUFFIX
	Filler	2	66	67		N/A
SAFV9	High HN Basic	7	68	74	RJSF	Obtained from AltSegmentData field L_HIGH_HN or R_HIGH_HN depending on value of SOS_INDICATOR
SAFV10	High HN Suffix	9	75	83	RJSF	AltSegmentData field HIGH_HN_SUFFIX
	Filler	2	84	85		N/A
SAFV11	SAF Record Type Code	1	86	86		AltSegmentData field SAFTYPE
	Filler	2	87	88		N/A
SAFV12	LGC1	2	89	90	RJZF	AltSegmentData field LGC1

FIC	Field	Len	Frm	To	Format	CSCL Source of Data
SAFV13	LGC2	2	91	92	RJZF, blanks if none	AltSegmentData field LGC2
SAFV14	LGC3	2	93	94	RJZF, blanks if none	AltSegmentData field LGC3
SAFV15	LGC4	2	95	96	RJZF, blanks if none	AltSegmentData field LGC4
SAFV16	Pointer to BOE Preferred LGC	1	97	97		AltSegmentData field BOE_PREFERRED_LGC _ FLAG
SAFV17	Segment Type Code	1	98	98		SEGMENT_TYPE attribute of appurtenant (actual) Centerline segment
SAFV18	Segment ID of Actual Segment	7	99	105	RJZF	AltSegmentData field SEGMENTID
SAFV19	X Coordinate	7	106	112		Populate with blanks. (Field is used only for type V records.)
SAFV20	Y Coordinate	7	113	119		Populate with blanks. (Field is used only for type V records.)

NOTES To Table 33:

1. SAFV1 (Street Name): this field will be populated with the preferred street name in Geosupport Sort Format. It will be obtained from the LOOKUP_KEY field in the pertinent StreetName table entry as follows. The AltSegmentData fields B5SC and LGC1 will be concatenated to form the Preferred B7SC. (**ASSUMPTION**: the AltSegmentData field LGC1 always contains the Preferred LGC.) StreetName will then be accessed with a composite key to retrieve the entry that contains the Preferred B7SC and in which the StreetName field PRINCIPAL_FLAG = 'Y'. The contents of LOOKUP_KEY in that entry will be converted to Sort Format as described in Note 1 to Table 20.

2. SAFV5 (SOS Indicator): If AltSegmentData field SOS_INDICATOR = '1', SAFV5 will be populated with 'L'. If SOS_INDICATOR = '2', SAFV5 will be populated with 'R'. Otherwise, SAFV5 will be populated with a blank.

7.7. Type S Records

SAF Type S records are extracted from the AddressPoint feature class. Output records are written for each address point for which the SPECIAL_CONDITION attribute contains the value 'S'. An output record is written into the generic SAFS file, the roadbed SAFS file

or both, depending on the SEGMENT_TYPE of the appurtenant Centerline segment, in accordance with the rules in Table 26. (Note: AltSegmentData does not have a field for SEGMENT_TYPE. Instead, proto-segments implicitly have the same SEGMENT_TYPE as their corresponding geometry-represented segments. Therefore, if a type S record pertains to a proto-segment, the SEGMENT_TYPE value of the corresponding Centerline segment will be used for purposes of applying the Table 26 rules, as well as to populate output field SAFS15.)

Table 34 contains a list of the fields in the SAF Type S output records. The CSCL source of data for populating each field is indicated in the last column in Table 34 and in the notes below the table.

Table 34 – Layout for SAF Record Type S

FIC	Field	Len	Frm	To	Format	CSCL Source of Data
SAFS1	Street Name	32	1	32	LJSF	Obtained from StreetName table – see Note 1
SAFS2	Borough Code	1	33	33	1, 2, 3, 4 or 5	AddressPoint attribute BOROUGHCODE
SAFS3	Face Code	4	34	37	RJZF	Obtained from StreetName table - see Note 1
SAFS4	Sequence Number	5	38	42	RJZF	See Note 2
SAFS5	SOS Indicator	1	43	43		AddressPoint attribute SOSINDICATOR– see Note 4
SAFS6	B5SC	6	44	49		Populate with first 6 bytes of AddressPoint attribute B7SC_ACTUAL
SAFS7	HN Basic	7	50	56	RJSF	AddressPoint attribute HOUSE_NUMBER – see Note 5
	Filler	1	57	57		
SAFS8	House Number Suffix	8	58	65	LJSF, blanks if null	See Note 6 AddressPoint attribute HOUSE_NUMBER_SUFFIX
	Filler	9	66	74		
SAF85	High Alpha House number suffix	1	75	75		AddressPoint attribute HOUSE_NUMBER_RANGE_SUFFIX
	Filler	10	76	85		
SAFS9	SAF Record Type Code	1	86	86		Populate with AddressPoint attribute SPECIAL_CONDITION
	Filler	2	87	88		
SAFS10	LGC1	2	89	90	RJZF	Obtain from Segment_LGC table – see Note 3

FIC	Field	Len	Frm	To	Format	CSCL Source of Data
SAFS11	LGC2	2	91	92	RJZF, blanks if none	Obtain from Segment_LGC table – see Note 3
SAFS12	LGC3	2	93	94	RJZF, blanks if none	Obtain from Segment_LGC table – see Note 3
SAFS13	LGC4	2	95	96	RJZF, blanks if none	Obtain from Segment_LGC table – see Note 3
SAFS14	Pointer to BOE Preferred LGC	1	97	97		Obtain from Segment_LGC table – see Note 3
SAFS15	Segment Type Code	1	98	98		See Note 2
SAFS16	Segment ID	7	99	105	RJZF	See Note 2

NOTES TO Table 34:

1. SAFS1 (Street Name) and SAFS3 (Face Code): These fields will be populated, respectively, with the preferred street name in Geosupport Sort Format and the Face Code corresponding to the preferred street name. Both items will be obtained from the pertinent StreetName table entry, from the fields LOOKUP_KEY and FACECODE, respectively. The table will be accessed using a composite key consisting of the AddressPoint attribute B7SC_ACTUAL combined with setting the StreetName field PRINCIPAL_FLAG to 'Y'. SAFS1 will be populated by converting the contents of LOOKUP_KEY in that StreetName entry to Sort Format as described in Note 1 to Table 20.

   When the B7SC_actuak doesn't have a face code, the lion key should be generated from the face code and sequence number of the segment that the address point is associated to.

   If StreetName contains no entry satisfying these search criteria, ~~or the entry does exist but it does not contain a Face Code value,~~ the ETL tool will not write out an output record and will issue an error message.

   (Note: the B7SC_ACTUAL field in a type S address point contains the preferred B7SC value of the segment. Since the preferred B7SC of every roadbed segment is the one assigned to the generic name of the street (and DCP has stated its intention to continue adhering to that rule), the use of the B7SC_ACTUAL value to obtain the street name and Face Code as described above is valid even when the appurtenant segment is a roadbed segment and the record is being written into the generic output file.)

2. SAFS4 (Sequence Number), SAFS15 (Segment Type Code) and SAFS16 (Segment ID): These fields will be populated as follows:

a. If the Centerline segment identified by the type S address point's SEGMENTID is a non-roadbed segment (SEGMENT_TYPE not = 'R') regardless of the output file, or if it is a roadbed segment and the output record is being written into the roadbed output file:

- SAFS15 and SAFS16 will be populated with the SEGMENT_TYPE and SEGMENTID of the Centerline segment, respectively.

- If the BOROUGHCODE fields of the type S address point and the Centerline segment are equal, the address point pertains to the Centerline segment. SAFS4 will be populated from the SEGMENT_SEQNUM attribute of the Centerline segment.

- If the BOROUGHCODE fields of the type S address point and the Centerline segment are not equal, the address point pertains to a borough boundary proto-segment having the given SEGMENTID. SAFS4 will be populated from the ALT_SEGMENT_SEQNUM field of the proto-segment's AltSegmentData entry.

b. For the generic output file only, if the Centerline segment identified by the type S address point's SEGMENTID is a roadbed segment:

- The SEGMENT_TYPE and SEGMENTID of the corresponding generic segment will be obtained from the RPL and will be populated into SAFS15 and SAFS16, respectively.

- If the BOROUGHCODE field of the type S address point and that of the Centerline segment identified by the generic SEGMENTID are equal, SAFS4 will be populated from the SEGMENT_SEQNUM attribute of the generic Centerline segment.

- If the BOROUGHCODE field of the type S address point and that of the generic Centerline segment are not equal, there is a borough boundary proto-segment having the given generic SEGMENTID. SAFS4 will be populated from the ALT_SEGMENT_SEQNUM field of that proto-segment's AltSegmentData entry.

3. SAFS10 (LGC1), SAFS11 (LGC2), SAFS12 (LGC3), SAFS13 (LGC4) and SAFS14 (Pointer to BOE Preferred LGC): These fields will be populated as follows:

a. If the BOROUGHCODE value in the type S address point is identical to the borough code in the first byte of the B5SC field in any Segment_LGC entry for the given SEGMENTID, the type S address pertains to a geometry-represented Centerline segment. In that case, the LGC output fields will be populated as follows:

- SAFS10 will be populated with the LGC value contained in that Segment_LGC entry for the given SEGMENTID in which the PREFERRED_LGC_FLAG contains a 'Y'.

- If there are any other Segment_LGC entries for the given SEGMENTID, then as many as necessary of the output fields SAFS11, SAFS12 and SAFS13 will be populated in turn from the LGC values in those entries. For example, if there are two other entries, then SAFS11 and SAFS12 will be populated with the respective values, and SAFS13 will be populated with blanks.

- SAFS14 will be set to the integer value corresponding to the output LGC field that was populated from that Segment_LGC entry in which BOE_PREFERRED_LGC_FLAG contained a 'Y'. For example, if SAF12 (LGC3) was populated from that entry, then SAFS14 will be populated with the value '3' (corresponding to LGC3).

b. If the BOROUGHCODE value in the type S address point differs from the borough code in the first byte of the B5SC field in any Segment_LGC entry for the given SEGMENTID, the type S address pertains to a borough-boundary proto-segment. In that case, the ETL tool will perform the following processing:

- First, the ETL tool will verify that there is a such a proto-segment entry in AltSegmentData. The entry should satisfy the following conditions:

    - It contains the same SEGMENTID as the type S address point

    - ALT_SEGDATA_TYPE = 'B' (signifying a borough boundary proto-segment)

    - The BOROUGH field in the entry equals the BOROUGHCODE value in the type S address point

    - The B7SC_ACTUAL of the type S address point equals the concatenation of the fields B5SC and LGC1 of the proto-segment. (Note: this validates that the proto-segment belongs to the same street as the type S address and that the B7SC_ACTUAL of the address point is the preferred B7SC of the proto-segment.)

- If there is no proto-segment satisfying the above conditions, the ETL tool will report the case as an error. The message will indicate which of the above conditions has been found to be violated. An SAF record will not be written out.

- If there is such a proto-segment, the output fields SAFS10 (LGC1) through SAFS13 (LGC4) will be populated from the fields named LGC1 through LGC4 in the proto-segment's AltSegmentData entry. Null values will be replaced with blanks. The field SAFS14 (the Pointer to BOE Preferred LGC) will be populated by copying the proto-segment's BOE_PREFERRED_LGC_FLAG.

4. SAFS5 (SOS Indicator): If the AddressPoint field SOS_INDICATOR = '1', SAFS5 will be populated with 'L'. If SOS_INDICATOR = '2', SAFS5 will be populated with 'R'. Otherwise, SAFS5 will be populated with a blank.

5. SAFS7 (Basic House Number): Will be populated as follows:

   - If the HYPHEN_TYPE attribute of the address point contains the value 'R', the address point's HOUSE_NUMBER attribute contains a range of unhyphenated house numbers expressed in the form of the low house number of the range followed by a dash followed by the high house number of the range. (Note: there might or might not be spaces between the dash and the digits to the left and/or right of the dash. For example, the data in HOUSE_NUMBER might appear as '22-28' or as '22 – 28'.) The ETL tool will populate SAFS7 with the low-address portion of the range only; the dash and the digits to the right of the dash will not be used. That value will then be modified as discussed below if the address point is in Edgewater Park.

   - If the HYPHEN_TYPE attribute of the address point contains any value other than 'R', the ETL tool will populate SAFS7 with the value in the address point's HOUSE_NUMBER field without modification (except for the special Edgewater Park reformatting if required).

   - If the address point is in Edgewater Park, the house number will be reformatted by adding an appropriate multiple of 10,000 to the house number. The method for determining whether an address is in Edgewater Park, and if so, the quantity that must be added to the house number, are described in Note 5 to Table 31.

6. SAFS8 (House Number Suffix): Will be populated as follows:

   - If the address point is not in Edgewater Park, SAFS8 will be populated from the address point's HOUSE_NUMBER_SUFFIX attribute, or with blanks if that is null.

   - If the address point is in Edgewater Park, the house number suffix will be reformatted by dropping the Edgewater Park section letter, as described in Note 5 to Table 31. If there is a suffix value in addition to the section letter, it will be retained and populated into SAFS8 without the dash separating it from the section letter. If there is no such suffix value, SAFS8 will be populated with blanks.

7.8. Type V Records

SAF Type V records are extracted from the AddressPoint feature class. Output records are written for each address point for which the SPECIAL_CONDITION attribute contains the value 'V'. The output records are written into the generic SAFV file, the roadbed SAFV file or both, depending on the SEGMENT_TYPE of the appurtenant Centerline segment, in accordance with the rules in Table 26.

Table 35 contains a list of the fields in the SAF Type V output records. The CSCL source of data for populating each field is indicated in the last column in Table 35 and in the notes below the table.

Note that Type V records have the same layout and are written into the same SAF output files as Type O records, but their CSCL data sources and therefore the methods for populating their fields are different. The spatial coordinate fields, SAFV19 and SAFV20, are populated with values only in type V records.

Table 35 – Layout and Populating Rules for SAF Record Type V

FIC	Field	Len	Frm	To	Format	CSCL Source of Data
SAFV1	Vanity Street Name	32	1	32		Obtained from StreetName table – see Note 1
SAFV2	Borough Code	1	33	33	1, 2, 3, 4 or 5	AddressPoint attribute BOROUGHCODE
SAFV3	Face Code of Actual Segment	4	34	37	RJZF	See Note 2
SAFV4	Sequence Number of Actual Segment	5	38	42	RJZF	See Note 3
SAFV5	SOS Indicator for Actual Segment	1	43	43	L, R or blank	AddressPoint attribute SOSINDICATOR – see Note 4
SAFV6	Vanity B5SC	6	44	49		Populate with first 6 bytes of AddressPoint attribute B7SC_VANITY
SAFV7	Low HN Basic	7	50	56	RJSF	See Note 5
SAFV8	Low HN Suffix	9	57	65	RJSF	See Note 5
	Filler	2	66	67		N/A
SAFV9	High HN Basic	7	68	74	RJSF	See Note 5
SAFV10	High HN Suffix	9	75	83	RJSF	See Note 5
	Filler	2	84	85		N/A
SAFV11	SAF Record Type Code	1	86	86		Populate with AddressPoint attribute SPECIAL_CONDITION
	Filler	2	87	88		N/A
SAFV12	Vanity LGC1	2	89	90	RJZF	Populate with 7th and 8th bytes of AddressPoint attribute B7SC_VANITY
SAFV13	Vanity LGC2	2	91	92	RJZF, blanks if none	See Note 6

FIC	Field	Len	Frm	To	Format	CSCL Source of Data
SAFV14	Vanity LGC3	2	93	94	RJZF, blanks if none	See Note 6
SAFV15	Vanity LGC4	2	95	96	RJZF, blanks if none	See Note 6
SAFV16	Pointer to BOE Preferred LGC	1	97	97		See Note 6
SAFV17	Segment Type Code for Actual Segment	1	98	98		See Note 3
SAFV18	Segment ID of Actual Segment	7	99	105	RJZF	See Note 3
SAFV19	X Coordinate	7	106	112	Round to nearest foot RJZF	Capture from address point's spatial coordinates
SAFV20	Y Coordinate	7	113	119	Round to nearest foot RJZF	Capture from address point's spatial coordinates

NOTES TO Table 35:

1. SAFV1 (Vanity Street Name): This field will be populated with the preferred name of the vanity street in Geosupport Sort Format. It will be converted from the LOOKUP_KEY field of the pertinent StreetName table entry. The table will be accessed using a composite key consisting of the address point field B7SC_VANITY combined with setting the StreetName field PRINCIPAL_FLAG to 'Y'. The contents of LOOKUP_KEY in that entry will be converted to Sort Format as described in Note 1 to Table 20.

   Note: the B7SC_VANITY field in a type V address point contains the preferred B7SC value of the vanity street name. Since the preferred B7SC of every roadbed segment is the B7SC that is assigned to the generic name of the given street (and DCP has stated its intention to continue adhering to that rule), the use of the B7SC_VANITY value to obtain the vanity street name as described above is valid even when the vanity street is a divided street and the record is being written into the generic output file.

2. SAFV3 (Face Code of Actual Segment): Populated from the FACECODE field in the StreetName table entry for the principal name of the actual street. Obtained by accessing StreetName using a composite key consisting of the AddressPoint attribute B7SC_ACTUAL combined with setting the StreetName field PRINCIPAL_FLAG = 'Y'.

Note: the B7SC_ACTUAL field in a type V address point contains the preferred B7SC value of the actual segment. Since the preferred B7SC of every roadbed segment is the one assigned to the generic name of the street (and DCP has stated its intention to continue adhering to that rule), the use of the B7SC_ACTUAL value to obtain the Face Code as described above is valid even when the appurtenant segment is a roadbed segment and the record is being written into the generic output file.

3. SAFV4 (Sequence Number of Actual Segment), SAFV17 (Segment Type Code of Actual Segment) and SAFV18 (Segment ID of Actual Segment): These fields must be populated with the values from the "actual" segment, that is, the segment on which the vanity address is physically located.

   ASSUMPTION: The ETL tool has been designed under the assumption that no vanity addresses are located on proto-segments (which is tantamount to assuming that there are no vanity addresses located on borough boundaries). That is in fact the case as of the writing of this document. **In the future, if there should ever be any cases in which the "actual" segment of a vanity address is a proto-segment, the ETL tool will have to be modified accordingly.**

   When creating the output record for the generic output file, if the vanity address is located on a divided street, these fields must be populated with the values from the generic segment that corresponds to the roadbed segment identified in the address point.

   The ETL tool will obtain the values for populating SAFV4, SAFV17 and SAFV18 as follows:

   a. If the Centerline segment identified by the type V address point's SEGMENTID is a non-roadbed segment (SEGMENT_TYPE not = 'R') regardless of the output file, or if it is a roadbed segment and the output record is being written into the roadbed output file:

      SAFV4, SAFV17 and SAFV18 will be populated with the Sequence Number, Segment Type Code and Segment ID values from the Centerline segment's SEGMENT_SEQNUM, SEGMENT_TYPE and SEGMENTID attributes, respectively.

   b. If the Centerline segment identified by the type V address point's SEGMENTID is a roadbed segment, and the output record is being written into the generic output file:

      SAFV4, SAFV17 and SAFV18 will be populated with the Sequence Number, Segment Type Code and Segment ID values from the SEGMENT_SEQNUM, SEGMENT_TYPE and SEGMENTID attributes, respectively, of the generic segment that corresponds to the Centerline segment. The generic segment will be identified by searching the RPL in the usual manner.

4. SAFV5 (SOS Indicator): If AddressPoint field SOS_INDICATOR = '1', SAFV5 will be populated with 'L'. If SOS_INDICATOR = '2', SAFV5 will be populated with 'R'. Otherwise, SAFV5 will be populated with a blank.

5. SAFV7 (Low HN Basic), SAFV8 (Low HN Suffix), SAFV9 (High HN Basic) and SAFV10 (High HN Suffix): An address point feature might contain a single house number or it might contain low and high house numbers that define an address range. The house number information is stored in the address point's attribute data as follows:

   - The attribute HYPHEN_TYPE indicates whether the address point house number data consist of a single house number or a range defined by low and high house numbers, as follows:

     - HYPHEN_TYPE = 'N' indicates a single unhyphenated house number.

     - HYPHEN_TYPE = 'R' indicates a range of unhyphenated house numbers.

     - HYPHEN_TYPE = 'Q' indicates a single Queens-style hyphenated house number.

     - HYPHEN_TYPE = 'U' indicates a single hyphenated house number in which the digits following the hyphen are a building unit identifier. (Note: As of this writing, there are no address points in the CSCL database for which HYPHEN_TYPE = 'U'.)

     - HYPHEN_TYPE = 'X' indicate a range of hyphenated house numbers. Other HYPHEN_TYPE values indicate a single house number.

   - A single house number, whether hyphenated (HYPHEN_TYPE = Q or U) or unhyphenated (HYPHEN_TYPE = N), is stored in the attribute HOUSE_NUMBER (and in the attribute HOUSE_NUMBER_SUFFIX, if it has a suffix).

   - The manner in which a range is stored depends on whether it is a range of unhyphenated or hyphenated house numbers:

     - If it is a range of unhyphenated house numbers (HYPHEN_TYPE = R), the low house number and high house number are stored together in a single character string in the HOUSE_NUMBER field with a dash separating them. Note the following:

       i. One or more spaces might or might not separate the dash from the low and/or high house numbers. For example, the range might appear as 342-344 or it might appear as 342 - 344.

       ii. In the HYPHEN_TYPE = 'R' case, if there is a value in the HOUSE_NUMBER_SUFFIX field, it might apply to the low house

number or to the high house number. Because of that ambiguity, the suffix will not be included in the output record.

- If it is a range of Queens-style hyphenated house numbers (HYPHEN_TYPE = X), the low house number is stored in the HOUSE_NUMBER field and the high house number is stored in the HOUSE_NUMBER_RANGE field. If the low house number of the range has a suffix, it is stored in the HOUSE_NUMBER_SUFFIX field. If the high house number has a suffix, it is stored in the HOUSE_NUMBER_RANGE_SUFFIX field.

The ETL tool will populate the output fields SAFV7, SAFV8, SAFV9 and SAFV10 as follows:

- If HYPHEN_TYPE = N, Q or U, the tool will populate SAFV7 from HOUSE_NUMBER, and will populate SAFV8 from HOUSE_NUMBER_SUFFIX if the latter contains a value, else with blanks. The tool will populate SAFV9 and SAFV10 with the same values that are populated into SAFV7 and SAFV8, respectively.

- If HYPHEN_TYPE = R, the tool will populate SAFV7 with the digits in HOUSE_NUMBER that are to the left of the dash, and will populate SAFV9 with the digits to the right of the dash. It will populate SAFV8 and SAFV10 with blanks.

- If HYPHEN_TYPE = X, the tool will populate SAFV7 from HOUSE_NUMBER, SAFV8 from HOUSE_NUMBER_SUFFIX if the latter contains a value, else with blanks, SAFV9 from HOUSE_NUMBER_RANGE, and SAFV10 from HOUSE_NUMBER_RANGE_SUFFIX if the latter contains a value, else with blanks.

ASSUMPTION: The ETL tool has been designed under the assumption that no vanity addresses are located in Edgewater Park. (In fact, as of the writing of this document, no vanity addresses exist in the CSCL database in the entire borough of the Bronx). **In the future, if there should ever be any such cases, the ETL tool will have to be modified to reformat the basic house number and house number suffix fields in those cases as described in Notes 5 and 6 to Table 34.**

6. SAFV13 (LGC2), SAFV14 (LGC3), SAFV15 (LGC4), SAFV16 (Pointer to BOE Preferred LGC): Any additional LGC values for vanity addresses beyond the one that is a component of B7SC_VANITY are stored in the AddressPointLGCs table, which can be accessed from the AddressPoint data by ADDRESSPOINTID via the relationship class AddressPointsHaveLGCs. The ETL Tool will insert an integer from 1 to 4 into SAFV16 to identify the LGC output field that contains the LGC value obtained from the AddressPointLGCs entry in which BOE_LGC = ‘Y’. For example, if LGC2 (stored in SAFV13) contains the LGC value that was obtained from the aforementioned AddressPointLGCs entry, then SAFV16 will be set to ‘2’. If instead it is LGC3 (in SAFV14) that contains that LGC value, then SAFV16 will be set to ‘3’. However, if none of the AddressPointLGCs entries have BOE_LGC = ‘Y, that

signifies that the LGC obtained from the AddressPoint B7SC_VANITY attribute and loaded into LGC1 (in SAFV12) is the BOE Preferred LGC; in that case, set SAFV16 to ‘1’.

8. SEDAT AND SPECIAL SEDAT

The Split Election District Address Table (SEDAT) and the Special SEDAT are extracted as two distinct output files that have identical record layouts. They are extracted from the CSCL SEDAT and SpecialSEDAT tables, respectively. One output record is written for each entry in those tables.

Entries in the CSCL SEDAT table are associated with Centerline segments. It is possible for multiple SEDAT entries to be associated with one Centerline segment. There is a relationship class called CenterlinesToSEDAT that links SEDAT table entries and Centerline segments by SEGMENTID.

Entries in the CSCL SPECIALSEDAT table are associated with SAF records, which are stored in the CSCL database as SAF entries in the AltSegmentData table or as point features in the AddressPoint or CommonPlace feature classes, depending on the SAF record type.

8.1. SEDAT

The ETL tool writes one record into the SEDAT output file for each entry in the CSCL SEDAT table. Table 36 contains a layout of the SEDAT output file. The last column in the table indicates the CSCL source of data for populating each field.

Table 36 – SEDAT Output File

FIC	Field	Len	Frm	To	CSCL Source of data
SEDAT1	LION Key	10	1	10	SEDAT table LIONKEY field
SEDAT2	Parity of Address Range	1	11	11	SEDAT table PARITY field
SEDAT3	Street Name	32	12	43	See Note 1
SEDAT4	Side of Street	1	44	44	See Note 2
SEDAT5	Low House Number	7	45	51	SEDAT table LOWADDRESS field
SEDAT6	Low House Number Suffix	8	52	59	SEDAT table LOW_ADDR_SUFFIX field
SEDAT7	High House Number	7	60	66	SEDAT table HIGHADDRESS field
SEDAT8	High House Number Suffix	8	67	74	SEDAT table HIGH_ADDR_SUFFIX field
SEDAT9	Election District	3	75	77	SEDAT table ELECTION_DISTRICT field
SEDAT10	Assembly District	2	78	79	SEDAT table ASSEMBLY_DISTRICT field
SEDAT11	B7SC	8	80	87	See Note 3

NOTES TO Table 36:

1. SEDAT3 (Street Name): This field will be populated after output field SEDAT11 has been populated (see Note 2 below). The Preferred B7SC value populated into SEDAT11 will be used in a composite key with PRINCIPAL_FLAG = 'Y' in the usual manner to access the entry in the StreetName table for the principal name of the Preferred LGC. SEDAT3 will then be populated with the principal preferred street name in Geosupport-normalized format from the LOOKUP_KEY field in that entry.

   Note that the 42-byte STREET field in the CSCL SEDAT table is not being used to populate the output record. **RECOMMENDATION**: The STREET field in the CSCL SEDAT table should be dropped from the CSCL data model. Beyond the fact that this field has an erroneous length of 42 bytes, its presence in this table is redundant and violates the general CSCL approach that street names will only be stored in one place in the CSCL database, in the StreetName table.

2. SEDAT4 (Side of Street). Populate from the CSCL SEDAT Table field SIDE as follows:

   - If SIDE = '1', populate SEDAT4 with the value 'L'
   - If SIDE = '2', populate SEDAT4 with 'R'

3. SEDAT11 (B7SC): This field will be populated with the Preferred B7SC of the given segment. It will be obtained in the usual manner by accessing the entry for the Preferred LGC in the Segment_LGC table via the composite key consisting of the SEGMENTID from the CSCL SEDAT entry and the PREFERRED_LGC_FLAG = 'Y'. The concatenation of the two fields B5SC and LGC from the given table entry is the desired Preferred LGC value.

   In performing the SEDAT11 processing, the ETL tool must handle the case of borough boundary coincident segments. In that case, street codes from two boroughs relate to the same SEGMENTID. The ETL tool will recognize this case by the fact that there will be two entries in Segment_LGC for the same SEGMENTID that have the PREFERRED_LGC_FLAG set to 'Y'. The one that the ETL tool will use is the one whose borough code (the first byte of the B5SC) is the same as that of the given SEDAT entry (the first byte of the LIONKEY field).

   Note that the 5-byte STREETCODE field in the CSCL SEDAT table is not being used to populate the output record, since the Segment_LGC table has to be accessed anyway to obtain the Preferred LGC with which to form the Preferred B7SC. **RECOMMENDATION**: The STREETCODE field in the CSCL SEDAT table appears superfluous and redundant, and should be dropped from the CSCL data model.

8.2. Special SEDAT

Table 37 contains a layout of the Special SEDAT output file and the CSCL source of data for each field. The last column in the table and the notes following the table indicate the CSCL source of data for populating each field.

Table 37 – Special SEDAT Output File

FIC	Field	Len	Frm	To	CSCL Source of data
SEDAT1	LION Key	10	1	10	SPECIALSEDAT table LIONKEY field
SEDAT2	Parity of Address Range	1	11	11	SPECIALSEDAT table PARITY field
SEDAT3	Street Name	32	12	43	See Note 1
SEDAT4	Side of Street	1	44	44	See Note 2
SEDAT5	Low House Number	7	45	51	SPECIALSEDAT table LOWADDRESS field
SEDAT6	Low House Number Suffix	8	52	59	SPECIALSEDAT table LOW_ADDR_SUFFIX field
SEDAT7	High House Number	7	60	66	SPECIALSEDAT table HIGHADDRESS field
SEDAT8	High House Number Suffix	8	67	74	SPECIALSEDAT table HIGH_ADDR_SUFFIX field
SEDAT9	Election District	3	75	77	SPECIALSEDAT table ELECTION_DISTRICT field
SEDAT10	Assembly District	2	78	79	SPECIALSEDAT table ASSEMBLY_DISTRICT field
SEDAT11	B7SC	8	80	87	SPECIALSEDAT table B7SC field

NOTES TO Table 37:

1. SEDAT3 (Street Name): This field will be populated with the principal street name of the B7SC value that is stored in the SpecialSEDAT table entry. The ETL tool will attempt to obtain the name from the StreetName table field LOOKUP_KEY by accessing StreetName via a composite key consisting of the B7SC value combined with the StreetName field PRINCIPAL_FLAG = 'Y'. If that does not succeed, the ETL tool will attempt to obtain the name from the LOOKUP_KEY field in the principal entry for the B7SC value in the FeatureName table. If that fails as well, the ETL tool will treat the case as an error; it will issue an error message and will not write an output record.

2. SEDAT4 (Side of Street). Populate from the CSCL SpecialSEDAT Table field SIDE as follows:

   - If SIDE = '1', populate SEDAT4 with the value 'L'
   - If SIDE = '2', populate SEDAT4 with 'R'

9. ROADBED POINTER LIST

Table 38 contains a layout of the Roadbed Pointer List (RPL) output file. The ETL tool writes one output record for every entry in the CSCL RoadbedPointerList table. Each record represents one roadbed segment associated with a given generic segment.

A single citywide RPL output file is produced. The output records are ordered as follows:

- The records containing each generic SEGMENTID, that is, each distinct value in the output field RPL1, will occur consecutively.
- The set of records for a given generic SEGMENTID will be ordered based on the value of the CSCL RoadbedPointerList field ROADBED_POSITION_CODE and on a spatial determination of the positions of the inner roadbed segments, as follows:
    - The field ROADBED_POSITION_CODE in each CSCL RoadbedPointerList entry contains one of the values 'L', 'R' or 'I', indicating that the given entry represents the outermost road segment on the left side of the generic segment, the right outermost roadbed segment, or an inner roadbed segment, respectively. There will always be exactly one 'L' entry and one 'R' entry. There can be any number of 'I' entries, including none.
    - To order the output records properly, it is necessary to determine which side of the generic segment each inner roadbed segment is on, and what their relative positions are. The ETL tool will use spatial methods to do so. The spatial methodology will make use of the imaginary line perpendicular to the generic segment through its midpoint. The ETL tool will determine the intersections of that line with the imaginary infinite extensions of the inner roadbed segments. The order in which those intersections occur along the perpendicular line corresponds to the relative positions of the inner roadbed segments. The possibility that any of the roadbed segments can be a polyline with intermediate vertices must be taken into consideration.
    - The first record that the ETL tool will write out for the given generic segment will be the one generated from the entry for the right outermost roadbed segment, that is, the one whose ROADBED_POSITION_CODE field contains the value 'R'.
    - The next record(s) that the ETL tool will write out will be the right inner roadbed segments, ordered in sequence from the one closest to the right outermost roadbed segment to the one closest to the generic segment.
    - The next record that the ETL tool will write out will be the one generated from the entry for the left outermost roadbed segment.
    - Finally, the ETL tool will write out the records for the left inner roadbed segments, ordered in sequence from the one closest to the left outermost roadbed segment to the one closest to the generic segment.

Table 38 – Roadbed Pointer List Output File

FIC	Field	Len	Frm	To	Format	CSCL Source Field
RPL1	Generic SEGMENTID	7	1	7	RJZF	RoadbedPointerList GENERIC_SEGMENTID field
RPL2	Segment Type of Generic Segment	1	8	8		See Note 1
RPL3	Roadbed SEGMENTID	7	9	15	RJZF	RoadbedPointerList ROADBED_SEGMENTID field
	Filler	1	16	16		Populate with blank
RPL4	Roadbed Position Code	1	17	17		RoadbedPointerList ROADBED_POSITION_CODE field
	Filler	1	18	18		Populate with blank
RPL5	Node Correspondence Indicator	1	19	19		RoadbedPointerList NODE_CORRESPONDENCE_IND field
	Filler	3	20	22		Populate with blanks
RPL6	From Node Level Code of Coincident Roadbed Segment (if any)	1	23	23		RoadbedPointerList FROM_NODE_LEVEL_COINCIDENT_RB field
	Filler	3	24	26		Populate with blanks
RPL7	To Node Level Code of Coincident Roadbed Segment (if any)	1	27	27		RoadbedPointerList TO_NODE_LEVEL_COINCIDENT_RB field
	Filler	1	28	28		Populate with blank
RPL8	From NODEID of Roadbed Segment	7	29	35	RJZF	See Note 2
	Filler	1	36	36		Populate with blank
RPL9	From NODEID of Generic Segment	7	37	43	RJZF	See Note 2
	Filler	1	44	44		Populate with blank
RPL10	To NODEID of Roadbed Segment	7	45	51	RJZF	See Note 2
	Filler	1	52	52		Populate with blank
RPL11	To NODEID of Generic Segment	7	53	59	RJZF	See Note 2

NOTES TO Table 38:

1. RPL2 (Segment Type of Generic Segment): This field will be populated from the SEGMENT_TYPE attribute field of the appurtenant Centerline segment, accessed by the GENERIC_SEGMENTID value in the given RoadbedPointerList table entry. The ETL tool will **ASSUME**, without validating it, that this is the SEGMENTID of a valid generic segment in Centerline, and therefore that its SEGMENT_TYPE contains one of the values ‘B’ or ‘G’.

2. RPL8 (From NODEID of Roadbed Segment), RPL9 (From NODEID of Generic Segment), RPL10 (To NODEID of Roadbed Segment), RPL11 (To NODEID of Generic Segment): These fields will be populated as follows. The respective SEGMENTID will be used to access the given segment (the generic or roadbed segment). For each endpoint of the segment, the ETL tool will perform a spatial query against the Node feature class to obtain that endpoint’s NODEID, if any, using standard ESRI functionality. The tool will populate the respective NODEID output field accordingly. If there is no node at a segment endpoint, the ETL tool will populate that output field with blanks, and will report the case as an error.

10. DISTRICT BOUNDARY GEODATABASE

The ETL tool creates an ESRI-formatted file geodatabase (as opposed to a personal geodatabase) of district boundary polygons. It will contain approximately two dozen polygon feature classes for the various types of district boundaries.

Appendix D contains a schema for the district boundary geodatabase provided by DCP, identified as Version 4 and dated January 29, 2009.

For many of the district types, the polygon geometry is extracted directly from the CSCL geodatabase, which contains corresponding polygon feature classes. However, for some of the district types, the ETL tool creates two sets of the district polygons, one set with the polygon geometry unmodified from how they are stored in the CSCL database, and one set with the polygons clipped to the shoreline.

Minor Census Economic Areas (MCEAs) are a required type of output district not modeled by polygon geometry in the CSCL database. Instead, there is an MCEA attribute in the CensusTract2000 feature class. The ETL tool generates MCEA polygons by dissolving CensusTract2000 polygons based on the MCEA attribute and clipping the results to the shoreline. (It is not required to produce a water-blocks-included version of MCEA.)

The attributes required for most of the polygon feature classes are listed in Appendix D. Their field names are self-explanatory, and they have corresponding fields in the CSCL database that will serve as the data sources. For some of the polygon feature classes, no attribute information is provided in DCP's documentation in Appendix D. Those are Urban Renewal Area, Business Improvement District, Empire Zone, Historic District, Zip Code and the 2010 Census districts. It is **assumed** that the only attribute fields they require (in addition to the mandatory ESRI OBJECTID and SHAPE fields) are the respective district identifiers, as defined in the CSCL data model.

The ETL tool generates a temporary set of shoreline polygons that it uses for clipping. It generates them by dissolving Atomic Polygons (APs) based on the WATER_FLAG attribute. APs for which WATER_FLAG = '2' (non-water) or '3' (pier) are dissolved to generate the shoreline polygons for use in clipping.

Table 39 lists the polygon feature classes to be included in the output district boundary geodatabase. The first column contains the feature class name from DCP's document in Appendix D. Feature class names ending in "wi" (which stands for "water blocks included" are the unclipped versions of district types that will also be extracted clipped to the shoreline. In those cases, the same feature class name but without the "wi" ending is the name of the same district type clipped to the shoreline.

District types for which no "wi" version is listed in Table 39 are extracted exactly as stored in the CSCL database unless otherwise noted in the "Descriptions" column in the table. (Note: Although Appendix D does not list a "wi" file for Community District, clipped and unclipped versions of this district type will be extracted, pursuant to DCP instructions.)

The template of District boundary Geo-database is at C:\NDPS\MyFiles\Geo-support-modernization\Templates\District Boundary.gdb.

Table 39 – District Boundary Features

Output Feature Class Name	Description	CSCL Source Feature Class
nyad	Assembly Districts clipped to shoreline	AssemblyDistrict
nyadwi	Assembly Districts	AssemblyDistrict
nyap	Atomic Polygons	AtomicPolygons
nybb	Borough Boundaries clipped to shoreline	Borough
nybbwi	Borough Boundaries unclipped	Borough
nybid	Business Improvement District	BusinessImprovementDistrict
nycb2010	2010 Census Blocks clipped to shoreline	CensusBlock2010 (*) See note 1
nycb2010wi	2010 Census Blocks unclipped	CensusBlock2010 (*), see note 1
nycb2020	2020 Census Blocks clipped to shoreline	CensusBlock2020 (*) See note 2
nycb2020wi	2020 Census Blocks unclipped	CensusBlock2020 (*), see note 2
nycc	City Council Districts	CityCouncilDistrict
nyccwi	City Council Districts unclipped	CityCouncilDistrict
nycd	Community Districts clipped to shoreline	CommunityDistrict
nycdwi	Community Districts unclipped	CommunityDistrict
nycdta2020		CDTA2020, see Note 5
nycg	Congressional Districts	CongressionalDistrict
nycgwi	Congressional Districts unclipped	CongressionalDistrict
nyct2010	2010 Census tracts clipped to shoreline	CensusTract2010
nyct2010wi	2010 Census tracts unclipped	CensusTract2010
nyct2020	2020 Census tracts clipped to shoreline	CensusTract2020, see Note 6
nyct2020wi	2020 Census tracts unclipped	CensusTract2020
nyed	Election Districts	ElectionDistrict
nyedwi	Election Districts unclipped	ElectionDistrict
nyfb	Fire Battalions	FireBattalion
nyfc	Fire Companies	FireCompany
nyfd	Fire Divisions	FireDivision
nyha	Health Areas clipped to shoreline	HealthArea
nyhc	Health Center Districts clipped to shoreline	HealthCenterDistrict

Output Feature Class Name	Description	CSCL Source Feature Class
nymc	Municipal Court Districts	MunicipalCourtDistrict
nymcwi	Municipal Court Districts unclipped	MunicipalCourtDistrict
nymcea	Minor Census Economic Areas clipped to shoreline	Generate by dissolving 2000 Census Tracts based on MCEA attribute, then clipping
Nynta2010	Projection Areas	Neighborhood, see Note 4
Nynta2020	Projection Areas 2020	NTA2020, see Note 3
nypp	Police Precincts	NYPDPrecinct
Nypuma2010	Puma	Census tract 2010
nysd	School Districts	SchoolDistrict (**)
nyss	State Senatorial Districts	StateSenateDistrict
nysswi	State Senatorial Districts	StateSenateDistrict
nyura	Urban Renewal Areas	UrbanRenewalArea
Nybid	Business Improvement Districts	BusinessImprovementDistrict
~~nyez~~	~~Empire Zones~~	~~EmpireZone~~
nyhd	Historic Districts	HistoricDistrict
nyhez	Hurricane Evacuation Zone	HurricaneEvacuationZone
nyzip	Zip Code Areas	ZipCode

(*) These output district types contain an 11-byte string field called BCTCB2000. These fields will be populated from the first 11 bytes of the 12-byte CensusBlock2000 attribute field BCTCB. (The last byte of the latter is the Census Block Suffix and will not be propagated to the output record.)

(**) School District 10 covers portions of two borough. A separate nysd polygon will be written out for the portion of this school district in each borough. The two borough-specific polygons can be generated by intersecting the CSCL polygon for School District 10 with each borough polygon. Both output polygons will have the same polygon ID.

~~(***) These district types are not yet implemented. They are not to be extracted until they have been implemented in CSCL.~~

Note:

1. The source feature class “CENSUSBLOCK2010 doesn’t have field of “boroName”. The attribute of BoroName will be got from field “BoroCode”.

   If boroCode == “1”, its BoroName ==”Manhattan”

   If boroCode == “2”, its BoroName ==”Bronx”

If boroCode == “3”, its BoroName ==”Brooklyn”

If boroCode == “4”, its BoroName ==”Queens”

If boroCode == “5”, its BoroName ==”Staten Island”

The attributes of “CB2010” are copied from field of “CB” of source feature class “CENSUSBLOCK2010. The attributes of “CT2010” are copied from field of “CT” of source feature class. The field “BCTCB2010” are copied from field of “BCTCB” of source feature class “CENSUSBLOCK2010.

2. The NYCB2020 and NYCB2020wi. Here are the rules to populate the field of GEOID:

   If the BoroCode ==”1” , GEOID = “36061” + BCTCB(Right 10 digit)

   If the BoroCode ==”2” , GEOID = “36005” + BCTCB(Right 10 digit)

   If the BoroCode ==”3” , GEOID = “36047” + BCTCB(Right 10 digit)

   If the BoroCode ==”4” , GEOID = “36081” + BCTCB(Right 10 digit)

   If the BoroCode ==”5” , GEOID = “36085” + BCTCB(Right 10 digit)

3. The source feature class for NyNTA2020 is CSCL.NTA2020. It only has “NEIGHBORHOOD_CODE” field. The rest of attributes need special process to retrieve them from other FC or table in CSCL.

   The field of “BoroCode”, “BoroName” and “CountyFIPS” are retrieved from CSCL.borough” feature class by using the centroid point of NTA2020 polygon to find the borough polygon containing the centroid point and populate out the corresponding attributes from it.

   The field “NTA2020” is copied from “Neighborhood_code” in CSCL.NTA2020.

   The field “NTAName”, “NTAAbbrev”, “NTAType” are retrieved from CSCL table of “NTAEquiv2020”. The three attribute will be populated from fields of “NTA_NAME”,”NTA_ABBREV” “NTA_TYPE” respectively from table “NTAEquiv2020” if the “Neighborhood_code” in NTA2020 is equal to NTA_Code” of CSCL. NTAEquiv2020 table.

   The field “CDTA2020”, “CDTAName” are retrieved from CSCL.CDTA2020 and CSCL.CDTAEquiv2020 respectively. Generate a centroid of the polygon and find the record of

CSCL.CDTA2020 that contains the centroid. The found record od CDTA2020's CDTA_Code will be copied to field of "CDTA2020" of NTA2020. The found CDTA_code will be used to find the corresponding "CDTA_name" in CSCL.CDTAEquiv2020 table. The found "CDTA_name" will be copied to the field of "CDTAName" of NTA2020.

4. Nynta2010

The attributes of "BoroCode", "BoroName" and "CountyFIPS" are retrieved from CSCL.borough" feature class by using the centroid point of NTA2010 polygon to find the borough polygon containing the centroid point and populate out the corresponding attributes from it.

NTACode attribute will be copied from "Neighborhood_code" in "Neighborhood" feature class.

NTAName attribute will be retrieved from the attribute of "Neighborhood_name" in "NEIGHBORHOODPUMACODES" table when the "NTACode" of Nynta2010 equals to "Neighborhood_code" of "NEIGHBORHOODPUMACODES" table.

5. Nycdta2020

The source feature class for Nycdta2020 is CSCL.CDTA2020.The attributes of "BoroCode", "BoroName" and "CountyFIPS" are retrieved from CSCL.borough" feature class by using the centroid point of CDTA2020 polygon to find the borough polygon containing the centroid point and populate out the corresponding attributes from it.

Attribute of "CDTA2020" is copied from "CDTA_code" in CSCL.CDTA2020.

Attribute of "CDTAName" is retrieved from attribute of "CDTA_Name" in table of CDTAEquiv2020 when CDTA_code of CDTA202 equals to the CDTA_code of CDTAEquiv2020.

Attribute of "CDTAType" is retrieved from attribute of "CDTA_Type" in table of "CDTAEquiv2020" when "CDTA_code" of "CSCL.CDTA2020" equals to the "CDTA_code" of "CDTAEquiv2020".

6. Nyct2020

The source feature class for Nyct2020 is "CSCL.CENSUSTRACT2020".

Boro_name is got from CSCL.borough in the same way as nycdta2020.

CT2020 is copied from attribute of "CT" in the source feature class.

BoroCT2020 is copied from attribute of "BoroCT" in the source feature class.

CDEligibil is copied from attribute of "CD_Eligibility" in the source feature class.

NTA2020 is copied from attribute of "Neighborhood_Code" in the source feature class.

CDTA2020 is copied from attribute of "CDTA_Code" in the source feature class.

PUMA is copied from attribute of "PUMA" in the source feature class.

??NTAName is retrieved from NTA_NAME in NTAEquiv2020 table when NTA_Code in NTAEquiv2020 equals NTA2020 in NYCT2020

??CDTAName is retrieved from CDTA_Name in CDTAEquiv2020 table when the CDTA2020 equals CDTA_code in CDTAEquiv2020 table.

GEOID is generated by replace the borough code in field of "BoroCT2020" with FIPS state and county code. It is total 11 digits.

Boro code	State and county FIPS
1	36061
2	36005
3	36047
4	36081
5	36085

APPENDIX A BYTES OF THE BIG APPLE™ LION IN ESRI FORMAT

The extraction of the ESRI LION is discussed in Section 2.7 and its sub-sections. This appendix contains a copy of an updated document on the ESRI LION provided by the Department of City Planning on April 20, 2009 and identified as V4. The document was produced using the ESRI ARCGIS Diagrammer tool.

ArcGIS Diagrammer

ETL LION fgdb V4.doc

Report Creation	
Date	Wenesday, January 28, 2009 **Modified 4/20/09**
Author	m_mcInne/DCP-DELL-1129 on DCP-DELL-1129
System Information	
Operating System	Microsoft Windows NT 5.1.2600 Service Pack 2
.Net Framework	2.0.50727.832
Diagrammer	1.0.3111.23173
Geodatabase	
Workspace Type	File Geodatabase

Table Of Contents

Domains	*Listing of Coded Value and Range Domains.*
ObjectClasses	*Listing of Tables and FeatureClasses.*
Spatial Reference	*Listing of Spatial References used by FeatureClasses and FeatureDatasets.*

Back to Top

Domains

Domain Name	Owner	Domain Type
Borough		Coded Value

Back to Top

Borough

Owner	
Description	New York City Borough Name
Domain Type	Coded Value
Field Type	Integer
Merge Policy	Default Value

Split Policy	Default Value
Domain Members	
Name	**Value**
Manhattan	1
Bronx	2
Brooklyn	3
Queens	4
Staten Island	5

Back to Top

ObjectClasses

ObjectClass Name	Type	Geometry	Subtype	
Stand Alone ObjectClass(s)				
altnames	Table	-	-	
lion	Simple FeatureClass	Polyline	-	SR

Back to Top

altnames

Alias	altnames
Dataset Type	Table
FeatureType	

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	ObjectID	ObjectID	OID	0	0	4	No
PDir	PreDir	ADDR_PD	String	0	0	2	Yes
PType	PreType	ADDR_PT	String	0	0	6	Yes
SName	StreetName	ADDR_SN	String	0	0	30	Yes
SType	SufType	ADDR_ST	String	0	0	6	Yes
SDir	SufDir	ADDR_SD	String	0	0	2	Yes
Street	Street		String	0	0	32	Yes
Join_ID	Join_ID		String	0	0	15	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_Join_ID	Yes	No	Join_ID
OBJECTID$$Index	Yes	No	OBJECTID

Back to Top

Lion (also see the template of Byte Lion geodatabase for the latest version:

C:\NDPS\MyFiles\Geo-support-modernization\Templates\lion.gdb)

Alias	lion	**Geometry:**Polyline
Dataset Type	FeatureClass	**Average Number of Points:**0
FeatureType	Simple	**Has M:**No
		Has Z: No
		Grid Size:730 ,5840

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null

OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
Street	Street		String	0	0	32	Yes
FeatureTyp	FeatureTyp		String	0	0	1	Yes
SegmentTyp	SegmentTyp		String	0	0	1	Yes
IncExFlag	IncExFlag		String	0	0	1	Yes
RB_Layer	RB_Layer		String	0	0	1	Yes
NonPed	NonPed		String	0	0	1	Yes
TrafDir	TrafDir		String	0	0	1	Yes
TrafSrc	TrafSrc		String	0	0	3	Yes
SpecAddr	SpecAddr		String	0	0	1	Yes
FaceCode	FaceCode		String	0	0	4	Yes
SeqNum	SeqNum		String	0	0	5	Yes
StreetCode	StreetCode		String	0	0	6	Yes
LGC1	LGC1		String	0	0	2	Yes
LGC2	LGC2		String	0	0	2	Yes
LGC3	LGC3		String	0	0	2	Yes
LGC4	LGC4		String	0	0	2	Yes
BOE_LGC	BOE_LGC		String	0	0	1	Yes
SegmentID	SegmentID		String	0	0	7	Yes
SegCount	SegCount		String	0	0	1	Yes
LocStatus	LocStatus		String	0	0	1	Yes
LZip	LZip		String	0	0	5	Yes
RZip	RZip		String	0	0	5	Yes
LBoro	LBoro	LBoro	Integer	0	0	4	Yes
RBoro	RBoro	RBoro	Integer	0	0	4	Yes
L_CD	L_CD		String	0	0	3	Yes
R_CD	R_CD		String	0	0	3	Yes
LATOMICPOLYGON	RATOMICPOLYGON		String	0	0	3	Yes
RATOMICPOLYGON	RATOMICPOLYGON		String	0	0	3	Yes
LCT2000	LCT2000		String	0	0	4	Yes
LCT2000Suf	LCT2000Suf		String	0	0	2	Yes
RCT2000	RCT2000		String	0	0	4	Yes
RCT2000Suf	RCT2000Suf		String	0	0	2	Yes
LCB2000	LCB2000		String	0	0	4	Yes

LCB2000Suf	LCB2000Suf	String	0	0	1	Yes
RCB2000	RCB2000	String	0	0	4	Yes
RCB2000Suf	RCB2000Suf	String	0	0	1	Yes
LCT1990	LCT1990	String	0	0	4	Yes
LCT1990Suf	LCT1990Suf	String	0	0	2	Yes
RCT1990	RCT1990	String	0	0	4	Yes
RCT1990Suf	RCT1990Suf	String	0	0	2	Yes
LAssmDist	LAssmDist	String	0	0	2	Yes
LElectDist	LElectDist	String	0	0	3	Yes
RAssmDist	RAssmDist	String	0	0	2	Yes
RElectDist	RElectDist	String	0	0	3	Yes
SplitElect	SplitElect	String	0	0	1	Yes
LSchlDist	LSchlDist	String	0	0	2	Yes
RSchlDist	RSchlDist	String	0	0	2	Yes
LSubSect	LSubSect	String	0	0	2	Yes
RSubSect	RSubSect	String	0	0	2	Yes
SanDistInd	SanDistInd	String	0	0	1	Yes
MapFrom	MapFrom	String	0	0	3	Yes
MapTo	MapTo	String	0	0	3	Yes
BoroBndry	BoroBndry	String	0	0	1	Yes
MH_RI_Flag	MH_RI_Flag	String	0	0	1	Yes
XFrom	XFrom	Integer	0	0	4	Yes
YFrom	YFrom	Integer	0	0	4	Yes
XTo	XTo	Integer	0	0	4	Yes
YTo	YTo	Integer	0	0	4	Yes
ArcCenterX	ArcCenterX	Integer	0	0	4	Yes
ArcCenterY	ArcCenterY	Integer	0	0	4	Yes
CurveFlag	CurveFlag	String	0	0	1	Yes
Radius	Radius	Integer	0	0	4	Yes
NodeIDFrom	NodeIDFrom	String	0	0	7	Yes
NodeIDTo	NodeIDTo	String	0	0	7	Yes
NodeLevelF	NodeLevelF	String	0	0	1	Yes
NodeLevelT	NodeLevelT	String	0	0	1	Yes

ConParity	ConParity	String	0	0	1	Yes
Twisted	Twisted	String	0	0	1	Yes
LLo_Hyphen	LLo_Hyphen	String	0	0	7	Yes
LHi_Hyphen	LHi_Hyphen	String	0	0	7	Yes
RLo_Hyphen	RLo_Hyphen	String	0	0	7	Yes
RHi_Hyphen	RHi_Hyphen	String	0	0	7	Yes
FromLeft	FromLeft	Integer	0	0	4	Yes
ToLeft	ToLeft	Integer	0	0	4	Yes
FromRight	FromRight	Integer	0	0	4	Yes
ToRight	ToRight	Integer	0	0	4	Yes
Join_ID	Join_ID	String	0	0	15	Yes
SHAPE_Length						
Additional CSCL Fields to be included:						
ROADWAY TYPE (RW_TYPE)						
PHYSICAL ID						
GENERIC ID						
NYPDID		Double			8	Yes
FDNYID	SHAPE_Length		0	0		
LEFT BLOCKFACE ID						
RIGHT BLOCKFACE ID						
STATUS						
STREETWIDTH						
STREETWIDTH_IRR						
BIKELANE						
FCC						
RIGHT OF WAY TYPE (ROW_TYPE)						
LEFT 2010 CENSUS TRACT BASIC						
LEFT 2010 CENSUS TRACT SUF						
RIGHT 2010 CENSUS TRACT BASIC						
RIGHT 2010 CENSUS TRACT SUF						
LGC 5						
LGC 6						
LGC 7						
LGC 8						
LGC 9						
LEGACY SEGMENT ID						
SAF Streetname						
SAF Streetcode						

Subtype Name	Default Value	Domain
ObjectClass		
LBoro		Borough
RBoro		Borough

Index Name	Ascending	Unique	Fields
OBJECTID$$Index	Yes	No	OBJECTID

Back to Top

Spatial References

Dimension	Minimum	Precision
lion		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-

Coordinate System Description
PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]

Back to Top

ArcGIS Diagrammer is prototype application and is not supported by ESRI. The commands assoicated with this application and the output generated by those commands are not to be used in a production environment. ESRI is not reponsible for errors, ommission or any damages resulting from the use of application and associated output. Use of this application is conditional on the acceptance of this statement.

(Update April, 2012. “Snow priority” is added. Locstat Value of X is based on 2010 CT instead of 2000 CT)

APPENDIX C BYTES LION: POPULATING "EMPTY" FIELDS

This appendix contains a DCP-supplied table indicating how to populate fields in the ESRI LION and the MapInfo LION when the field is "empty" (devoid of meaningful contents). For fields for which the last column does not contain a notation, a meaningful value is mandatory.

Field	ESRI GDB Type	ESRI Len	MapInfo Format	When no value…
OBJECTID	Object ID	4		
SHAPE	Line			
Street	Text	32	Char(32)	
FeatureTyp	Text	1	Char(1)	
SegmentTyp	Text	1	Char(1)	
IncExFlag	Text	1	Char(1)	" " (a blank)
RB_Layer	Text	1	Char(1)	
NonPed	Text	1	Char(1)	" " (a blank)
TrafDir	Text	1	Char(1)	" " (a blank)
TrafSrc	Text	3	Char(3)	
SpecAddr	Text	1	Char(1)	" " (a blank)
FaceCode	Text	4	Char(4)	
SeqNum	Text	5	Char(5)	
StreetCode	Text	6	Char(6)	
LGC1	Text	2	Char(2)	" " (2 blanks)
LGC2	Text	2	Char(2)	" " (2 blanks)
LGC3	Text	2	Char(2)	" " (2 blanks)
LGC4	Text	2	Char(2)	" " (2 blanks)
BOE_LGC	Text	1	Char(1)	" " (a blank)
SegmentID	Text	7	Char(7)	
SegCount	Text	1	Char(1)	
LocStatus	Text	1	Char(1)	" " (a blank)
LZip	Text	5	Char(5)	"" NULL
RZip	Text	5	Char(5)	"" NULL
LBoro	Long	4	Integer	"" NULL
RBoro	Long	4	Integer	"" NULL
L_CD	Text	3	Char(3)	" " (a blank)
R_CD	Text	3	Char(3)	" " (a blank)
LDynBlock	Text	3	Char(3)	"" NULL
RDynBlock	Text	3	Char(3)	"" NULL
LCT2000	Text	4	Char(4)	"" NULL
LCT2000Suf	Text	2	Char(2)	" " (2 Blanks) if LCT2000 is not NULL; NULL if LCT2000 is NULL
RCT2000	Text	4	Char(4)	"" NULL
RCT2000Suf	Text	2	Char(2)	" " (2 Blanks) if RCT2000 is not NULL; NULL if RCT2000 is NULL
LCB2000	Text	4	Char(4)	"" NULL

Field	ESRI GDB Type	ESRI Len	MapInfo Format	When no value…
LCB2000Suf	Text	1	Char(1)	" " (blank) if LCB2000 is not NULL; NULL if LCB2000 is NULL
RCB2000	Text	4	Char(4)	"" NULL
RCB2000Suf	Text	1	Char(1)	" " (a blank) if RCB2000 is not NULL; NULL if RCB2000 is NULL
LCT1990	Text	4	Char(4)	" " (4 Blanks)
LCT1990Suf	Text	2	Char(2)	" " (2 Blanks)
RCT1990	Text	4	Char(4)	" " (4 Blanks)
RCT1990Suf	Text	2	Char(2)	" " (2 Blanks)
LAssmDist	Text	2	Char(2)	"" NULL
LElectDist	Text	3	Char(3)	"" NULL
RAssmDist	Text	2	Char(2)	"" NULL
RElectDist	Text	3	Char(3)	"" NULL
SplitElect	Text	1	Char(1)	" " (a blank)
LSchlDist	Text	2	Char(2)	"" NULL
RSchlDist	Text	2	Char(2)	"" NULL
SplitSchl	Text	1	Char(1)	" " (a blank)
SchlCount	Long	4	Integer	0 (Zero)
LSubSect	Text	2	Char(2)	" " (2 blanks)
RSubSect	Text	2	Char(2)	" " (2 blanks)
SanDistInd	Text	1	Char(1)	" " (a blank)
MapFrom	Text	3	Char(3)	
MapTo	Text	3	Char(3)	
NearBQ	Text	1	Char(1)	" " (a blank)
BoroBndry	Text	1	Char(1)	" " (a blank)
MH_RI_Flag	Text	1	Char(1)	" " (a blank)
XFrom	Long	4	Integer	
YFrom	Long	4	Integer	
XTo	Long	4	Integer	
YTo	Long	4	Integer	
ArcCenterX	Long	4	Integer	0 (Zero)
ArcCenterY	Long	4	Integer	0 (Zero)
CurveFlag	Text	1	Char(1)	" " (a blank)
Radius	Long	4	Integer	0 (Zero)
NodeIDFrom	Text	7	Char(7)	
NodeIDTo	Text	7	Char(7)	
NodeLevelF	Text	1	Char(1)	
NodeLevelT	Text	1	Char(1)	
ConParity	Text	1	Char(1)	" " (a blank)
Twisted	Text	1	Char(1)	" " (a blank)
LLo_Hyphen	Text	7	Char(7)	"" NULL
LHi_Hyphen	Text	7	Char(7)	"" NULL
RLo_Hyphen	Text	7	Char(7)	"" NULL
RHi_Hyphen	Text	7	Char(7)	"" NULL
FromLeft	Long	4	Integer	0 (Zero)
ToLeft	Long	4	Integer	0 (Zero)
FromRight	Long	4	Integer	0 (Zero)

Field	ESRI GDB Type	ESRI Len	MapInfo Format	When no value…
ToRight	Long	4	Integer	0 (Zero)
Join_ID	Text	15	Char(15)	
ShapeLength	Double	8		

APPENDIX D DISTRICT BOUNDARY GEODATABASE

The extraction of an ESRI file geodatabase of district boundaries is discussed in Chapter 10. This appendix contains a copy of a DCP document that lists the feature classes, attributes, and spatial references for that geodatabase. This is Version 4 of this DCP document, dated January 29, 2009. It was produced using the ESRI ARCGIS Diagrammer tool. (Note that the DCP document is missing information on attribute fields required for the district types listed under "Additional CSCL Polygons" in the list of object classes. It is assumed that the only attribute information required is the district identifiers.)

(Also see the template of Byte District boundary for the latest version:

C:\NDPS\MyFiles\Geo-support-modernization\Templates\District Boundary.gdb)

ArcGIS Diagrammer

Report Creation	
Date	Thursday, January 22, 2009 Modified 4/29/09
Author	m_mcInne/DCP-DELL-1129 on DCP-DELL-1129
System Information	
Operating System	Microsoft Windows NT 5.1.2600 Service Pack 2
.Net Framework	2.0.50727.832
Diagrammer	1.0.3111.23173
Geodatabase	
Workspace Type	File Geodatabase

Table Of Contents

ObjectClasses *Listing of Tables and FeatureClasses.*

Spatial Reference *Listing of Spatial References used by FeatureClasses and FeatureDatasets.*

Back to Top

ObjectClasses

ObjectClass Name	Type	Geometry	Subtype	
Stand Alone ObjectClass(s)				
nyad	Simple FeatureClass	Polygon	-	SR
nybb	Simple FeatureClass	Polygon	-	SR
nybbwi	Simple FeatureClass	Polygon	-	SR

nycb2000	Simple FeatureClass	Polygon	-	SR
nycb2000wi	Simple FeatureClass	Polygon	-	SR
nycc	Simple FeatureClass	Polygon	-	SR
nycd	Simple FeatureClass	Polygon	-	SR
nycg	Simple FeatureClass	Polygon	-	SR
nyct2000	Simple FeatureClass	Polygon	-	SR
nyct2000wi	Simple FeatureClass	Polygon	-	SR
nyed	Simple FeatureClass	Polygon	-	SR
nyfb	Simple FeatureClass	Polygon	-	SR
nyfc	Simple FeatureClass	Polygon	-	SR
nyfd	Simple FeatureClass	Polygon	-	SR
nyha	Simple FeatureClass	Polygon	-	SR
nyhc	Simple FeatureClass	Polygon	-	SR
nymc	Simple FeatureClass	Polygon	-	SR
nypa	Simple FeatureClass	Polygon	-	SR
nypp	Simple FeatureClass	Polygon	-	SR
nysd	Simple FeatureClass	Polygon	-	SR
nyss	Simple FeatureClass	Polygon	-	SR
Additional CSCL Polygons: nymcea UrbanRenewal Area BusinessImprovementDistrict EmpireZone HistoricDistrict ZipCode nyct2010 nyct2010wi nycb2010 nycb2010wi				

Back to Top

nyad

Alias nyad
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:40000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
AssemDist			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nybb

Alias nybb
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:95000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
BoroCode			Small Integer	0	0	2	Yes
BoroName			String	0	0	32	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nybbwi

Alias nybbwi
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:110000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
BoroCode			Small Integer	0	0	2	Yes
BoroName			String	0	0	32	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nycb2000

Alias nycb2000
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:1100 ,8800

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
CB2000			String	0	0	4	Yes
BoroCode			String	0	0	1	Yes
BoroName			String	0	0	32	Yes
CT2000			String	0	0	6	Yes
BCTCB2000			String	0	0	11	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nycb2000wi

Alias nycb2000wi
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:1300 ,15600

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
CB2000			String	0	0	4	Yes
BoroCode			String	0	0	1	Yes
BoroName			String	0	0	32	Yes
CT2000			String	0	0	6	Yes
BCTCB2000			String	0	0	11	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nycc

Alias nycc
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:41000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
CounDist			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nycd

Alias nycd
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:41000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
BoroCD			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nycg

Alias nycg
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:81000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
CongDist			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyct2000

Alias nyct2000
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:5800

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
CTLabel			String	0	0	7	Yes
BoroCode			String	0	0	1	Yes
BoroName			String	0	0	32	Yes
CT2000			String	0	0	6	Yes
BoroCT2000			String	0	0	7	Yes
CDEligibil			String	0	0	1	Yes
PjAreaCode			String	0	0	4	Yes
PjAreaName			String	0	0	40	Yes
PUMA			String	0	0	4	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyct2000wi

Alias nyct2000wi
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:6900

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
CTLabel			String	0	0	7	Yes
BoroCode			String	0	0	1	Yes

BoroName			String	0	0	32	Yes
CT2000			String	0	0	6	Yes
BoroCT2000			String	0	0	7	Yes
CDEligibil			String	0	0	1	Yes
PjAreaCode			String	0	0	4	Yes
PjAreaName			String	0	0	40	Yes
PUMA			String	0	0	4	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes
Subtype Name	**Default Value**		**Domain**				

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyed

Alias nyed
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:3400

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
ElectDist			Integer	0	0	4	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes
Subtype Name	**Default Value**		**Domain**				

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyfb

Alias nyfb
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:43000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
FireBN			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes
Subtype Name	**Default Value**		**Domain**				

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyfc

Alias nyfc
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:13000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
FireCoType			String	0	0	1	Yes
FireCoNum			Small Integer	0	0	2	Yes
FireBN			Small Integer	0	0	2	Yes
FireDiv			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyfd

Alias nyfd
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:96000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
FireDiv			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyha

Alias nyha
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:15000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
BoroCode			Small Integer	0	0	2	Yes
BoroName			String	0	0	32	Yes
HealthArea			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyhc

Alias nyhc
Dataset Type FeatureClass

Geometry: Polygon
Average Number of Points:0

FeatureType Simple

Has M: No
Has Z: No
Grid Size:57000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
BoroCode			Small Integer	0	0	2	Yes
BoroName			String	0	0	32	Yes
HCentDist			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nymc

Alias nymc
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:57000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
BoroCode			Small Integer	0	0	2	Yes
BoroName			String	0	0	32	Yes
MuniCourt			String	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nypa

Alias nypa
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:23000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
BoroCode			Small Integer	0	0	2	Yes
BoroName			String	0	0	13	Yes
CountyFIPS			String	0	0	3	Yes
PjAreaCode			String	0	0	4	Yes
PjAreaName			String	0	0	40	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nypp

Alias nypp
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:38000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
Precinct			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nysd

Alias nysd
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:39000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
SchoolDist			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

nyss

Alias nyss
Dataset Type FeatureClass
FeatureType Simple

Geometry: Polygon
Average Number of Points:0
Has M: No
Has Z: No
Grid Size:53000

Field Name	Alias Name	Model Name	Type	Precn.	Scale	Length	Null
OBJECTID	OBJECTID	OBJECTID	OID	0	0	4	No
SHAPE	SHAPE	SHAPE	Geometry	0	0	0	Yes
StSenDist			Small Integer	0	0	2	Yes
SHAPE_Length	SHAPE_Length	SHAPE_Length	Double	0	0	8	Yes
SHAPE_Area	SHAPE_Area	SHAPE_Area	Double	0	0	8	Yes

Subtype Name	Default Value	Domain

Index Name	Ascending	Unique	Fields
FDO_OBJECTID	Yes	Yes	OBJECTID
FDO_SHAPE	Yes	No	SHAPE

Back to Top

Spatial References

Dimension	Minimum	Precision
nyad		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nybb		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nybbwi		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nycb2000		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nycb2000wi		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTI		

ON["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]

nycc

X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-

Coordinate System Description
PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]

nycd

X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-

Coordinate System Description
PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]

nycg

X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-

Coordinate System Description
PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]

nyct2000

X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-

Coordinate System Description
PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]

nyct2000wi

X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-

Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nyed		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nyfb		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nyfc		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nyfd		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nyha		
X	-120039300	10000
Y	-96540300	
M	-	-

Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nyhc		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nymc		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nypa		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nypp		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nysd		
X	-120039300	10000
Y	-96540300	

M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		
nyss		
X	-120039300	10000
Y	-96540300	
M	-	-
Z	-	-
Coordinate System Description PROJCS["NAD_1983_StatePlane_New_York_Long_Island_FIPS_3104_Feet",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.257222101]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",984250.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-74.0],PARAMETER["Standard_Parallel_1",40.66666666666666],PARAMETER["Standard_Parallel_2",41.03333333333333],PARAMETER["Latitude_Of_Origin",40.16666666666666],UNIT["Foot_US",0.3048006096012192]]		

Back to Top

ArcGIS Diagrammer is prototype application and is not supported by ESRI. The commands assoicated with this application and the output generated by those commands are not to be used in a production environment. ESRI is not reponsible for errors, ommission or any damages resulting from the use of application and associated output. Use of this application is conditional on the acceptance of this statement.

APPENDIX E FILE NAMING REQUIREMENTS

This appendix contains a table of ETL output file naming requirements provided by DCP on 5/19/2009 and identified as Version 3.

File Name	Naming Convention	Comment
LION	*lion.dat	*=Manhattan, Bronx, Brooklyn, Queens, or StatenIsland
ThinED	thinED.dat	not by borough
Face Code	*Face.txt	*=Manhattan, Bronx, Brooklyn, Queens, or StatenIsland
LDF	LDFBASE.dat	
LDF header file	LDFHEADER.dat	
RPL	RPL.txt	
ThinFire	*thinfire.txt	*=Manhattan, Bronx, Brooklyn, Queens, or StatenIsland
SAF	Generic*.txt	*=**ABCEGPNX**,**D**,**OV**, or **S**
	Roadbed*.txt	*=**ABCEGPNX**,**D**,**OV**, or **S**
	Intersection.txt	
Sedat	Sedat.txt	The ETL file needed for processing is a single Seda file that includes Sedat records only
Special Sedat	specialSEDAT.txt	The ETL file needed for processing is a single special sedat file that includes special Sedat records only
Split School	N/A	There no longer is a split school file
ThinTract	*thintract.dat	*=Manhattan, Bronx, Brooklyn, Queens, or StatenIsland
ThinLION	*thinlion.dat	*=Manhattan, Bronx, Brooklyn, Queens, or StatenIsland
SND	SND.txt	
Exception Table	Exception.txt	
Universal Table	Universal.txt	
Enders Table	Enders.txt	
Last Word Table	LastWord.txt	
BYTES LION (ESRI fgdb)	v*_LION	*=cycle name (24A) v24A_LION.gdb
~~BYTES LION (MapInfo)~~	~~NYLION1.mif and NYLION2.mif~~	
BYTES Districts (ESRI fgdb)	v*_DISTRICTS Feature Class names as per the ArcGIS diagrammer ETL document	*= cycle name (24A) v24A_DISTRICTS (assume tha a user would have to enter the cycle information when running the extract). The ArcGIS Team has requested one fgdb containing all the districts for our internal SDE. They will break out into Shapefile format for the website)